

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева (Самарский университет)»

Институт _____ информатики и кибернетики _____

Кафедра _____ программных систем _____

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

_____ к курсовому проекту по дисциплине «Программная инженерия»
_____ по теме «Автоматизированная система «Игра «Puzzle»
_____ с функциями администратора» _____

Обучающийся _____ 29.12.2021 В.Д. Гижевская

Обучающийся _____ 29.12.2021 О.В. Николаев

Руководитель _____ 29.12.2021 Л.С. Зеленко

Самара 2021

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева (Самарский университет)»

Институт _____ информатики и кибернетики
Кафедра _____ программных систем

ЗАДАНИЕ

на курсовой проект по дисциплине
«Программная инженерия»
обучающимся в группе № 6413-020302D
В.Д. Гижевской
О.В. Николаеву

- 1 Тема проекта: «Автоматизированная система «Игра «Puzzle» с функциями администратора»
- 2 Исходные данные к проекту: см. приложение к заданию
- 3 Перечень вопросов, подлежащих разработке:
 - 3.1 Произвести анализ предметной области: изучить правила сборки Игры «Puzzle»
 - 3.2 Выполнить обзор существующих систем-аналогов
 - 3.3 Разработать информационно-логический проект системы по методологии UML
 - 3.4 Разработать и реализовать программное и информационное обеспечение, провести его тестирование и отладку
 - 3.5 Оформить документацию курсового проекта
 - 3.6 Подготовить презентацию по разработанной системе
- 4 Перечень графических разработок:
 - 4.1 Структурная схема системы
 - 4.2 Канонические диаграммы UML
 - 4.3 Схемы основных алгоритмов

5 Календарный план выполнения работ

№ п/ п	Содержание работы по этапам	Объем этапа в % к общему объему проекта	Срок окончания	Фактическое выполнение
1	Оформление технического задания и его утверждение	5	20.09.2021	
2	Описание и анализ предметной области	10	06.10.2021	
3	Проектирование системы	40	01.12.2021	
3.1	Разработка структурной схемы системы	5	13.10.2021	
3.2	Разработка функциональной спецификации системы и прототипа интерфейса пользователя	10	27.10.2021	
3.3	Разработка информационно-логического проекта системы и его предъявление руководителю	20	01.12.2021	
4	Реализация проекта, разработка контрольных примеров. Предъявление реализации руководителю	40	15.12.2021	
5	Корректировка проекта и оформление документации проекта. Защита проекта с представлением презентации.	10	29.12.2021	

Задание принял

к исполнению _____

13.09.2021 В.Д. Гижевская

13.09.2021 О.В. Николаев

ПРИЛОЖЕНИЕ

к заданию на курсовой проект
обучающимся в группе № 6413-020302D

В.Д. Гижевской

О.В. Николаеву

Тема проекта: «Автоматизированная система «Игра «Puzzle»
с функциями администратора»

Исходные данные к проекту:

1 Характеристика объекта автоматизации:

- 1) объект автоматизации: Игра «Puzzle»;
- 2) виды автоматизируемой деятельности:
 - процесс авторизации пользователей;
 - процесс регистрации пользователей;
 - процесс генерирования игры;
 - процесс составления игры;
 - процесс приведения картинки к формату 4x3;
 - процесс работы с галереей картинок;
 - процесс визуализации сбора игры «Puzzle»;
- 3) количество ролей пользователей – 2;
- 4) минимальная длина логина – 4 символа;
- 5) максимальная длина логина – 8 символов;
- 6) минимальная длина пароля – 4 символа;
- 7) максимальная длина пароля – 10 символов;
- 8) минимальное количество разбиений по горизонтали – 4;
- 9) максимальное количество разбиений по горизонтали – 8;
- 10) минимальное количество разбиений по вертикали – 4;
- 11) максимальное количество разбиений по вертикали – 12;
- 12) количество уровней сложности – 5;
- 13) количество видов деталей – 3;
- 14) количество способов подсчёта результатов – 1;

- 15) количество лучших результатов – 10;
- 16) количество видов сборки картинки – 3;
- 17) количество видов сортировки галереи картинок – 1;
- 18) минимальное количество подсказок – 1;
- 19) минимальное количество категорий – 4;
- 20) максимальное количество категорий – 8;
- 21) максимальное количество подсказок на 1 уровне сложности – 1;
- 22) максимальное количество подсказок на 2 уровне сложности – 2;
- 23) максимальное количество подсказок на 3 уровне сложности – 3;
- 24) максимальное количество подсказок на 4 уровне сложности – 4;
- 25) максимальное количество подсказок на 5 уровне сложности – 5.

2 Требования к информационному обеспечению:

- 1) информационное обеспечение разрабатывается на основе следующих источников:

- Пазл [Электронный ресурс] // Википедия: [сайт]. URL: <https://ru.wikipedia.org/wiki/Пазл> (дата обращения: 18.09.2021).

- 2) внешние картинки хранятся в файлах формата *.png;
- 3) игры хранятся в файлах, структура файла определяется в процессе проектирования;
- 4) музыкальное сопровождение хранится в файлах формата *.mp3
- 5) структура базы данных разрабатывается на основании следующих сведений:

- об игроке (логин, пароль);
- о картинке (название картинки, картинка, уровень сложности, категория);
- рейтинг (логин игрока, количество баллов);
- уровень сложности (форма деталей, количество разбиений, вариант сборки);
- об игре (логин игрока, статус игры, время, название картинки);

- 6) должна быть обеспечена целостность базы данных и защита от

несанкционированного доступа.

3 Требования к техническому обеспечению:

3.1 Требования к техническому обеспечению серверной части:

- 1) тип ЭВМ – IBM PC совместимый;
- 2) объем ОЗУ – не менее 500 Мб;
- 3) объем свободного пространства на внешнем диске – не менее 20 Гб;
- 4) наличие подключения к сети Интернет;
- 5) манипулятор – мышь, клавиатура;
- 6) технические характеристики определяются в процессе выполнения проекта.

3.2 Требования к техническому обеспечению клиентской части:

- 1) тип ЭВМ – IBM PC совместимый;
- 1) монитор с разрешающей способностью не ниже 800 x 600;
- 2) манипулятор – мышь, клавиатура;
- 3) технические характеристики определяются в процессе выполнения проекта.

4 Требования к программному обеспечению:

4.1 Требования к программному обеспечению серверной части:

- 1) тип операционной системы – Windows 7 и выше;
- 2) СУБД – MySQL 10.

4.2 Требования к программному обеспечению клиентской части:

- 1) тип операционной системы – Windows 7 и выше;

4.3 Требования к программному обеспечению рабочего места разработчика:

- 1) тип операционной системы – Windows 7 и выше;
- 2) язык программирования – C#;
- 3) среда программирования – Visual Studio 2019;
- 4) СУБД – MySQL 10;
- 5) среда проектирования – StarUML 4.0.1.

5 Общие требования к проектируемой системе:

5.1 Функции, реализуемые системой:

1) общесистемные функции:

- авторизация пользователя в системе (ввод логина/пароля);
- регистрация пользователя в системе (ввод логина, пароля);
- аутентификация пользователя в системе, настройка интерфейса пользователя на заданную роль;
- автоматическое составление игры по заданным параметрам;
- визуализация процессов работы с пазлом;
- контроль количества взятых подсказок;
- проверка правильности сбора пазла;
- формирование рейтинга по заданному критерию;
- выдача справочной информации о системе;

2) функции администратора:

- настройка уровня сложности для картинки:
 - 1 задание размеров;
 - 2 выбор формы деталей;
 - 3 выбор типа сборки;
- работа с галереей:
 - 1 добавление картинки (загрузка картинки из файла);
 - 2 удаление картинки;
 - 3 сортировка галереи по категориям;
 - 4 сохранение картинки в галерею;
 - 5 навигация по галерее;

3) функции игрока:

- выбор уровня сложности;
- выбор игры (сохранённая игра; новая игра);
- дополнительные настройки:
 - 1 включение/выключение звукового сопровождения;

- 2 включение/выключение музыкальное сопровождение;
- разгадывание игры с организацией системы подсказок:
 - 3 выбор детали;
 - 4 перемещение детали на нужное место;
 - 5 взятие подсказки;
 - 6 просмотр картинки;
- сохранение игры;
- просмотр рейтинга.

5.2 Технические требования к системе:

- 1) режим работы - диалоговый;
- 2) время автоматической генерации пазла – не более 3 с;
- 3) система должна удовлетворять санитарным правилам и нормам СанПин 2.2.2./2.4.2198-07;
- 4) условия работы средств вычислительной техники (содержание вредных веществ, пыли и подвижность воздуха) должны соответствовать ГОСТ 12.1.005, 12.01.007;
- 5) температура окружающего воздуха – 15-35°C;
- 6) влажность воздуха – 45-75%.

Руководитель

проекта _____ 06.10.2021 _____ Л.С. Зеленко

Задание принял

к исполнению _____ 13.09.2021 _____ В.Д. Гижевская

_____ 13.09.2021 _____ О.В. Николев

РЕФЕРАТ

Пояснительная записка 145 с, 80 рисунков, 14 таблиц, 45 источников, 2 приложения.

Графическая часть: 23 слайда презентации PowerPoint.

ПАЗЛ, ИГРА, ФРАГМЕНТ, УРОВЕНЬ СЛОЖНОСТИ, КАРТИНКА, ГАЛЕРЕЯ, ПОДСКАЗКА, КАТЕГОРИЯ.

Во время курсового проектирования разработаны алгоритмы и соответствующая им программа, предназначенная для создания и сбора пазла, с помощью которой игрок сможет собирать пазлы разного уровня сложности, созданные администратором, а также использовать систему подсказок и просматривать рейтинг. В системе имеется возможность сохранения игр с целью последующей их сборки.

Программа написана на языке C# в среде Visual Studio 2022 и функционирует под управлением операционной системы Windows 7 и выше. Доступ к данным осуществляется с помощью СУБД MySQL 8.0.

СОДЕРЖАНИЕ

Введение.....	12
1 Описание и анализ предметной области.....	14
1.1 Описание игры «Puzzle».....	14
1.2 Описание систем-аналогов.....	18
1.2.1 Система «Пазл – ДИКИЕ ЖИВОТНЫЕ».....	18
1.2.2 Система «Сказочный пазл».....	20
1.3 Диаграмма объектов предметной области	23
1.4 Постановка задачи	24
2 Проектирование системы	28
2.1 Выбор и обоснование архитектуры системы	28
2.1.1 Виды архитектур	28
2.1.2 Клиент-серверные архитектуры	31
2.1.3 Типы клиентов.....	34
2.1.4 Протоколы передачи данных.....	35
2.2 Структурная схема системы	36
2.3 Разработка спецификации требований	39
2.3.5 Функциональная спецификация	39
2.3.6 Перечень исключительных ситуаций	47
2.4 Разработка прототипа интерфейса пользователя системы.....	48
2.5 Разработка информационно-логического проекта системы.....	58
2.5.1 Язык UML	63
2.5.2 Диаграмма вариантов использования	63
2.5.3 Сценарии	67
2.5.4 Диаграмма состояний	73
2.5.5 Диаграмма деятельности.....	78
2.5.6 Диаграмма последовательности	82
2.5.7 Диаграмма классов.....	84
2.6 Логическая модель данных	86
2.7 Разработка и описание алгоритмов обработки данных	89

2.8	Выбор и обоснование комплекса программных средств	94
2.8.1	Выбор языка программирования	94
2.8.2	Выбор операционной системы	95
2.8.3	Выбор среды программирования	95
2.8.4	Выбор системы управления базами данных	96
3	Реализация системы	98
3.1	Разработка и описание интерфейса пользователя	98
3.2	Диаграммы реализации	108
3.2.1	Диаграмма компонентов	108
3.2.2	Диаграмма развертывания.....	111
3.2.3	Диаграмма классов.....	112
3.3	Физическая модель данных.....	113
3.4	Выбор и обоснование комплекса технических средств.....	114
3.4.1	Расчёт объёма занимаемой памяти клиентской части системы.....	114
3.4.2	Минимальные требования, предъявляемые к системе.....	116
	Заключение	118
	Список использованных источников	119
	Приложение А Руководство пользователя	124
A.1	Назначение системы	124
A.2	Условия работы системы	124
A.3	Установка системы	125
A.4	Работа с системой	125
A.4.1	Работа с системой в режиме администратора.....	126
A.4.2	Работа с системой в режиме пользователя	128
	Приложение Б Листинг элемента управления GameController	135

ВВЕДЕНИЕ

Пазл (или мозаика) – это игра-головоломка, в которой необходимо собрать одну картинку из множества фрагментов различной формы. Слово "пазл" переводится с английского как "загадка" или "головоломка". Точных сведений, где и когда именно появились первые пазлы, нет [1].

Существует множество версий пазлов, самая распространённая из них – в 1760 году издатель, гравёр и картограф Джон Спилсбери придумал увлекательную головоломку. Предварительно наклеенную на деревянную основу чёрно-белую карту он распилил на несколько кусков по линиям государственных границ. В качестве развлечения было предложено собрать карту заново. Изначально пазлы использовали как обучающее пособие. До конца 19 века такая головоломка была только на деревянной основе. Позже её заменили на более дешёвый материал – картон. Только в начале 20 века в США был открыто первое фабричное производство пазлов. Игра настолько понравилась людям, что началось массовое производство этих головоломок, которое в свою очередь значительно снижало на них цену.

С начала 1909 года и по сегодняшний день принцип производства изменился не сильно. Изображение, напечатанное на картонной основе, разрезается с помощью перфоратора на множество маленьких деталей. Популярность пазлов не обошла и Россию. Уже в середине 19 века эта игра была распространена на территории нашей страны, только называлась она смешным словом на немецкий манер «пузеля». Пузеля состояли из множества фрагментов, обычно их число не превышало ста штук. В советский период пазлы практически исчезли из магазинов, популярность к ним вернулась только к концу 20 века [1].

Во время курсового проектирования необходимо разработать автоматизированную систему «Игра «Puzzle», с помощью которой администратор сможет создавать пазл и задавать ему уровень сложности, а игрок сможет собирать пазл, используя систему подсказок.

Разработка системы будет производиться по технологии быстрой разработки приложений RAD (Rapid Application Development), которая поддерживается методологией структурного проектирования и включает элементы объектно-ориентированного проектирования и анализа предметной области [2].

При проектировании системы будут использоваться методология ООАП (Object-Oriented Analysis/Design), в основу которой положена объектно-ориентированная методология представления предметной области в виде объектов, являющихся экземплярами соответствующих классов, и язык моделирования UML (Unified Modeling Language), который является стандартным инструментом для разработки «чертежей» программного обеспечения [3].

1 Описание и анализ предметной области

Под предметной областью (application domain) принято понимать ту часть реального мира, которая имеет существенное значение или непосредственное отношение к процессу функционирования программы. Другими словами, предметная область включает в себя только те объекты и взаимосвязи между ними, которые необходимы для описания требований и условий решения некоторой задачи [4].

1.1 Описание игры «Puzzle»

Пазлы – это головоломка из множества разрезанных картинок. Их складывают в определённом порядке, чтобы получилась одна большая картина [5].

На рисунке 1 приведён внешний вид стандартного картонного пазла.

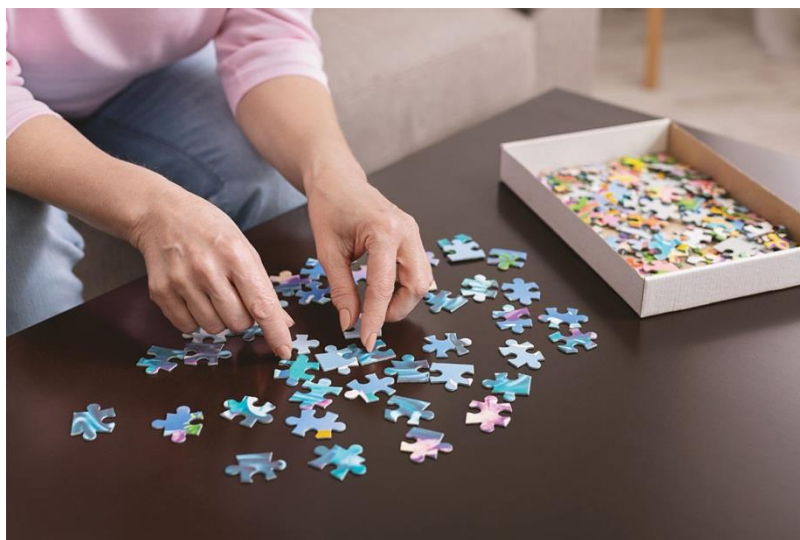


Рисунок 1 – Внешний вид стандартного пазла

В наши дни существует огромное количество любившихся складных головоломок, основными различиями которых являются [6]:

- количество элементов (пазлы разной сложности: до 30 элементов – подойдут детям; до 1000 – вызовут интерес у подростков и более взрослого населения; до 30000 элементов – для тех, кто имеет большой опыт в сборке

таких головоломок и любит трудные задачи (на рисунке 2 представлен пример пазла из 2000 деталей));

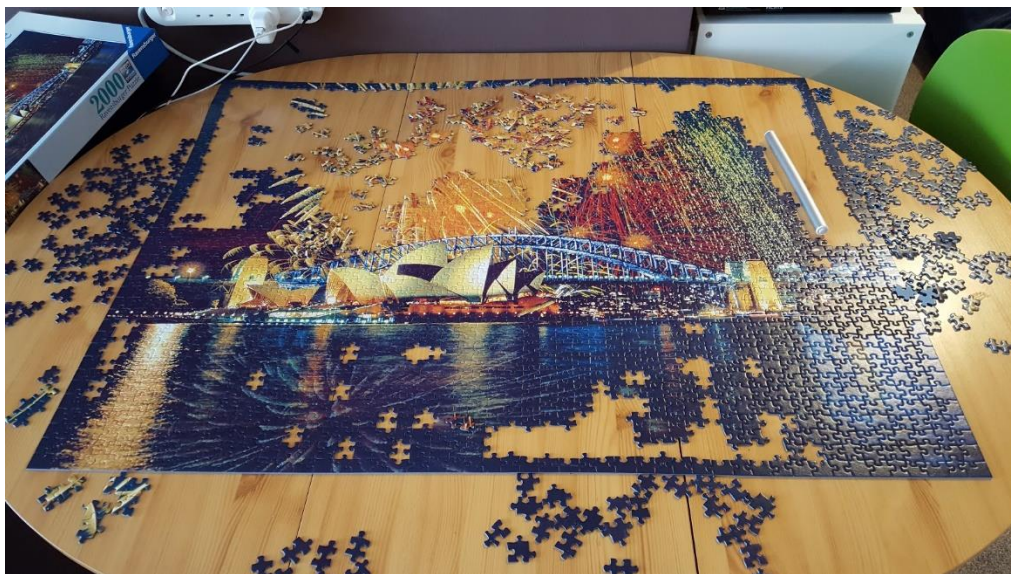


Рисунок 2 – Пример пазла из 2000 элементов

– возраст игроков (пазлы для детей – более крупный и яркий рисунок (на рисунке 3 показан пример пазла для детей) и размер – скорее картинка, а не фотография; и пазлы для людей постарше);

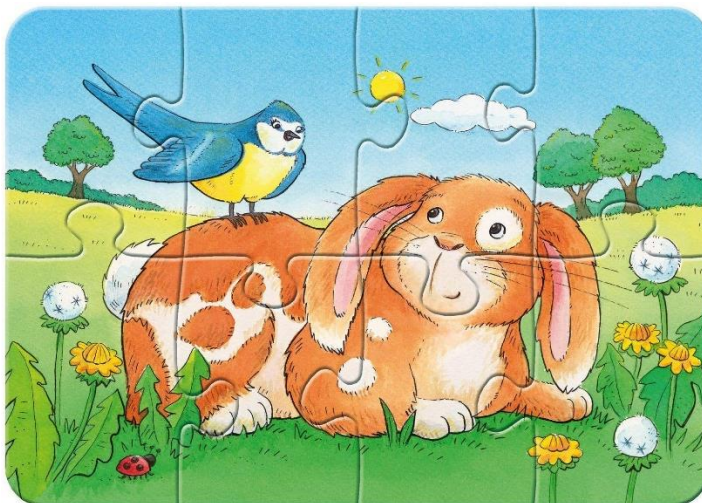


Рисунок 3 – Пример детского пазла

– материал (может быть разным: традиционный картонный, забытый деревянный (см. рисунок 4), удобный пластмассовый/пластиковый, мягкий/поролоновый, необычный магнитный);



Рисунок 4 – Пример пазла из дерева

- тип элементов (круги, овалы, треугольники, в форме животных, с подсветкой и многие другие);
- объём (пазлы 3D и плоские). На рисунке 5 приведён внешний вид 3D пазла;



Рисунок 5 – Внешний вид 3D пазла

- место (площадка) для сбора (пол, стол, магнитная доска или стена, пазл онлайн – играем на компьютере);

Общая схема сборки пазлов [6]:

1 Поиск и сборка деталей с гладкими краями. Собрав рамку, вы получите ограниченное пространство и приблизительное понимание, куда двигаться дальше.

2 Перевертывание всех элементов лицевой частью вверх. Это, на первый взгляд, утомительное занятие в разы упрощает сборку пазлов. Видя перед собой все фрагменты, вы намного быстрее ориентируетесь в оттенках и формах деталей, что, в свою очередь, сильно ускоряет процесс.

3 Сортировка по цвету. Для большинства головоломок – это отличный ход, поскольку отличить оттенки, принадлежащие к той или иной области изображения, чаще всего достаточно просто. Намного сложнее справиться с «невозможными» пазлами (англ. impossible), где картинка представляет собой повторяющийся узор одного цвета. Но и там можно попытаться отделить оттенки, которые появляются реже. Все, что невозможно определить с первого раза, сыпайте в кучу, к которой возвратитесь, когда исчерпаете все основные цветовые фрагменты.

4 Сортировка по форме. Каждый производитель характеризуется своим типом нарезки: от классической прямоугольной с выступами и впадинами до круглой или треугольной с волнистыми краями. Чем разнообразнее детали, тем легче собирать пазл.

5 Особенные детали. Некоторые элементы представляют собой часть самых заметных фрагментов изображения (например, надписей или единственно повторяющегося цветного мотива). Старайтесь сразу же отделять их от остальных деталей: чем больше у вас таких «островков», тем проще будет потом сложить головоломку в единое целое [6].

Не пытайтесь охватить весь пазл сразу: гораздо эффективнее разбить его на небольшие участки. Визуальный прогресс будет мотивировать вас продолжать собирать картинку. Готовые секции поместите в приблизительные места возле рамки, даже если они ещё не связаны с ней. Визуализация пространства облегчит поиск других недостающих элементов. Если вы устали или пазл вам надоел, сделайте перерыв. Через некоторое время свежим взглядом можно легко заметить все, что вы пропустили [6]!

Трудно даже представить, сколько пазлов придумано и создано, тем более, сложно вообразить, как уместить в наших стандартных

(малогабаритных) квартирах это богатство, да и вряд ли найдётся много людей, готовых на постоянной основе и в огромных количествах скупать много детальные игрушки. Выход есть – собирать головоломки онлайн, используя компьютеры, планшеты, телефоны [7].

1.2 Описание систем-аналогов

В интернете существует большое количество версий игры «Puzzle». Все они разные и имеют свои особенности и функционал. Рассмотрим основные характеристики нескольких систем-аналогов.

1.2.1 Система «Пазл – ДИКИЕ ЖИВОТНЫЕ»

Система «Пазл – ДИКИЕ ЖИВОТНЫЕ» на платформе «ЯндексИгры» является классической головоломкой. Она предлагает несколько бесплатных и премиальных головоломок, а также позволяет составлять собственные пазлы из своих картинок и фотографий [8].

На рисунке 6 приведена главная экранная форма игры.



Рисунок 6 – Экранная форма программы «Пазл – ДИКИЕ ЖИВОТНЫЕ»

Данная система разбивает изображение на то количество деталей, которое выбирает сам пользователь (25, 49 или 100 частей). После начала

игры, фрагменты перемешиваются на поле (см. рисунок 7). При правильной сборке детали соединяются и больше не разъединяются. Собранные фрагменты можно перемещать по всему игровому полю, но у деталей поле ограниченное.

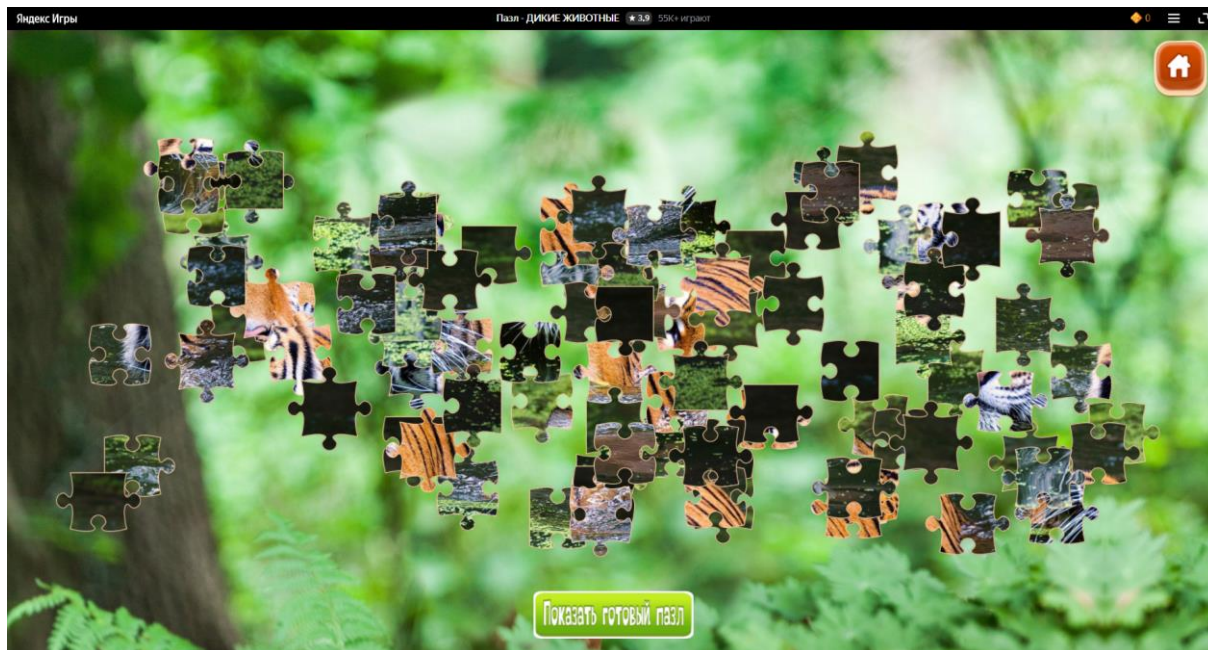


Рисунок 7 – Игровое поле программы «Пазл – ДИКИЕ ЖИВОТНЫЕ»

После сборки всего пазла, игроку присваиваются звёзды и монеты (зависит от количества деталей), за которые он может приобрести другие картинки.

К достоинствам данной системы относятся:

- понятный интерфейс;
- возможность создания собственного пазла;
- разбиение картинки на разное количество деталей;
- возможность посмотреть полную картинку;
- ненавязчивое звуковое и музыкальное сопровождение.

К недостаткам системы относятся:

- отсутствие подсказок;
- наложение пазлов друг на друга;
- маленькое количество картинок;
- отсутствие рейтинга;

- отсутствие счёта времени;
- ограниченное игровое поле.

1.2.2 Система «Сказочный пазл»

Система «Сказочный пазл» на платформе «ЯндексИгры» является классической головоломкой, в которой необходимо перетаскивать подходящий кусочек пазла на другой, чтобы объединить их и собрать весь пазл [9].

На рисунке 8 приведена главная экранная форма игры.



Рисунок 8 – Экранная форма программы «Сказочный пазл»

Данная система разбивает изображение детали (от 15 штук) и размещает их на ленту внизу (см. рисунок 9). Количество деталей зависит от уровня. После начала игры, необходимо перетаскивать фрагменты на игровое поле. При правильной сборке детали соединяются и больше не разъединяются. Новое изображение открывается при прохождении текущей картинки. Подсказку можно получить только после просмотра рекламного ролика.

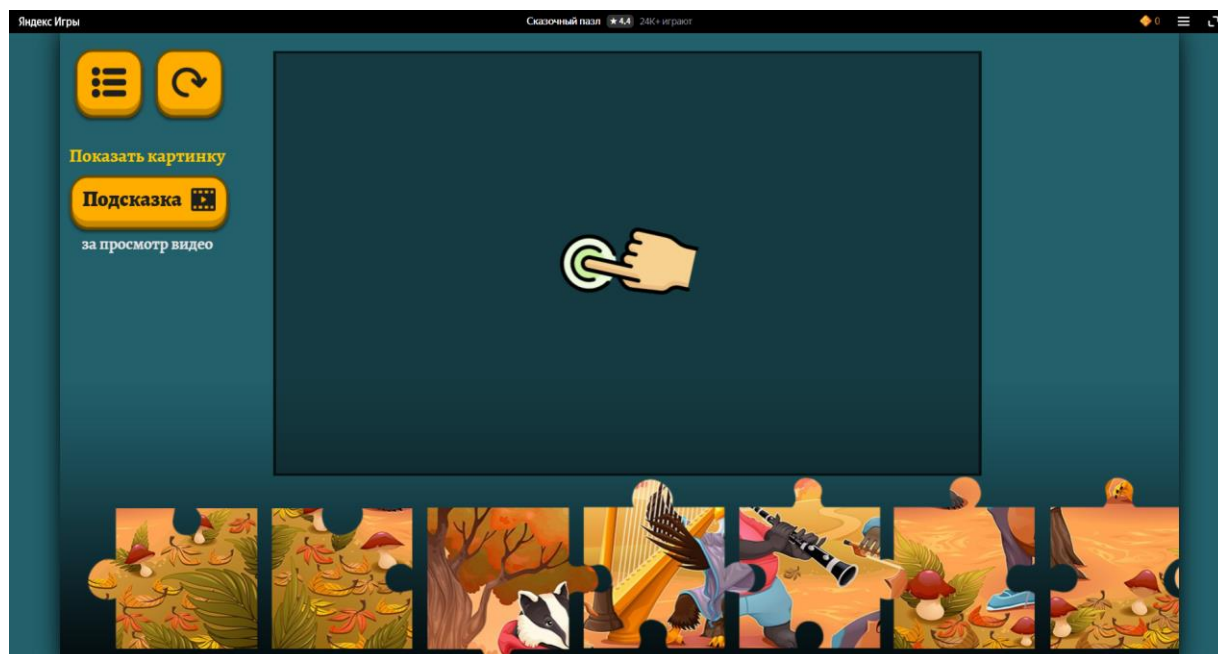


Рисунок 9 – Игровое поле программы «Сказочный пазл»

К достоинствам данной системы относятся:

- понятный интерфейс;
- возможность выбрать язык программы;
- возможность посмотреть полную картинку;
- разнообразие интересных картинок.

К недостаткам системы относятся:

- отсутствие выбора желаемой картинки;
- отсутствие выбора уровня сложности;
- отсутствие подсказок;
- отсутствие счёта времени;
- наложение пазлов друг на друга;
- агрессивное звуковое сопровождение;
- отсутствие рейтинга;
- ограниченное игровое поле.

На основании анализа возможностей систем-аналогов были сформулированы требования к разрабатываемой системе (см. таблицу 1).

Таблица 1 – Сравнительные характеристики систем-аналогов

Название показателя	Система «Пазлы – ДИКИЕ ЖИВОТНЫЕ»	Система «Сказочный пазл»	Разрабатываемая система
Понятный интерфейс	Да	Да	Да
Уровень сложности	Да	Нет	Да
Просмотр исходной картинки	Да	Да	Да
Взятие подсказки	Нет	Нет	Да
Звуковое сопровождение	Да	Да	Да
Разнообразие картинок	Нет	Нет	Да
Рейтинг	Нет	Нет	Да
Счёт времени	Нет	Нет	Да
Простое и функциональное игровое поле	Нет	Нет	Да
Выбор языка	Нет	Да	Нет
Загрузка своего изображения	Да	Нет	Нет

1.3 Диаграмма объектов предметной области

Анализ предметной области, позволяет выделить её сущности, определить первоначальные требования к функциональности и определить границы проекта. Модель предметной области должна быть документирована, храниться и поддерживаться в актуальном состоянии до этапа реализации [10].

Диаграмма представляет статическое расположение взаимодействующих между собой объектов, называемых сущностями. На диаграмме обычно отображаются объекты, отношения между ними, и атрибуты (свойства сущности) по необходимости. На диаграмму включаются только те объекты, описание которых может быть представлено в виде классов [11].

На рисунке 10 приведена диаграмма объектов предметной области. На диаграмме центральным объектом является картинка, которая собирается игроком в игре «Puzzle». Картинка состоит из деталей. Информация о текущем состоянии игры хранится в базе данных. Галереей возможных игровых картинок управляет администратор, загружая по необходимости новые. Галерея хранится в базе данных.

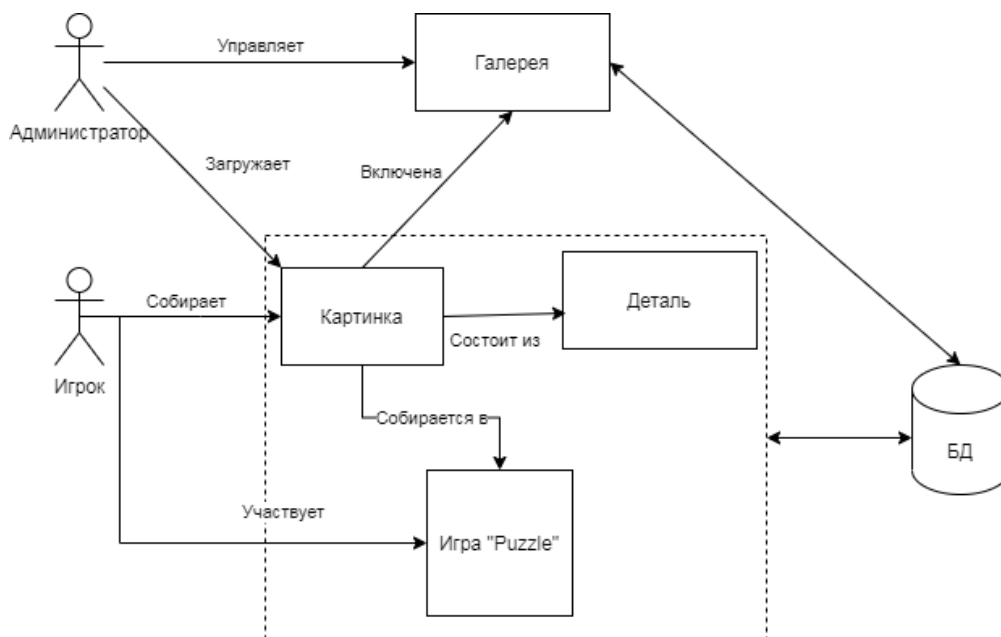


Рисунок 10 – Диаграмма объектов системы

1.4 Постановка задачи

Во время курсового проектирования необходимо разработать автоматизированную систему «Игра «Puzzle», предназначенную для создания и сбора пазла, с помощью которой игрок сможет собирать пазлы разного уровня сложности, созданные администратором, а также использовать систему подсказок и просматривать рейтинг.

В системе должно быть реализовано две роли пользователей: администратор и игрок, поэтому для того, чтобы работать с системой, при первом входе в систему пользователи должны пройти процедуру регистрации: ввести логин и пароль (длина логина должна быть от 4 до 8 символов, длина пароля – от 4 до 10 символов). При повторном входе, пользователю необходимо авторизоваться в системе и затем, после аутентификации, система должна настроить интерфейс пользователя на заданную роль.

Администратору должны быть доступны следующие основные функции:

- 1 Работа с галереей картинок: администратор должен иметь возможность добавлять картинки из файла (формат файлов *.png) в галерею. При добавлении картинки система должна привести картинку к единому формату – 4х3. Также администратор будет иметь возможность сохранять картинки в базу данных, удалять картинки из галереи, сортировать галерею по определённым категориям (максимальное количество категорий – 8).

- 2 Настройка уровня сложности для картинки: администратор должен иметь возможность настраивать для каждой картинки уровень сложности, который будет зависеть от количества фрагментов (по горизонтали от 4 до 8 и вертикали от 4 до 12), формы деталей (прямоугольные, треугольные или фигурные) и типа сборки (на поле, лента или куча).

- 3 Сохранение пазла в базу данных (БД) в заданной структуре.

Основная функция игрока – это сбор пазла. Перед сбором пазла игрок должен выбрать уровень сложности (в системе должно быть реализовано 5 уровней сложности) и картинку из доступного списка, который сформирует система для каждого уровня сложности или же продолжить сохранённую игру. Также система должна быть обеспечена звуковым сопровождением, которое пользователь, при желании, сможет выключить.

Процесс сбора пазла подразумевает под собой выбор фрагмента (на поле, на ленте или в куче) и перемещение его на нужное месторасположение на поле. Процесс сборки пазла должен быть визуализирован. При постановке фрагмента в нужное место, он должен соединиться с уже собранным пазлом. Собранный пазл не может разъединиться до конца игры. У пользователя должна быть возможность посмотреть картинки целиком или воспользоваться подсказкой (количество подсказок должно зависеть от уровня сложности и варьироваться в диапазоне от 1 до 5). Система должна контролировать количество взятых подсказок. В случае взятия подсказки система должна указать, на каком месте необходимо поставить выбранный фрагмент. Кроме того, во время игры пользователь может сохранить пазл в базу данных, чтобы иметь возможность продолжить игру.

У пользователя должна быть возможность посмотреть рейтинг 10 лучших игроков количеству набранных очков, которые считаются исходя из уровня сложности и времени, затраченного на сбор конкретного пазла.

В системе также должна быть обеспечена возможность получения справочной информации как о самой системе, так и предоставляемых ею возможностях.

Таким образом, системы должна решать следующие задачи:

1) общесистемные функции:

- авторизация пользователя в системе (ввод логина/пароля);
- регистрация пользователя в системе (ввод логина, пароля);
- аутентификация пользователя в системе, настройка интерфейса пользователя на заданную роль;

- приведение картинки к формату 4х3 (исходя из разрешения экрана);
- автоматическое составление игры по заданным параметрам;
- визуализация процессов работы с пазлом;
- контроль количества взятых подсказок;
- проверка правильности сбора пазла;
- формирование рейтинга по заданному критерию;
- выдача справочной информации о системе;

2) функции администратора:

- настройка уровня сложности для картинки:
 - 1 задание размеров;
 - 2 выбор формы деталей;
 - 3 выбор типа сборки;
- работа с галереей:
 - 1 добавление картинки (загрузка картинки из файла);
 - 2 удаление картинки;
 - 3 сортировка галереи по категориям;
 - 4 сохранение картинки в галерею;
 - 5 навигация по галерее;

3) функции игрока:

- выбор уровня сложности;
- выбор игры (сохранённая игра; новая игра);
- дополнительные настройки:
 - 1 включение/выключение звукового сопровождения;
 - 2 включение/выключение музыкальное сопровождение;
- разгадывание игры с организацией системы подсказок:
 - 1 выбор детали;
 - 2 перемещение детали на нужное место;
 - 3 взятие подсказки;

- 4 просмотр картинки;
- сохранение игры;
- просмотр рейтинга.

2 Проектирование системы

2.1 Выбор и обоснование архитектуры системы

Архитектура – совокупность важнейших решений об организации программной системы. Архитектура включает [11]:

- выбор структурных элементов и их интерфейсов, с помощью которых составлена система, а также их поведения в рамках сотрудничества структурных элементов;
- соединение выбранных элементов структуры и поведения во всё более крупные системы;
- архитектурный стиль, который направляет всю организацию – все элементы, их интерфейсы, их сотрудничество и их соединение.

2.1.1 Виды архитектур

За годы развития ПО разработчикам удалось придумать надёжные подходы, чтобы устранить недостатки проектирования без архитектуры [12].

Рассмотрим следующие виды архитектур.

Многослойная архитектура

Этот подход работает по принципу разделения ответственностей. ПО разделено на слои (см. рисунок 11), лежащие друг на друге, и каждый из них выполняет определённую обязанность.

Архитектура делит ПО на следующие слои:

- Слой представления (Presentation Layer) содержит пользовательский интерфейс и отвечает за обеспечение хорошего пользовательского опыта.
- Слой бизнес-логики (Business Logic Layer), как следует из названия, содержит бизнес-логику приложения. Он отделяет UI/UX от вычислений, связанных с бизнесом. Это позволяет с лёгкостью изменять логику в зависимости от постоянно меняющихся бизнес-требований, никак не влияя на другие слои.

– Слой передачи данных (Data Link Layer) отвечает за взаимодействие с постоянными хранилищами, такими как базы данных, и прочую обработку информации, которая не связана с бизнесом.

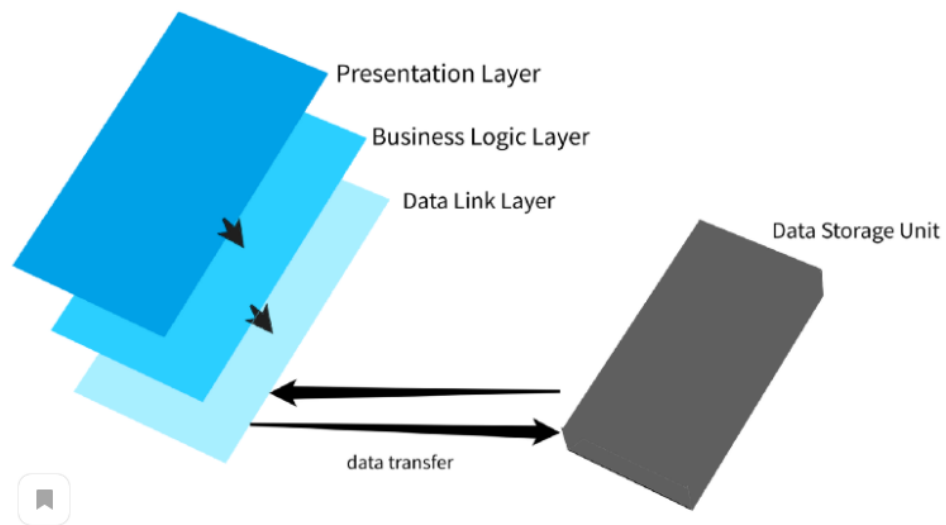


Рисунок 11 – Многослойная архитектура

Данные и элементы управления проходят через каждый слой в дизайне и передаются от одного к другому. Эта система также повышает уровень абстракции и в некоторой степени даже стабильность ПО.

Двухуровневая система

Эта система состоит из двух физических машин в качестве сервера и клиента (см. рисунок 12). Она обеспечивает изоляцию операций управления данными, обработки данных и операций представления.

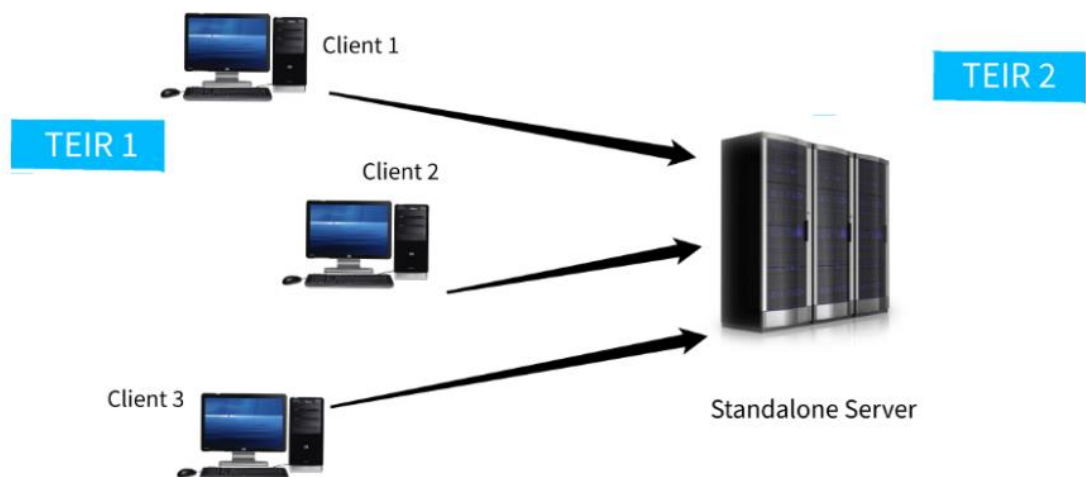


Рисунок 12 – Двухуровневая система

Клиент содержит слои презентации, бизнес-логики и передачи данных.

Сервер включает хранилища и базы данных.

N-уровневая система

Такие архитектуры обладают высокой масштабируемостью как по горизонтали, так и по вертикали. Реализация n-уровневой системы, как правило, обходится дороже, но обеспечивает высокую производительность. Поэтому она обычно применяется в крупных и комплексных программных решениях.

Сервис-ориентированная архитектура (SOA)

Эта архитектурная модель состоит из компонентов и приложений, которые связываются друг с другом с помощью чётко определённых сервисов (см. рисунок 13).

Она состоит из 5 элементов:

- сервисы (Services);
- сервисная шина (Service Bus);
- сервисный репозиторий (Service Repository catalogue of services);
- безопасность SOA (SOA Security);
- управление SOA (SOA Governance).

Клиент отправляет запрос с использованием стандартного протокола и формата данных по сети. Этот запрос обрабатывается ESB (сервисная шина предприятия), которая считается сердцем сервис-ориентированной архитектуры и отвечает за оркестровку и маршрутизацию. С помощью сервисного репозитория ESB направляет запрос в специальный сервис, который может взаимодействовать с другими сервисами и базами данных, чтобы составить полезную нагрузку (данные) ответа.

Полный вызов ответа на запрос согласуется с правилами управления и безопасности SOA для выполнения безопасной и корректной транзакции.

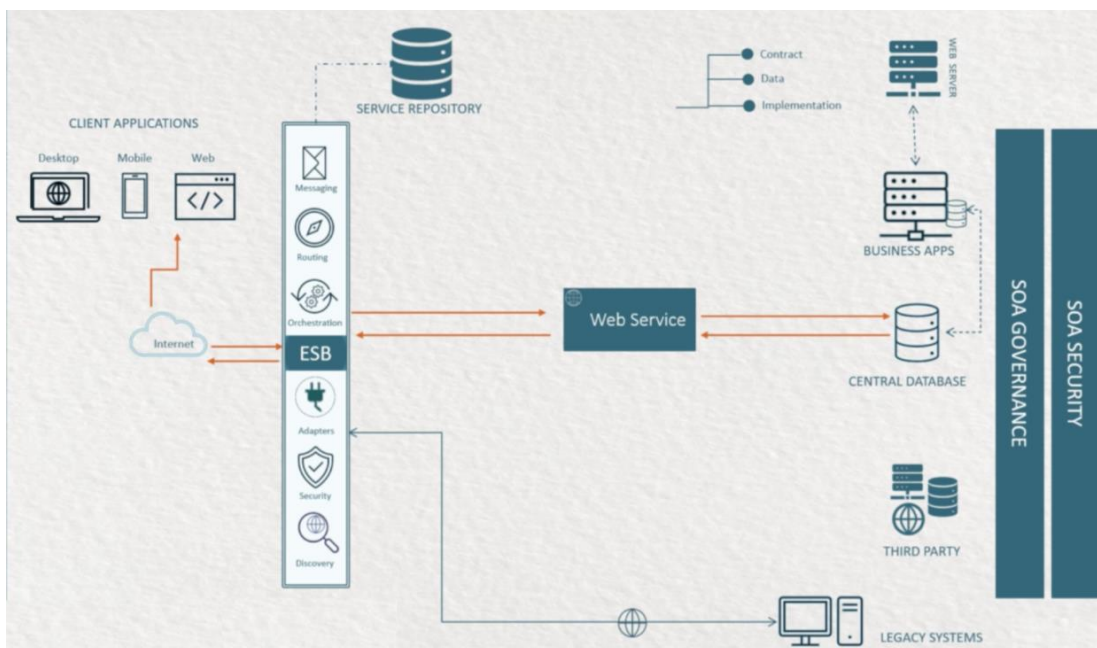


Рисунок 13 – Сервис-ориентированная архитектура (SOA)

Микросервисная архитектура

При таком подходе приложение разрабатывается как набор небольших сервисов, каждый из которых работает в собственном процессе и связывается с легковесными механизмами, обычно API для HTTP-ресурса.

Эти сервисы основываются на бизнес-возможностях и могут развёртываться независимо друг от друга с помощью полностью автоматизированного механизма.

Централизованное управление между сервисами минимально. Они могут быть написаны на разных языках, использовать разные технологии хранения данных.

Архитектура работает по принципу компонентизации сервисов. Она разделяет программное обеспечение на различные изолированные компоненты (сервисы), каждый из которых несёт единую ответственность. Изменения в одной сервисе не должны затрагивать другие.

2.1.2 Клиент-серверные архитектуры

Клиент-серверные архитектуры подразделяются по уровням. Схема клиент-серверной архитектуры представлена на рисунке 14.

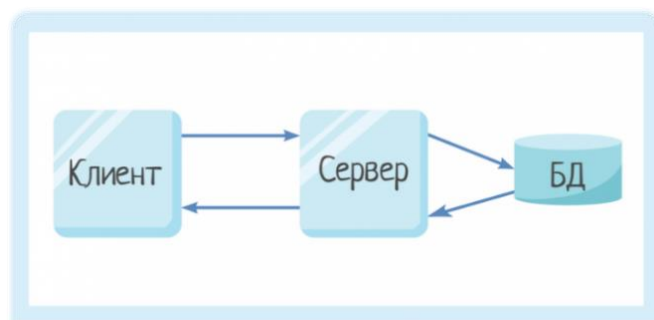


Рисунок 14 – Схема клиент-серверной архитектуры

Самыми используемыми являются следующие типы клиент-серверных архитектур [13]:

- Одноуровневая архитектура «клиент-сервер» (см. рисунок 15). В ней все прикладные программы рассредоточены по рабочим станциям, которые обращаются к общему серверу баз данных или к общему файловому серверу. Никаких прикладных программ сервер при этом не исполняет, только предоставляет данные.

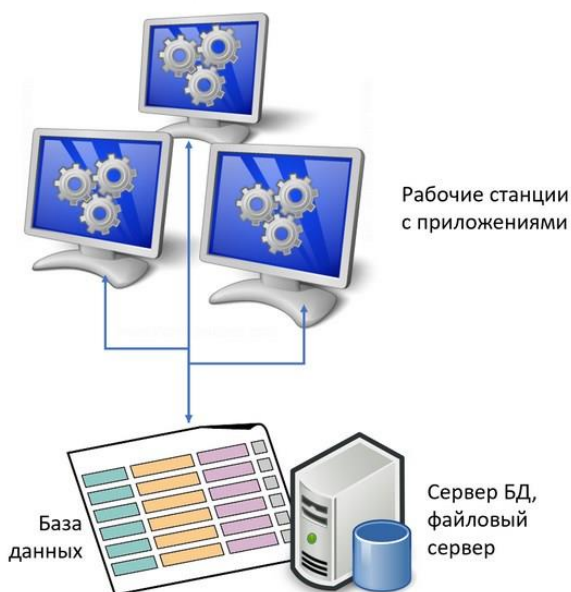


Рисунок 15 – Одноуровневая архитектура

- К двухуровневой архитектуре (см. рисунок 16) «клиент-сервер» следует относить такую, в которой прикладные программы сосредоточены на сервере приложений (Application Server), а в рабочих станциях находятся программы-клиенты, которые предоставляют для пользователей интерфейс для работы с приложениями на общем сервере.

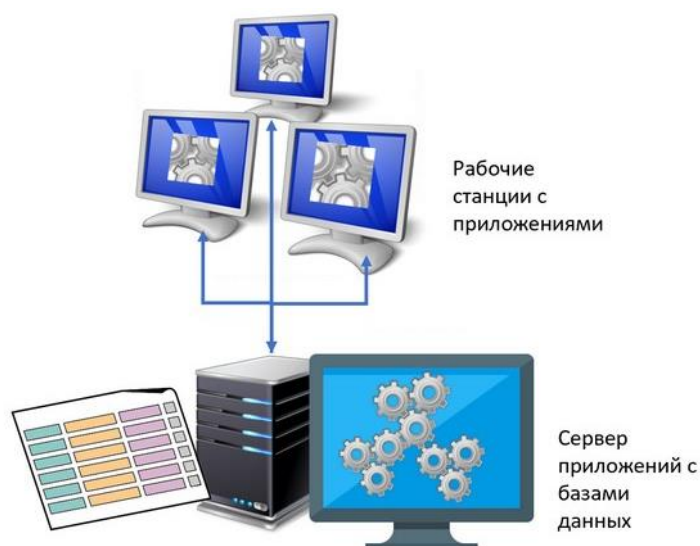


Рисунок 16 – Двухуровневая архитектура

– В трёхуровневой архитектуре (см. рисунок 17) сервер баз данных, файловый сервер и другие представляют собой отдельный уровень, результаты работы которого использует сервер приложений. Логика данных и бизнес-логика находятся в сервере приложений. Все обращения клиентов к базе данных происходят через промежуточное программное обеспечение (middleware), которое находится на сервере приложений. Вследствие этого, повышается гибкость работы и производительность.



Рисунок 17 – Трёхуровневая архитектура

- в многоуровневой архитектуре несколько серверов приложений используют результаты работы друг друга, а также данные от различных серверов баз данных, файловых серверов и других видов серверов.

Приложение будет реализовано на основе двухуровневой клиент-серверной архитектуре.

Преимущества двухуровневой архитектуры:

- легко конфигурировать и модифицировать приложения;
- пользователю обычно легко работать в такой среде;
- хорошая производительность и масштабируемость;

Однако, у двухуровневой архитектуры есть и ограничения:

- производительность может падать при увеличении числа пользователей;
- потенциальные проблемы с безопасностью, поскольку все данные и программы находятся на центральном сервере;
- все клиенты зависимы от базы данных одного производителя.

2.1.3 Типы клиентов

Для двухуровневой архитектуры характерно три типа клиентов:

- Тонкий клиент – компьютер или программа-клиент в сетях с клиент-серверной или терминальной архитектурой, где большая часть задач по обработке информации перенесена на сервер и права доступа клиента строго ограничены.

- Толстый клиент – это приложение, обеспечивающее (в противовес тонкому клиенту) полную функциональность и независимость от центрального сервера. Часто сервер в этом случае является лишь хранилищем данных, а вся работа по обработке и представлению этих данных переносится на машину клиента.

- Веб-клиент – тип клиента, при котором все вычисления производятся на стороне сервера, а на стороне клиента не используется никаких прикладных программ, за исключением браузера.

В приложении будет использоваться тип толстого клиента. Все вычисления будут производиться на стороне клиента, на стороне сервера будет осуществляться хранение данных.

2.1.4 Протоколы передачи данных

Протокол передачи данных – набор определённых правил или соглашений интерфейса логического уровня, который определяет обмен данными между различными программами. Эти правила задают единообразный способ передачи сообщений и обработки ошибок [14].

Различают множество различных протоколов, которые относятся к различным уровням в стеке TCP/IP. Некоторыми из самых известных являются [15]:

1 MAC – Media Access Control, низкоуровневый протокол, который используется для идентификации сетевых устройств.

2 IP – Internet Protocol, протокол который объединил различные устройства в единую сеть. Основная его задача – маршрутизация датаграмм. Данный протокол не устанавливает соединение, не подтверждает доставку пакетов получателю, как и не контролирует целостность данных.

3 TCP – Transmission Control Protocol, обеспечивает и контролирует передачу данных и следит за надёжностью и целостностью.

4 UDP – User Datagram Protocol, обеспечивает передачу данных, не создавая соединения. Этот протокол считается недостаточно надёжным, но имеет более высокую скорость доставки данных по сравнению с TCP

5 HTTP – Hyper Text Transfer protocol, обеспечивает передачу гипертекста, на его основе работают веб-сайты.

6 FTP – File Transfer Protocol, служит для передачи файлов.

7 SSH – Secure Shell. Протокол, отвечающий за создание удалённого канала для удаленного управления другой операционной системой.

Для реализации взаимодействия между сервером и клиентом устанавливается соединение по протоколу TCP. Взаимодействие с СУБД, в

свою очередь, будет осуществляться по протоколу `mysql protocol`, который передаёт данные в бинарном виде, и устанавливается «поверх» TCP соединения.

2.2 Структурная схема системы

Система (греч. «составленное из частей», «соединение» от «соединяю») – множество элементов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство [16].

Как следует из определения, отличительным (главным свойством) системы является её целостность: комплекс объектов, рассматриваемых в качестве системы, должен обладать общими свойствами и поведением. Очевидно, необходимо рассматривать и связи системы с внешней средой. В самом общем случае понятие «система» характеризуется [17]:

- наличием множества элементов;
- наличием связей между ними;
- целостным характером данного устройства или процесса.

Структурная схема – это схема, определяющая основные функциональные части изделия, их назначение и взаимосвязи [18].

Сущность структурного подхода к разработке системы заключается в её декомпозиции (разбиении) на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее. Процесс разбиения продолжается вплоть до конкретных процедур. При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимосвязаны [19].

На рисунке 18 приведена структурная схема разрабатываемой системы, разделяется клиентскую и серверную часть. Взаимодействие между ними осуществляется по TCP протоколу.

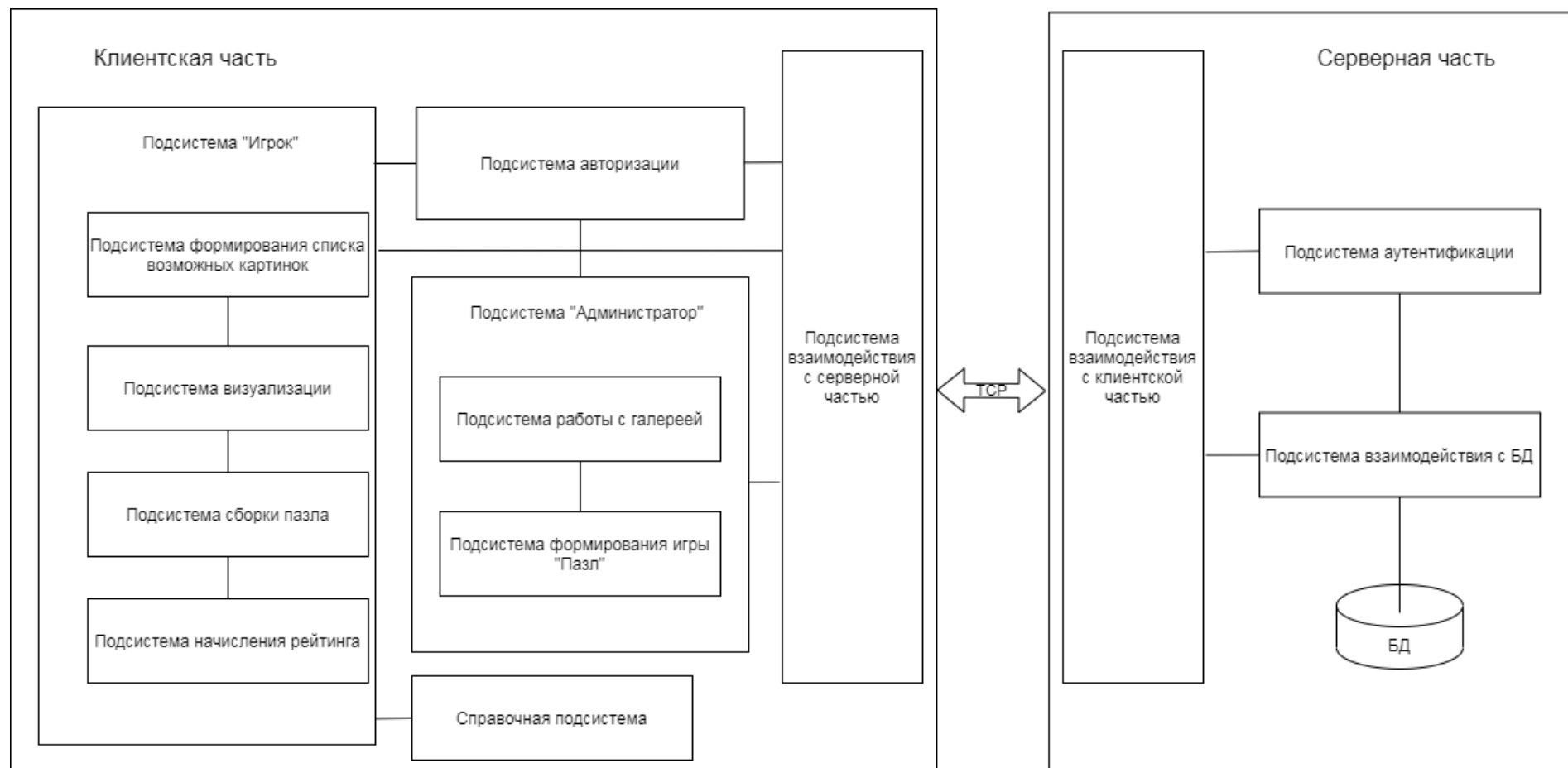


Рисунок 18 – Структурная схема системы

В состав клиентской части входит:

1 Система «Администратор», которая включает в себя:

- подсистему работы с галереей, которая отвечает за добавление, сохранение и удаление картинок в галерею;
- подсистему формирования игры, которая определяет уровень сложность и параметры игры (количество разбиений, форма пазлов) для заданной картинки;

2 Система «Игрок», которая включает в себя:

- подсистему формирования списка возможных картинок, которая формирует список пазлов, соответствующих конкретному уровню сложности;
- подсистему визуализации, которая отображает игровое поле и результаты действий пользователя на экран;
- подсистему сборки пазла, которая контролирует правильность составления фрагментов пазла;
- подсистему начисления рейтинга, которая считает количество набранных баллов.

3 Подсистема авторизации, которая осуществляет выдачу прав пользователю в зависимости от его роли.

4 Справочная подсистема, которая содержит сведения о системе (руководство пользователю) и об её разработчиках.

5 Подсистема взаимодействия с сервером, которая осуществляет установку соединения с сервером, формирование и отправку запросов.

В состав серверной части входит:

1 Подсистема взаимодействия с клиентом, которая осуществляет приём данных с клиента и передачу их на обработку.

2 Подсистема аутентификации, которая осуществляет проверку введённых пользователем идентификационных данных для подтверждения входа.

3 Подсистема взаимодействия с базой данных, которая представляет собой СУБД, позволяющую управлять БД.

4 База данных, которая осуществляет хранение данных о совершенных играх, рейтинге игроков, картинках в галерее.

2.3 Разработка спецификации требований

Разработка спецификации программного обеспечения является одним из фундаментальных процессов технологии разработки ПО. Этот процесс анализа, формирования, документирования и проверки функциональных возможностей и ограничений системы называется «разработка требований» (спецификация требований). Он является критическим этапом в создании всех видов программных систем, что обусловлено тем, что ошибки, допущенные на этой стадии, ведут к возникновению серьёзных проблем на этапах проектирования и разработки [20].

Требования – это свойства, которыми должно обладать ПО для адекватного определения функций, условий и ограничений выполнения ПО, а также объёмов данных, технического обеспечения и среды функционирования [21].

2.3.5 Функциональная спецификация

Функциональные требования задают «что» система должна делать; нефункциональные – с соблюдением «каких условий» (например, скорость отклика при выполнении заданной операции). При разработке этих требований в первую очередь необходимо учитывать потребности пользователя (заказчика). Пользовательские требования (User Requirements) – описывают цели/задачи пользователей системы, которые должны достигаться/выполняться пользователями при помощи создаваемой программной системы. Часто пользовательские требования представляют в виде сценариев (вариантов использования) Use Case [22].

Функциональная спецификация системы приведена в таблице 2.

Таблица 2 – Перечень функций, выполняемых системой (клиентская часть)

Название подсистемы	Название функции	Информационная среда			
		Входные данные		Выходные данные	
		Назначение (наименование)	Тип, ограничения	Назначение (наименование)	Тип, ограничения
1	2	3	4	5	6
Справочная	Выдать сведения о разработчиках	Сведения о разработчиках системы (ФИО, номер группы)	Текст	Визуальное отображение информации	-
	Выдать сведения о системе	Файл справки	Текст (*.HTML)		
				Авторизации	Ввести пароль
	Допустимая длина	Целое 4..10	Код ошибки	Целое	
	Ввести логин	Допустимые символы	Латинские буквы, цифры, символ «_»	Логин	
		Допустимая длина	4...8	Код ошибки	Целое

Продолжение таблицы 2

1	2	3	4	5	6
Работы с галереей	Добавить изображение в галерею	Имя файла	Строка (*.PNG)	Изображение	Изображение
				Название изображение	Строка
				Код ошибки	Целое
	Выбрать изображение	Список изображений	Список строк	Название изображения	Строка
	Удалить изображение	Название изображения	Строка	Список названий изображений	Список строк
Формирования списка возможных картинок	Получить список новых изображений	Текущий список изображений	Список имён файлов формата PNG	Список изображений для загрузки	Список строк
		Сложность	Целое 1...5	Код ошибки	Целое
	Загрузить изображение с сервера	Название изображения	Строка	Изображение	Изображение (*.PNG)
				Код ошибки	Целое

Продолжение таблицы 2

1	2	3	4	5	6
Формирования списка возможных картинок	Получить список картинок для сборки	Категория	Строка	Список изображений	Список строк
		Сложность	Целое 1...5		
		Список изображений	Список имён файлов формата PNG	Код ошибки	Целое
Формирования игры	Выбрать уровень сложности	Список уровней сложности	Целое 1...5	Уровень сложности	Целое
	Выбрать форму деталей	Список формы детали	треугольная, квадратная, особая	Формы детали	Перечисление
	Установить размер по ширине	Допустимый диапазон значений	Целое 4...8	Количество фрагментов по ширине	Целое
	Установить размер по высоте	Допустимый диапазон значений	Целое 4...12	Количество фрагментов по высоте	Целое
	Выбрать тип сборки	Список типов сборки	на поле, на ленте, в куче	Тип сборки	Перечисление

Продолжение таблицы 2

1	2	3	4	5	6
Формирования игры	Выбрать категорию	Список категорий	абстракция, животные, детские, искусство, природа, градиент	Название категории	Строка
Подсистема визуализации	Отобразить поле	Список фрагментов	Список целых чисел	Отображение всех фрагментов	—
	Отобразить кучу	Список фрагментов в куче	Список целых чисел	Отображение фрагментов в куче	—
	Отобразить ленту	Список фрагментов на ленте	Список целых чисел	Отображение фрагментов в ленте	—
		Количество видимых фрагментов	Целое		
	Отобразить группу фрагментов	Список фрагментов	Список целых чисел	Отображение фрагментов в группе	—

Продолжение таблицы 2

1	2	3	4	5	6
Подсистема визуализации	Отобразить фрагмент	Индекс фрагмента	Целое	Отображение фрагмента	—
		Изображение	Изображение (*.PNG)		
		Форма деталей	треугольная, квадратная, особая		
	Отобразить подсказку	Индекс фрагмента	Целое	Необходимое положение	Пара целых чисел
Сборки пазла	Выбрать группу фрагментов	Позиция мыши	Список координат	Индекс группы	Целое
	Передвинуть группу фрагментов	Индекс группы	Целое	Новые координаты группы	Список координат
		Координаты группы	Список координат		
	Выбрать фрагмент на поле	Список фрагментов на поле	Список целых чисел	Индекс фрагмента	Целое
	Поменять два фрагмента местами на поле	Индекс первого фрагмента	Целое	Список фрагментов	Список целых чисел
		Индекс второго фрагмента	Целое		

Продолжение таблицы 2

1	2	3	4	5	6
Взаимодействия с серверной частью	Открыть соединение	Данные клиента	Бинарный	Код ошибки	Целое
	Передать данные	Данные	Бинарный	Код ошибки	Целое

Таблица 3 – Перечень функций, выполняемых системой (серверная часть)

Название подсистемы	Название функции	Информационная среда			
		Входные данные		Выходные данные	
		Назначение (наименование)	Тип, ограничения	Назначение (наименование)	Тип, ограничения
Взаимодействие с клиентской частью	Принять соединение	Данные подключённого устройства	Бинарный	Код ошибки	Целое
	Передать данные	Данные	Бинарный	Код ошибки	Целое
Аутентификации	Проверить учетные данные пользователя	Логин	Строка	Наличие пользователя в БД	Логическое
		Пароль	Строка	Корректность пароля	Логическое
		Список пользователей	Сущность БД «Пользователи»	Роль	Логическое
	Зарегистрировать пользователя	Логин	Строка	Наличие пользователя в БД	Логическое
		Список пользователей	Сущность БД «Пользователи»		
				Список пользователей	Сущность БД «Пользователи»

2.3.6 Перечень исключительных ситуаций

Исключительная ситуация – это ситуация, при которой система не может выполнить возложенных на неё функций или которая может привести к денормализации работы системы.

В таблице 4 приведён перечень исключительных ситуаций для разрабатываемой системы и описаны реакции системы на их возникновение.

Таблица 4 – Перечень исключительных ситуаций

Название подсистемы	Название исключительной ситуации	Реакция системы
1 Справочная	1.1 Невозможно открыть файл справки	Выдача сообщения «Файл справки повреждён»
	1.2 Невозможно найти файл справки	Выдача сообщения «Отсутствует файл справки»
2 Работы с галереей	2.1 Попытка открытия изображения другого формата	Выдача сообщения «Допустимы только изображения формата PNG»
	2.2 Попытка выбора уже существующего названия	Выдача сообщения «Картинка с заданным названием уже определена в галерее»
3 Взаимодействия с серверной частью	3.1 Невозможно подключиться к серверу	Выдача сообщения «Сервер не доступен. Попробуйте позже»
	3.2 Нарушение при передаче данных	Выдача сообщения «Ошибка при передаче данных. Попробуйте снова»

2.4 Разработка прототипа интерфейса пользователя системы

Интерфейс пользователя является одним из важнейших элементов программы, это та часть программы, которая находится у всех на виду. Недочёты в пользовательском интерфейсе могут серьёзно испортить впечатление даже о самых multifunctional программах. Именно поэтому разработке и проектированию пользовательского интерфейса нужно уделять особое внимание [23].

Процесс создания интерфейса начинается с определения целей проекта, а также внутренних и внешних обстоятельств, которые необходимо принять во внимание. Для того чтобы правильно расставить приоритеты, необходимо учитывать [23]:

- опыт работы пользователей с компьютером, типовые ситуации использования;
- какая информация необходима и когда, какие результаты должны быть получены;
- технологию разработки и платформа, на которой будут работать пользователи.

На рисунке 19 приведён прототип начальной формы приложения. Здесь пользователь должен будет выбрать роль (администратор или игрок), ввести логин и пароль и нажать кнопку «Войти» или «Регистрация», после чего система должна выполнить вход, либо же зарегистрировать нового пользователя.

При авторизации пользователя в роли «Администратор» откроется форма с галереей. Прототип данной экранной формы представлен на рисунке 20. Здесь будут находиться картинки, которые добавлены администратором, но ещё не используются в играх. Администратору будет предоставлена возможность удалить картинку из галереи, а также добавить новую картинку из файла (*.png). При добавлении новой картинки, система

должна будет сама преобразовать её в формат 4х3, после чего картинка в новом формате появится в галерее.

The image shows a prototype of the initial application form for a game called 'PUZZLES'. The title 'PUZZLES' is at the top center in a colorful, bubbly font. Below it are two input fields for 'ЛОГИН' (Login) and 'ПАРОЛЬ' (Password). At the bottom are two buttons: 'РЕГИСТРАЦИЯ' (Registration) and 'ВОЙТИ' (Login).

Рисунок 19 – Прототип начальной формы приложения

The image shows a prototype of the 'Галерея' (Gallery) screen. On the left is a sidebar with four buttons: 'ГАЛЕРЕЯ' (Gallery), 'КАТЕГОРИИ' (Categories), 'УРОВНИ СЛОЖНОСТИ' (Difficulty Levels), and 'ДОБАВЛЕНИЕ ИГРЫ' (Add Game). The main area displays a grid of six puzzle images. At the bottom are two buttons: 'ДОБАВИТЬ КАРТИНКУ' (Add Picture) and 'УДАЛИТЬ' (Delete).

Рисунок 20 – Прототип экранной формы вкладки «Галерея»

Вкладки «Категории» и «Уровни сложности» (см. рисунок 21) позволят производить навигацию по каталогу игр, которые доступны пользователю с ролью «Игрок». Также в функции администратора будет входить удаление игры или изменение её настроек. Для этих действий будет необходимо сначала выбрать пазл (он будет обводиться зелёной рамкой), а затем нажать на соответствующую кнопку. При нажатии на кнопку «Удалить», данная игра будет удаляться из каталога игр, а также из БД. При нажатии на кнопку «Изменить», администратор увидит форму изменения игры, которая изображена на рисунке 22.

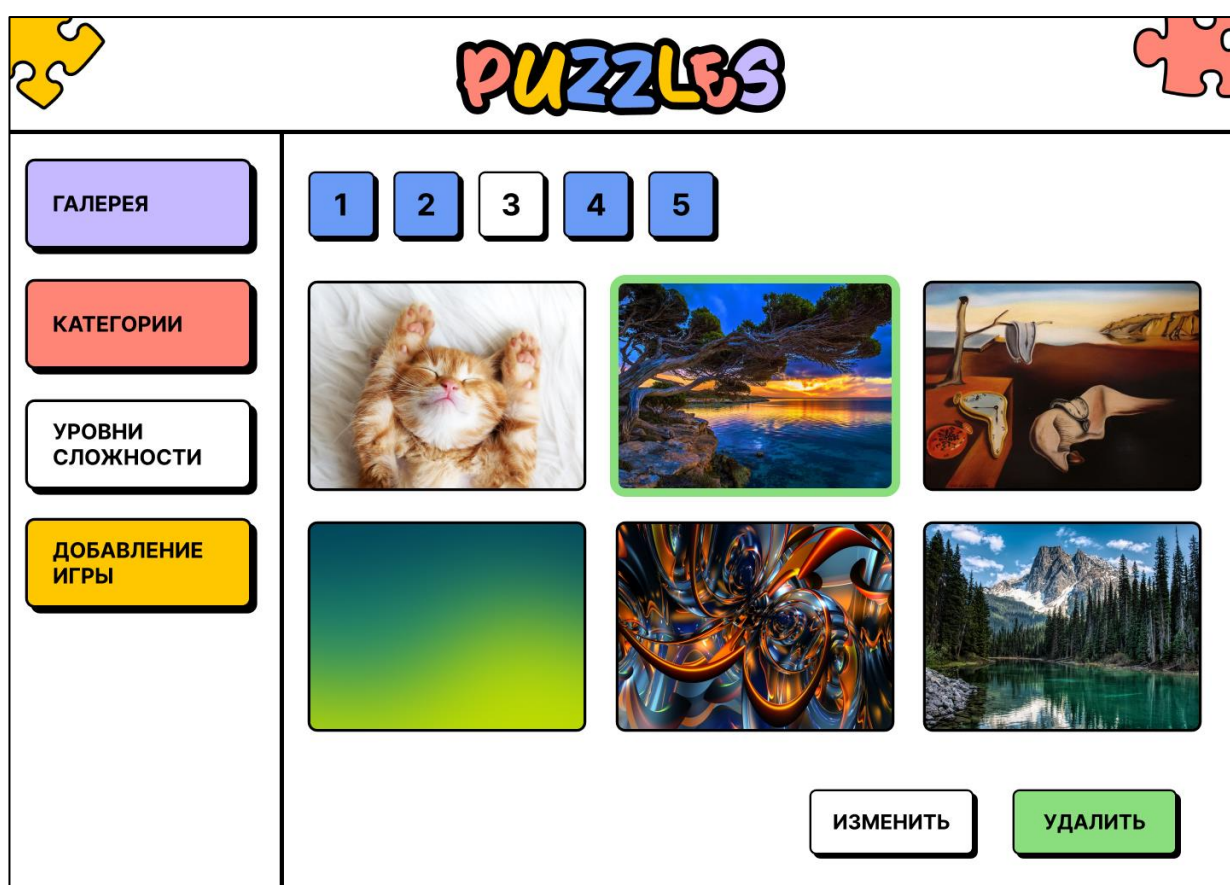


Рисунок 21 – Прототип экранной формы вкладки «Уровни сложности»

В форме изменения игры, администратор сможет изменять параметры (форму детали, размерность, тип сборки, категорию и уровень сложности), указанные ранее для этой игры. После чего необходимо будет нажать кнопку «Сохранить». После сохранения изменённая игра появится в каталоге игр.

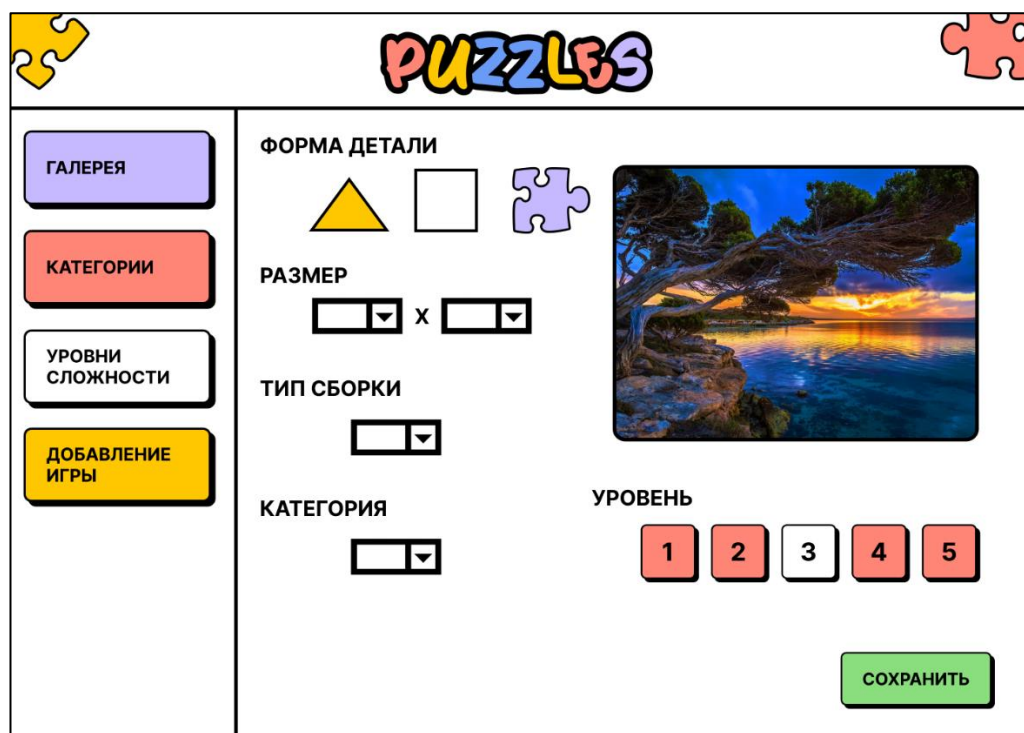


Рисунок 22 – Прототип экранной формы изменения игры

Во вкладке «Добавление игры» (см. рисунок 23), у администратора будет возможность добавить в каталог игр новый пазл. Для этого нужно будет выбрать картинку из галереи, задать для неё уровень сложности и остальные необходимые параметры (размерность, тип сборки, категорию) и нажать кнопку «Добавить».

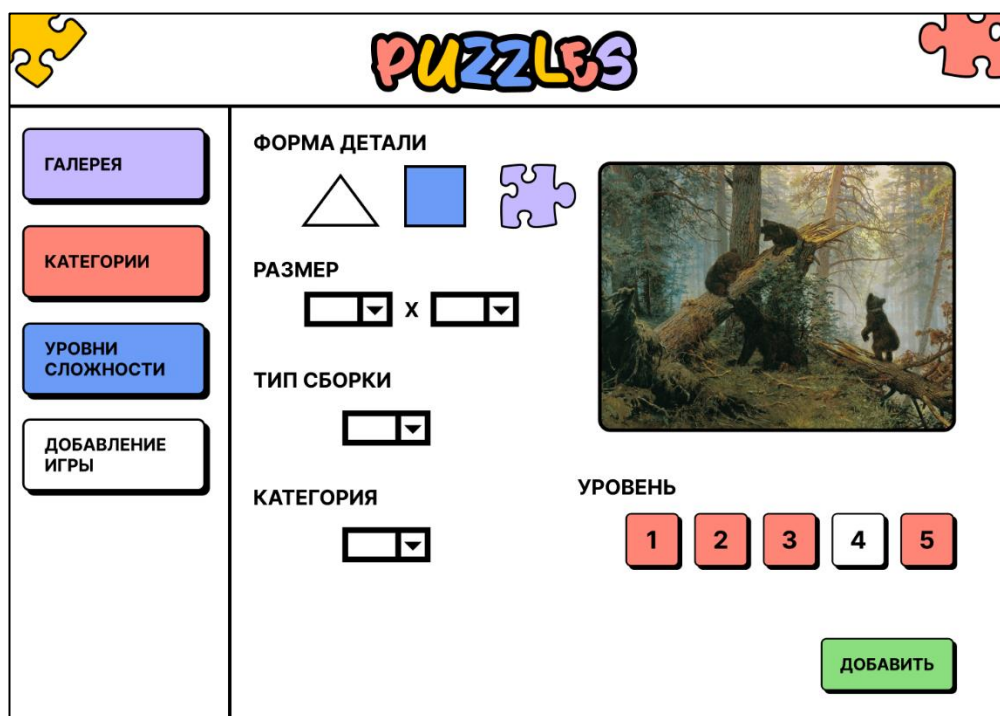


Рисунок 23 – Прототип экранной формы вкладки «Добавление игры»

При выборе роли «Игрок» система откроет вкладку «Сохранённые игры», где будут храниться все пазлы, которые он не закончил собирать (см. рисунок 24). В нижнем правом углу у каждой картинке будет отображен уровень сложности для данной игры. Если у пользователя сохранённых игр не будет или он только зарегистрировался в системе, то во вкладке «Сохранённые игры» будет надпись: «Сохранённых игр нет».

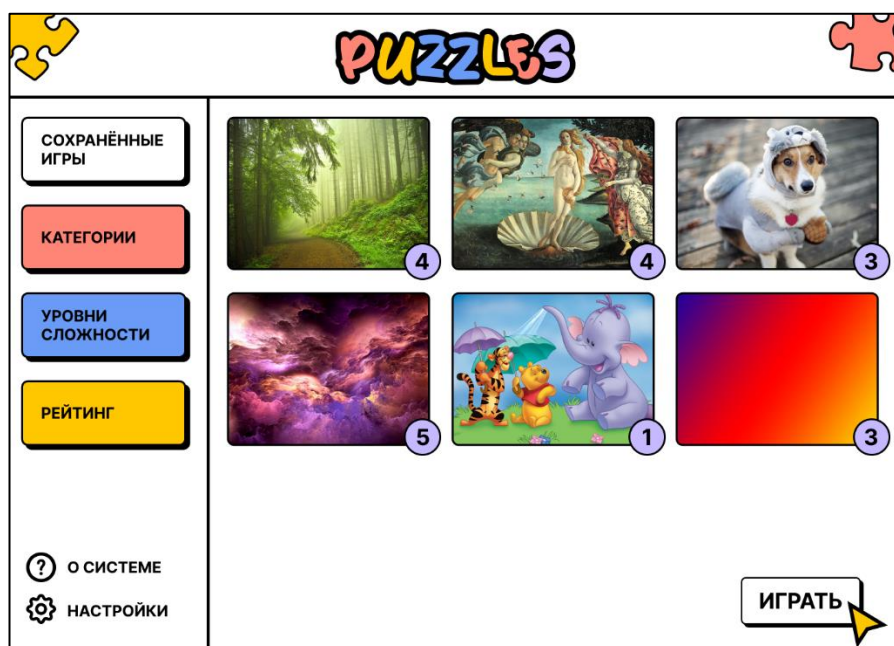


Рисунок 24 – Прототип экранной формы вкладки «Сохранённые игры»

Прототип вкладки «Категории» представлен на рисунке 25.

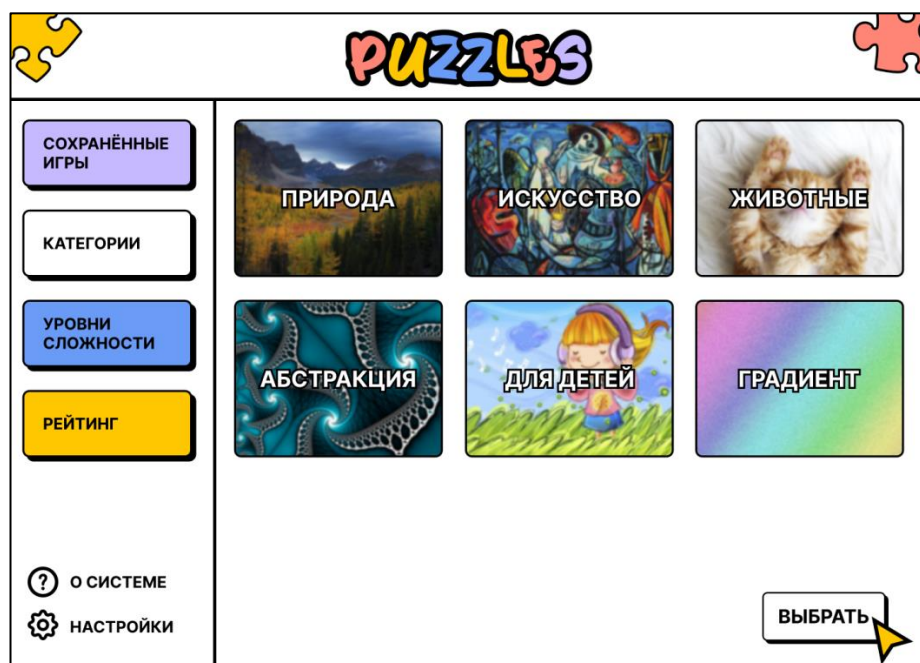


Рисунок 25 – Прототип экранной формы вкладки «Категории»

В данной вкладке все картинки будут распределены по категориям. После выбора конкретной категории, пользователь сможет выбрать интересующий его пазл из возможных картинок данной категории (см. рисунок 26).

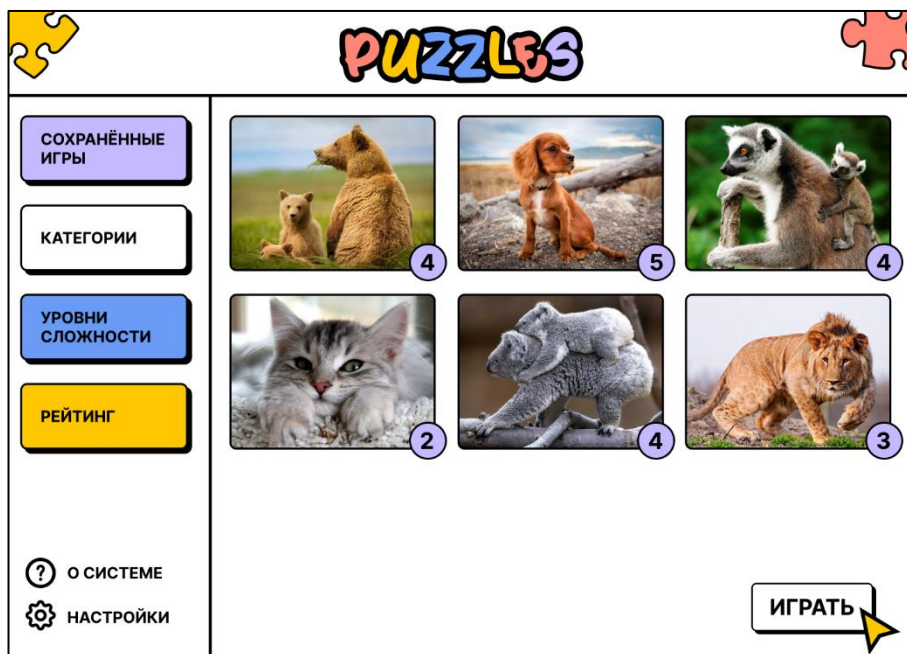


Рисунок 26 – Прототип экранной формы конкретной категории

Во вкладке «Рейтинг», пользователь сможет изучить список из 10 игроков, занимающих лидерские позиции.



Рисунок 27 – Прототип экранной формы вкладки «Рейтинг»

Во вкладке «Уровни сложности» игрок сможет выбрать пазл, исходя из необходимого ему уровня сложности. Переключаться между уровнями можно будет через специальные кнопки над картинками. Прототип вкладки «Уровни сложности» представлен на рисунке 28.

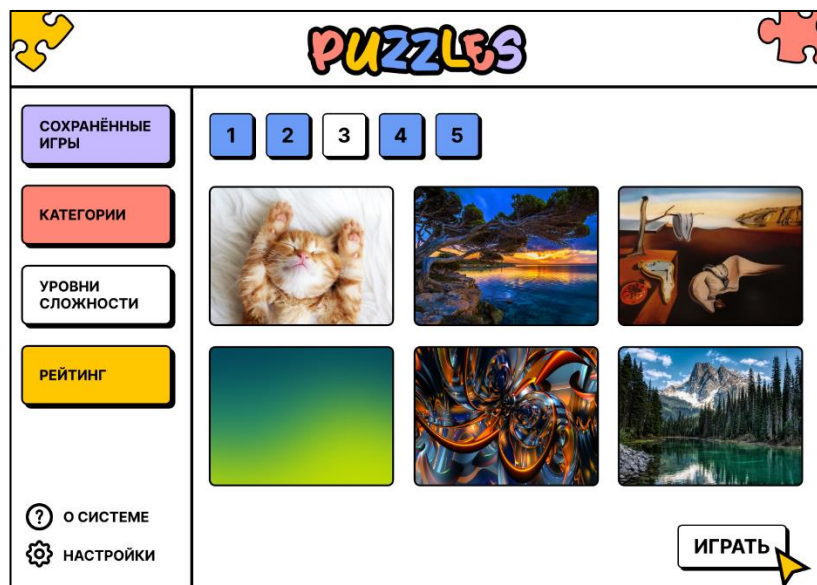


Рисунок 28 – Прототип экранной формы вкладки «Уровни сложности»

При нажатии на кнопку «О системе», система откроет информацию по данному приложению. При нажатии на кнопку «Настройки» система должна открыть окно, где игрок сможет включить/выключить звук или музыкальное сопровождение, а также прочитать всю интересующую его информацию об разработчиках игры.

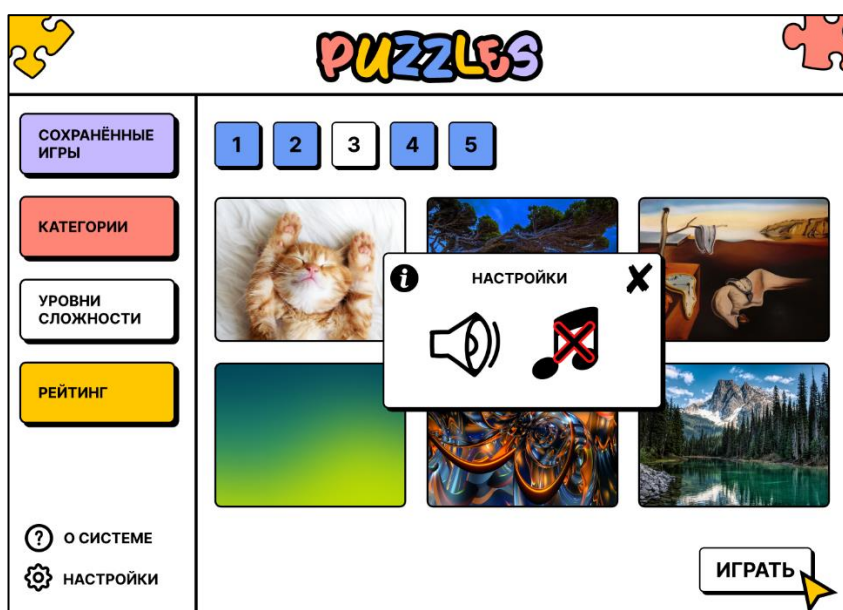


Рисунок 29 – Прототип экранной формы настроек приложения

Для начала игры пользователю необходимо выбрать картинку (она подсветится зелёным), а затем нажать кнопку «Играть» (см. рисунок 30). После этого, система откроет игровое поле.

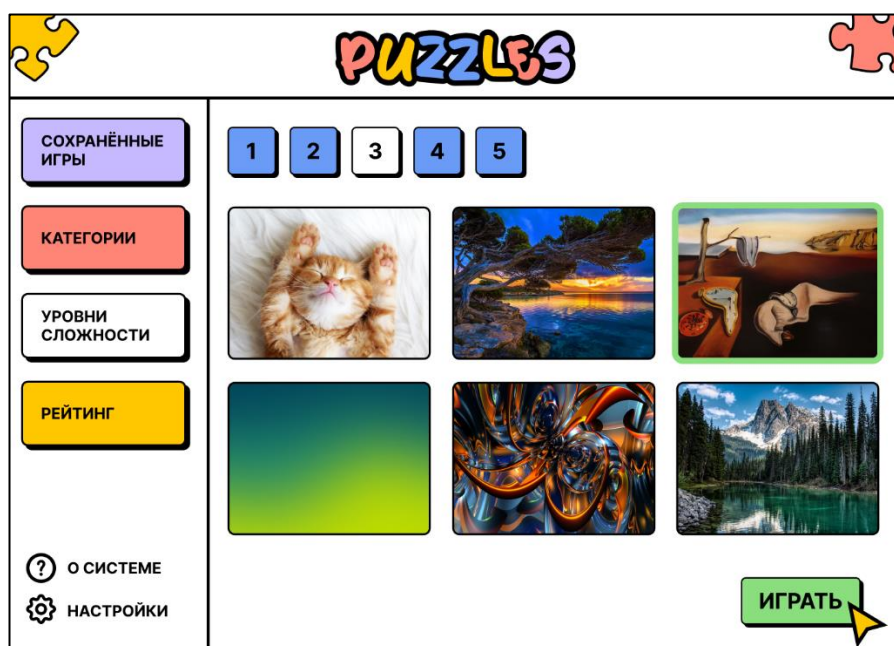


Рисунок 30 – Прототип экранной формы с визуализацией выбора пазла

На рисунке 31 показан прототип игрового поля с типом сборки «на ленте».

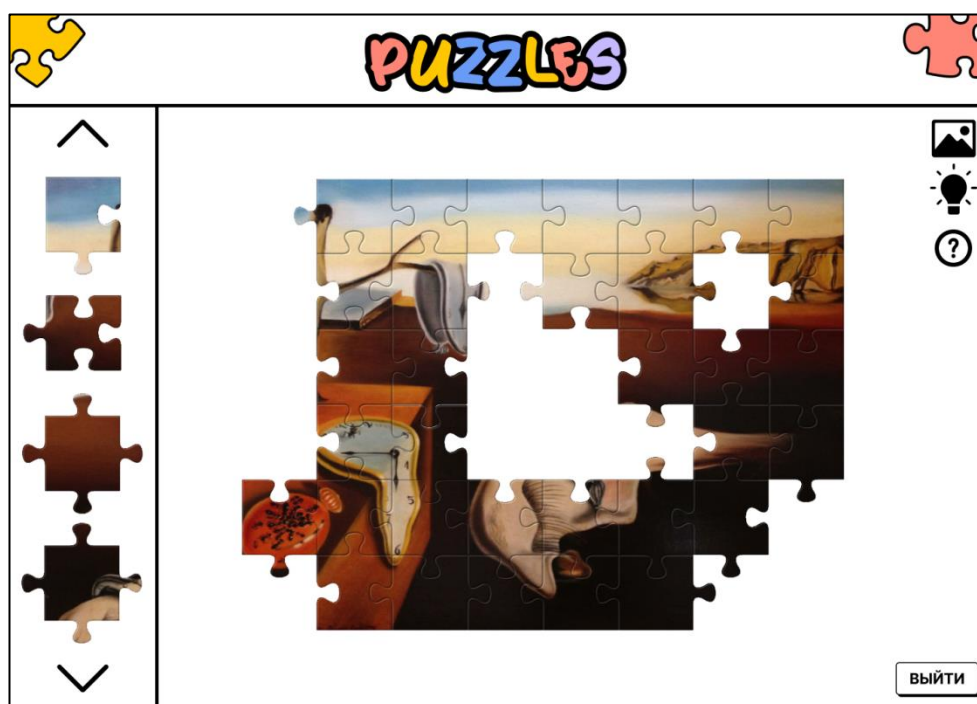


Рисунок 31 – Прототип экранной формы игрового поля с типом сборки «на ленте»

Для того, чтобы пользователь смог увидеть исходную картинку, ему необходимо будет нажать на иконку «Картинка», в правом верхнем углу. Пример просмотра исходной картинки показан на рисунке 32.

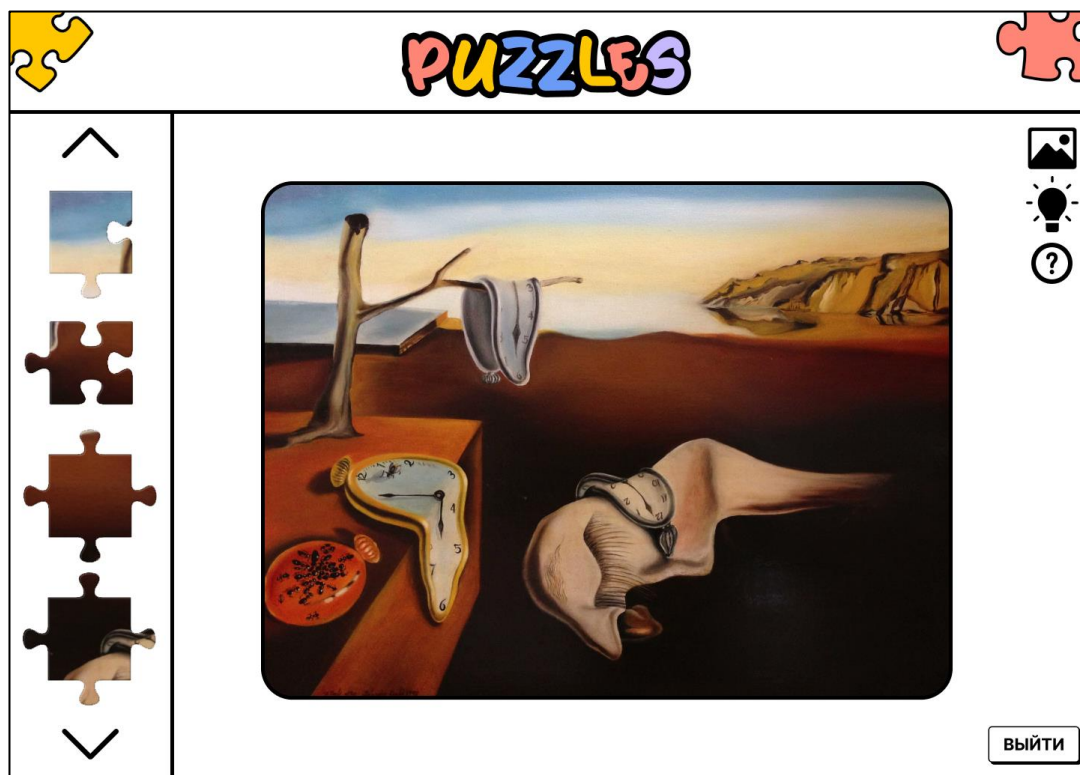


Рисунок 32 – Прототип экранной формы просмотра исходной картинки

Взятие подсказки будет происходить следующим образом: пользователь нажимает на иконку «Подсказка» в правом верхнем углу, затем система подсвечивает фрагмент и место, на которое он должен встать зелёным цветом. Пользователю необходимо будет перенести данный фрагмент с ленты на подсвечиваемое место (см. рисунок 33).

При нажатии иконки «Знак вопроса», система откроет пользователю страницу с информацией о данном приложении.

На рисунках 34 и 35 изображены прототипы игрового поля с типом сборки «в куче» и «вперемешку» соответственно. При типе сборки «в куче», пользователю необходимо будет искать подходящие друг к другу фрагменты. При типе сборки «вперемешку», игрок должен будет выбрать два фрагмента, которые он хочет поменять местами. При правильной сборке фрагменты будут склеиваться.

После завершения игры, пользователю будут начисляться баллы, которые в дальнейшем учитываются в рейтинге.

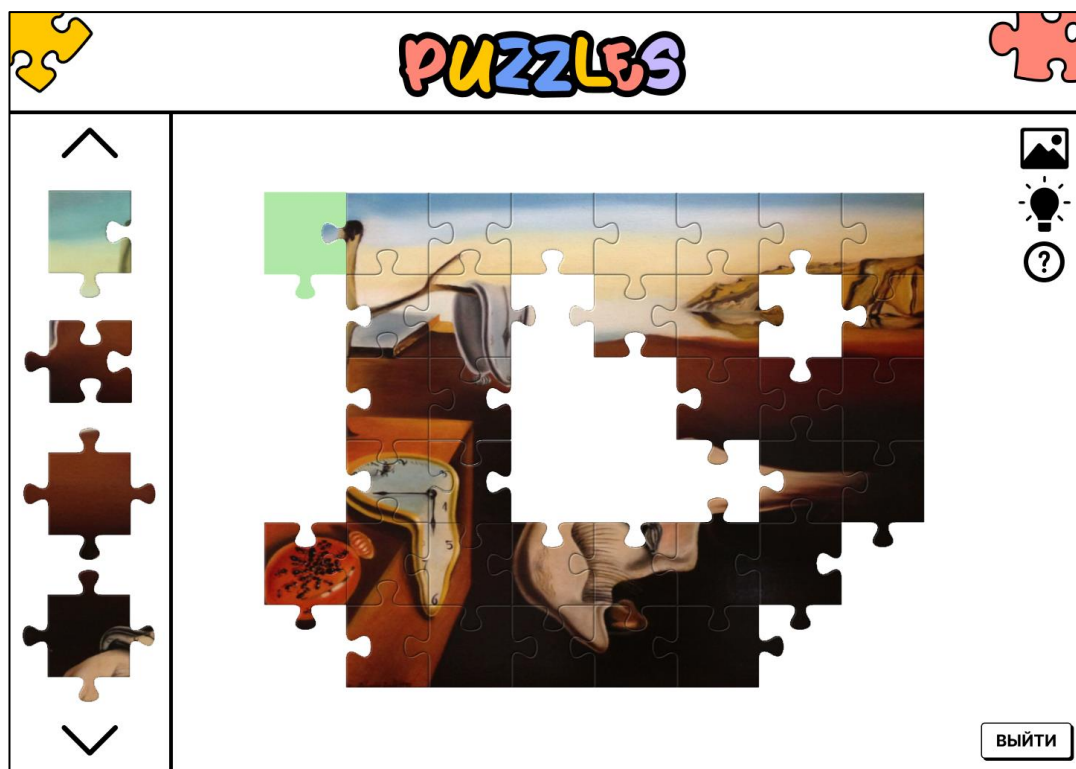


Рисунок 33 – Прототип экранной формы взятия подсказки

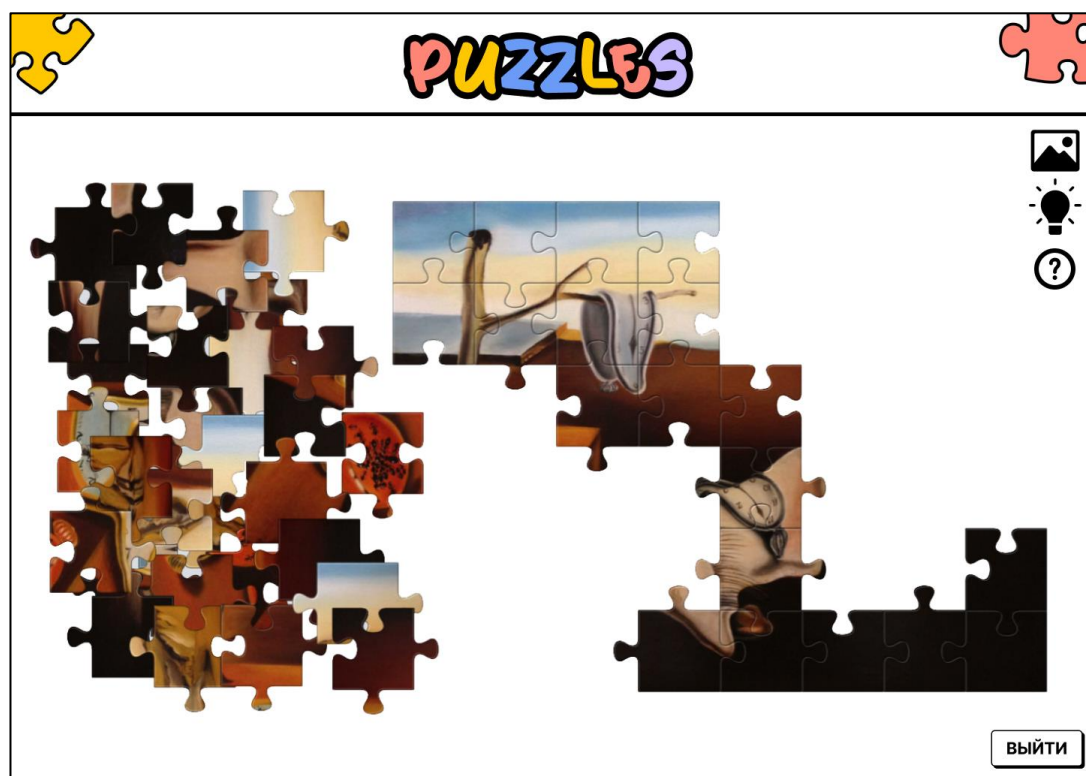


Рисунок 34 – Прототип экранной формы игрового поля
с типом сборки «В куче»

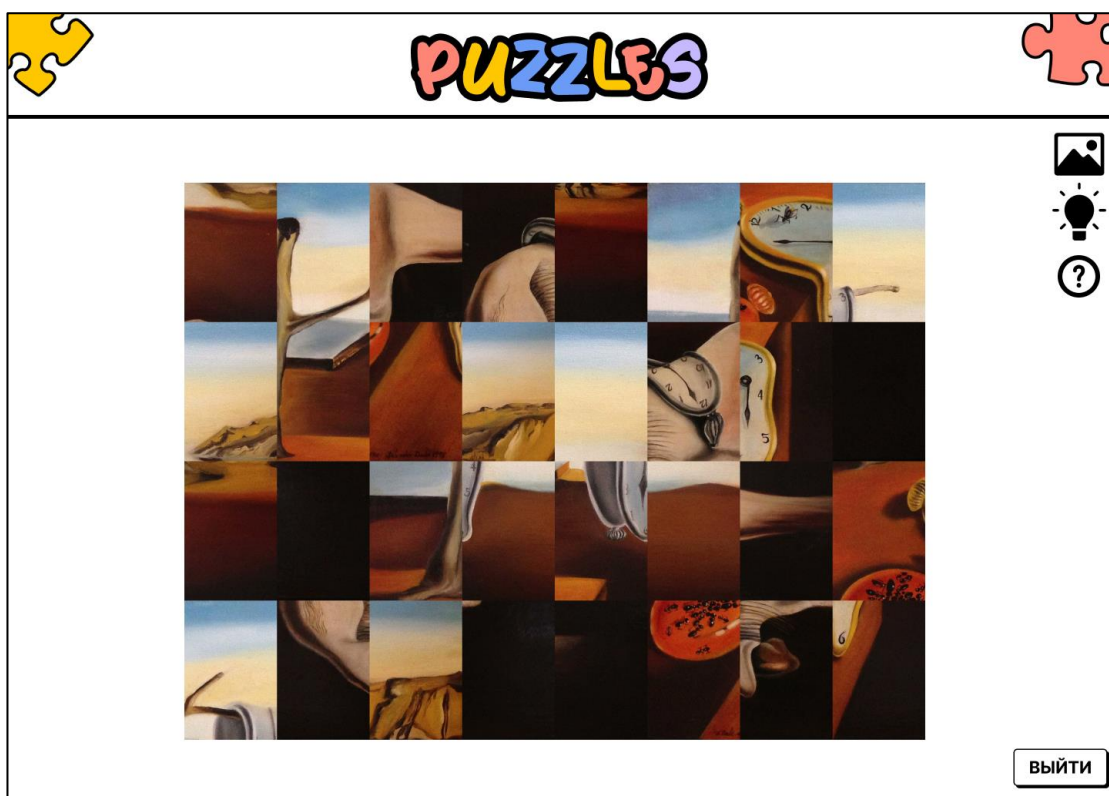


Рисунок 35 – Прототип экранной формы игрового поля
с типом сборки «Вперемешку»

Если пользователь захочет выйти из текущей игры, ему необходимо будет нажать на кнопку «Выйти», после чего система предложит ему сохранить игру.

На рисунке 36 приведена навигационная модель для администратора, на рисунке 37 приведена навигационная модель для игрока.

2.5 Разработка информационно-логического проекта системы

Цель инфологического проектирования является обеспечение наиболее естественных для человека способов сбора и представления той информации, которую предполагается хранить в создаваемой базе данных. Поэтому инфологическая модель данных построена по аналогии с естественным языком. Основными конструктивными элементами инфологических моделей являются сущности, связи между ними и их атрибуты [24].

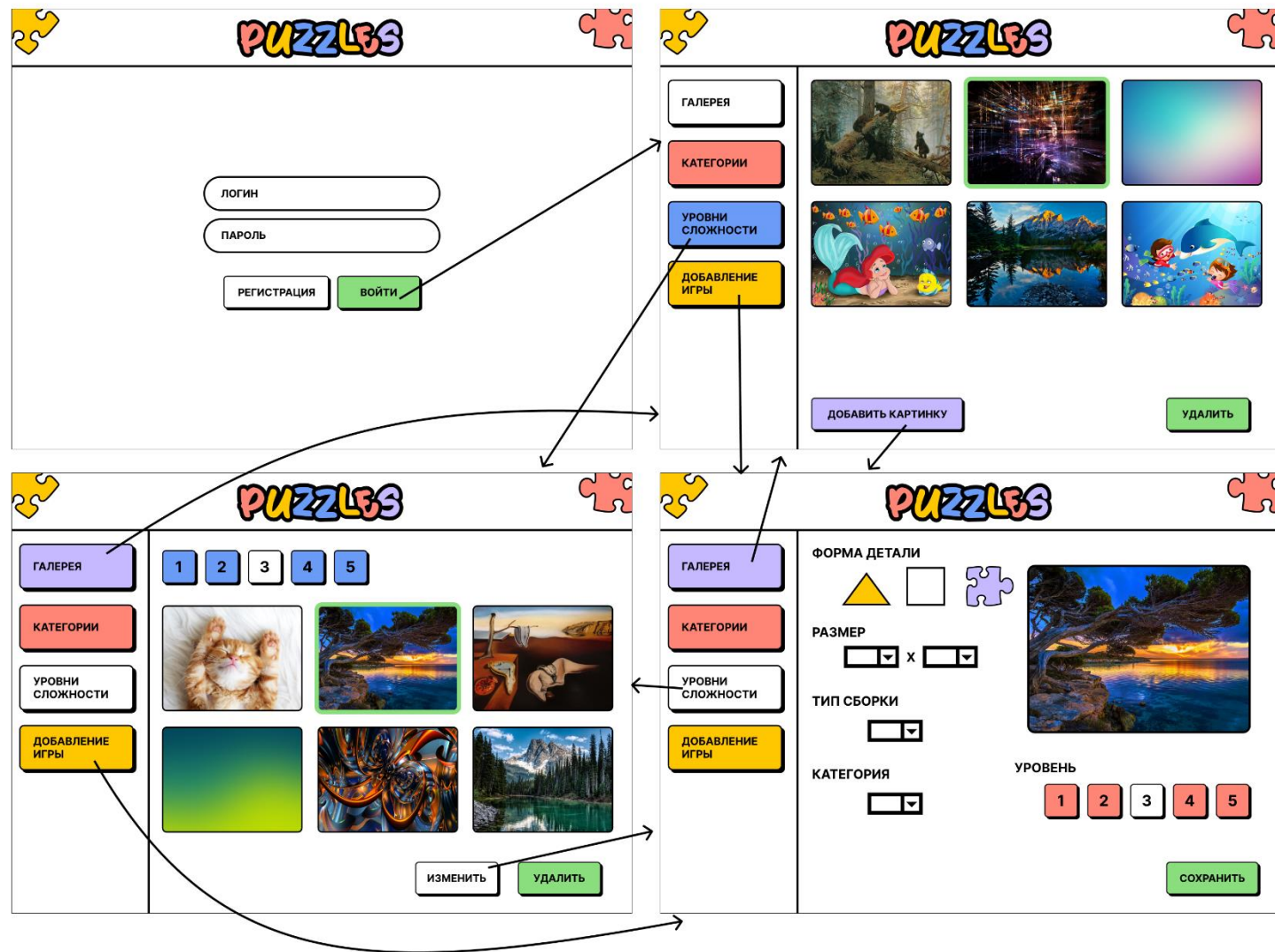


Рисунок 36 – Навигационная модель приложения для администратора

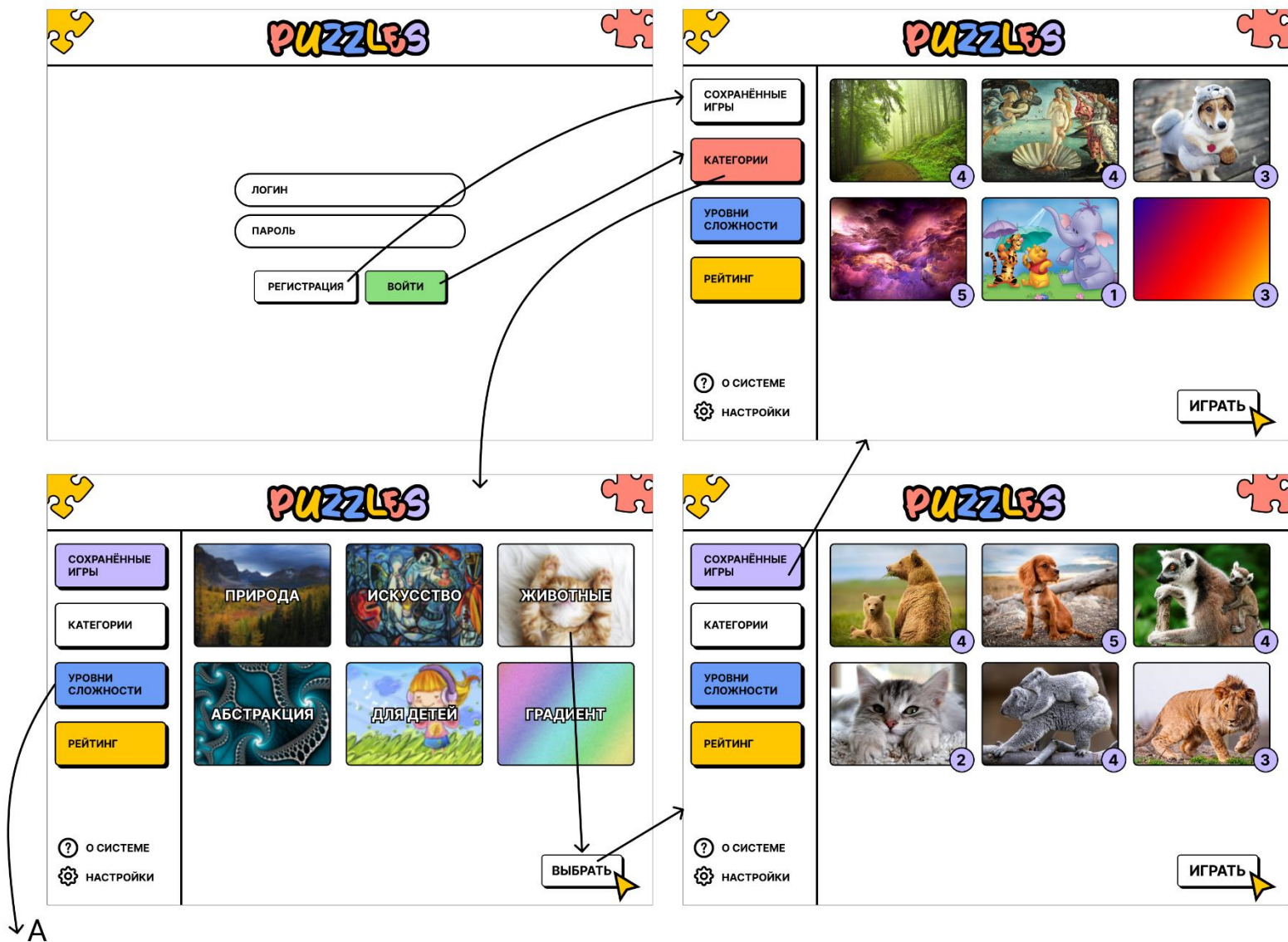


Рисунок 37 – Навигационная модель приложения для игрока (начало)

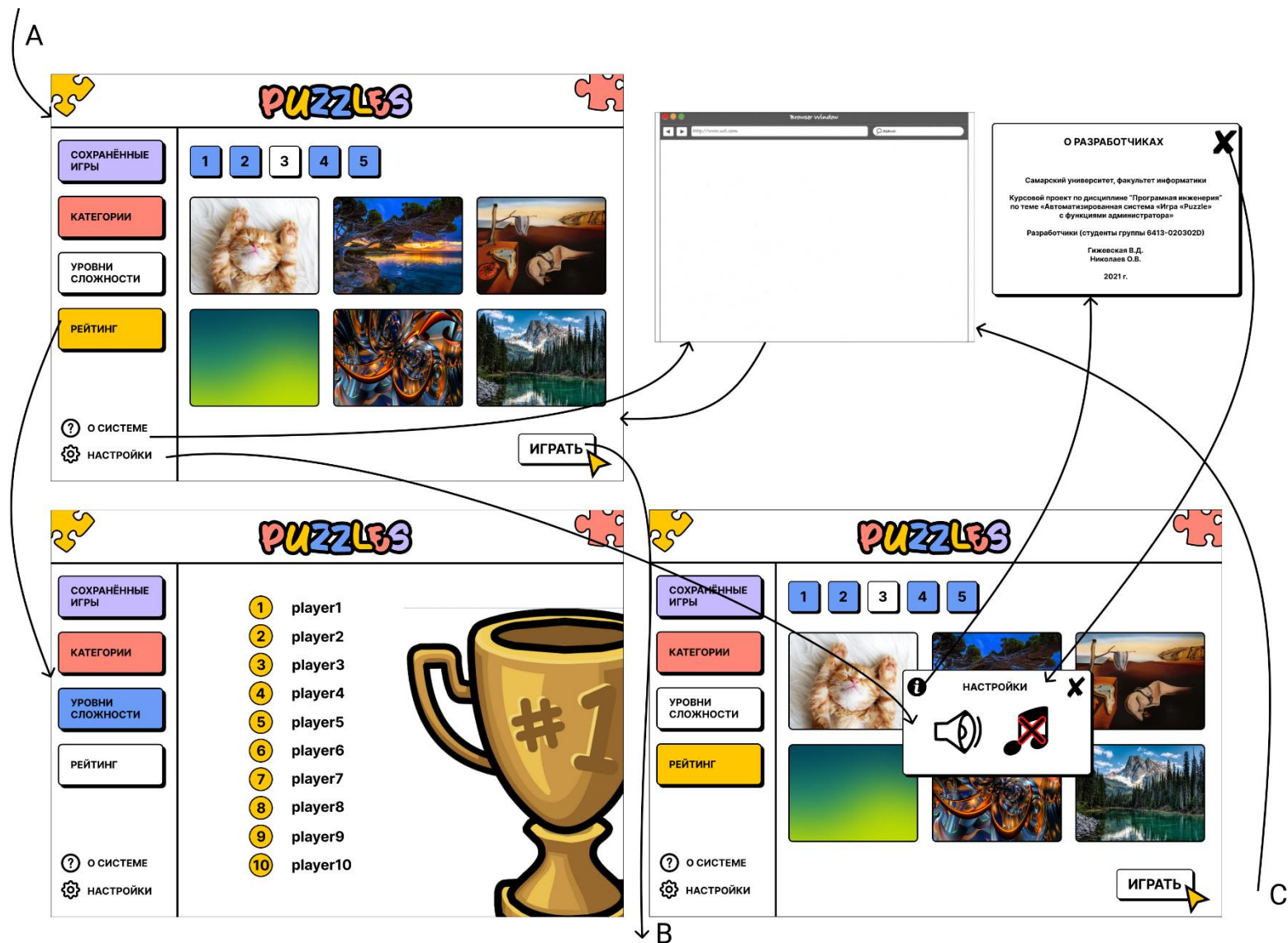


Рисунок 37 – Навигационная модель приложения для игрока (продолжение)

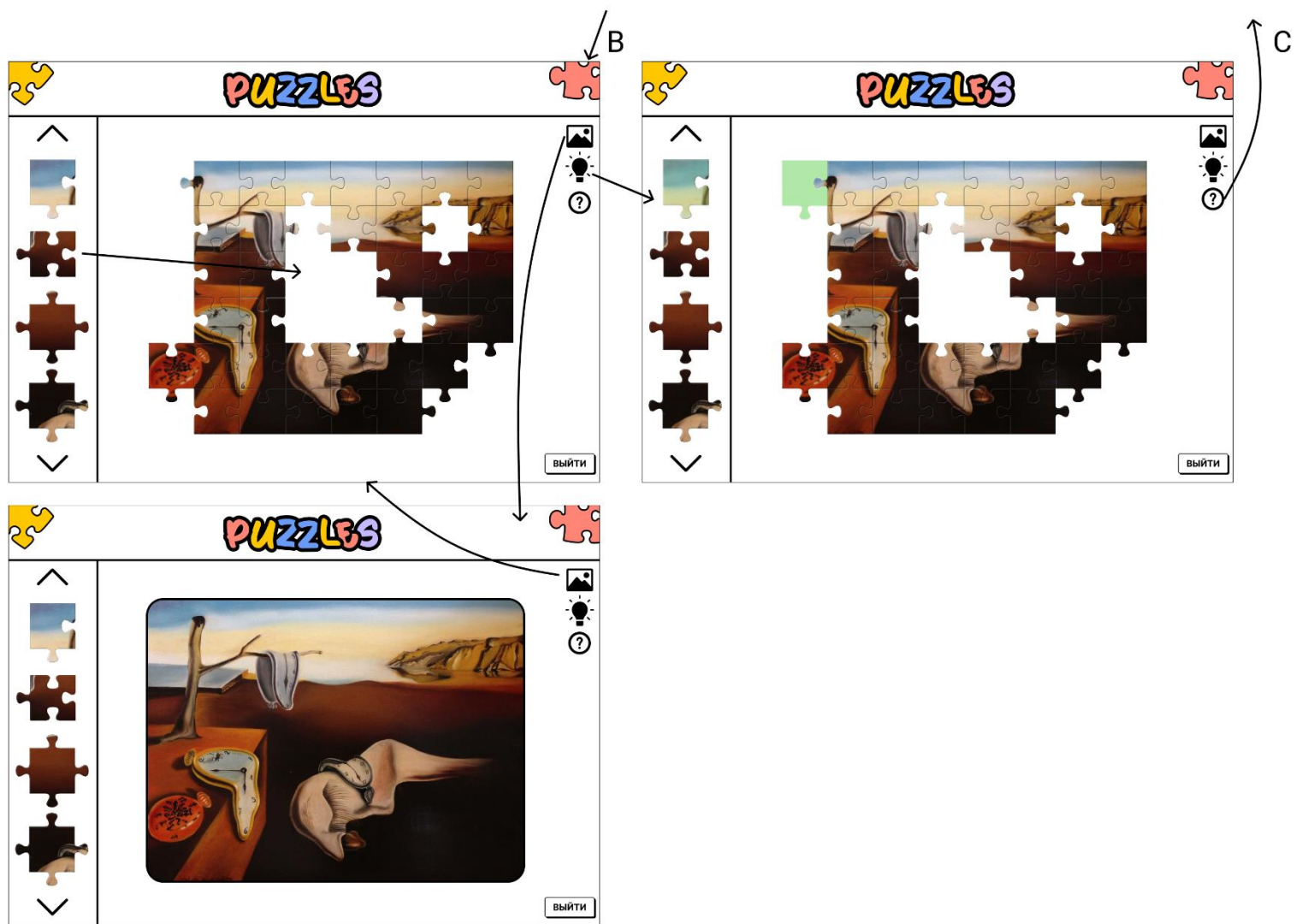


Рисунок 37 – Навигационная модель приложения для игрока (окончание)

2.5.1 Язык UML

Для специфицирования (построения точных, недвусмысленных и полных моделей) системы и ее документирования используется унифицированный язык моделирования UML.

UML (англ. Unified Modeling Language – унифицированный язык моделирования) – язык графического описания для объектного моделирования в области разработки программного обеспечения. UML является языком широкого профиля, это – открытый стандарт, использующий графические обозначения для создания абстрактной модели системы, называемой UML-моделью. UML был создан для определения, визуализации, проектирования и документирования, в основном, программных систем. UML не является языком программирования, но на основании UML-моделей возможна генерация кода [25].

Использование UML не ограничивается моделированием программного обеспечения. Его также используют для моделирования бизнес-процессов, системного проектирования и отображения организационных структур.

UML позволяет разработчикам программного обеспечения достигнуть соглашения в графических обозначениях для представления общих понятий, таких как класс, компонент, обобщение, агрегация и поведение и больше сконцентрироваться на проектировании и архитектуре.

2.5.2 Диаграмма вариантов использования

Диаграмма вариантов использования представляет собой наиболее общую концептуальную модель сложной системы, которая является исходной для построения всех остальных диаграмм. На ней изображаются отношения между актёрами и вариантами использования.

Актёр (actor) – согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними.

Вариант использования – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актёрами.

Цель спецификации варианта использования заключается в том, чтобы зафиксировать некоторый аспект или фрагмент поведения проектируемой системы без указания особенностей реализации данной функциональности. В этом смысле каждый вариант использования соответствует отдельному сервису, который предоставляет моделируемая система по запросу актёра. Сервис, который инициализируется по запросу актёра, должен представлять собой законченную последовательность действий. Это означает, что после того как система закончит обработку запроса актёра, она должна возвратиться в исходное состояние, в котором снова готова к выполнению следующих запросов.

На рисунке 38 приведена общая диаграмма вариантов использования. Игрок может посмотреть Пользователь может регистрироваться и авторизоваться в системе (ввести логин и пароль). Система проверит введённые данные и настроит интерфейс пользователя на соответствующую роль. Игрок может посмотреть справочную информацию о системе и о разработчиках. В системе предусмотрены две роли пользователя: администратор и игрок.

На рисунке 39 приведена диаграмма вариантов использования для пользователя с ролью «Администратор». Администратор может добавлять игру, для этого ему необходимо выбрать уровень сложности, картинку из галереи, задать количество пазлов по горизонтали и вертикали, выбрать категорию и способ расположения фрагментов. Кроме того, он может изменить параметры игры. Также администратор может работать с галереей, ему доступно удаление и добавление изображений.



Рисунок 38 – Диаграмма вариантов использования системы (Пользователь)

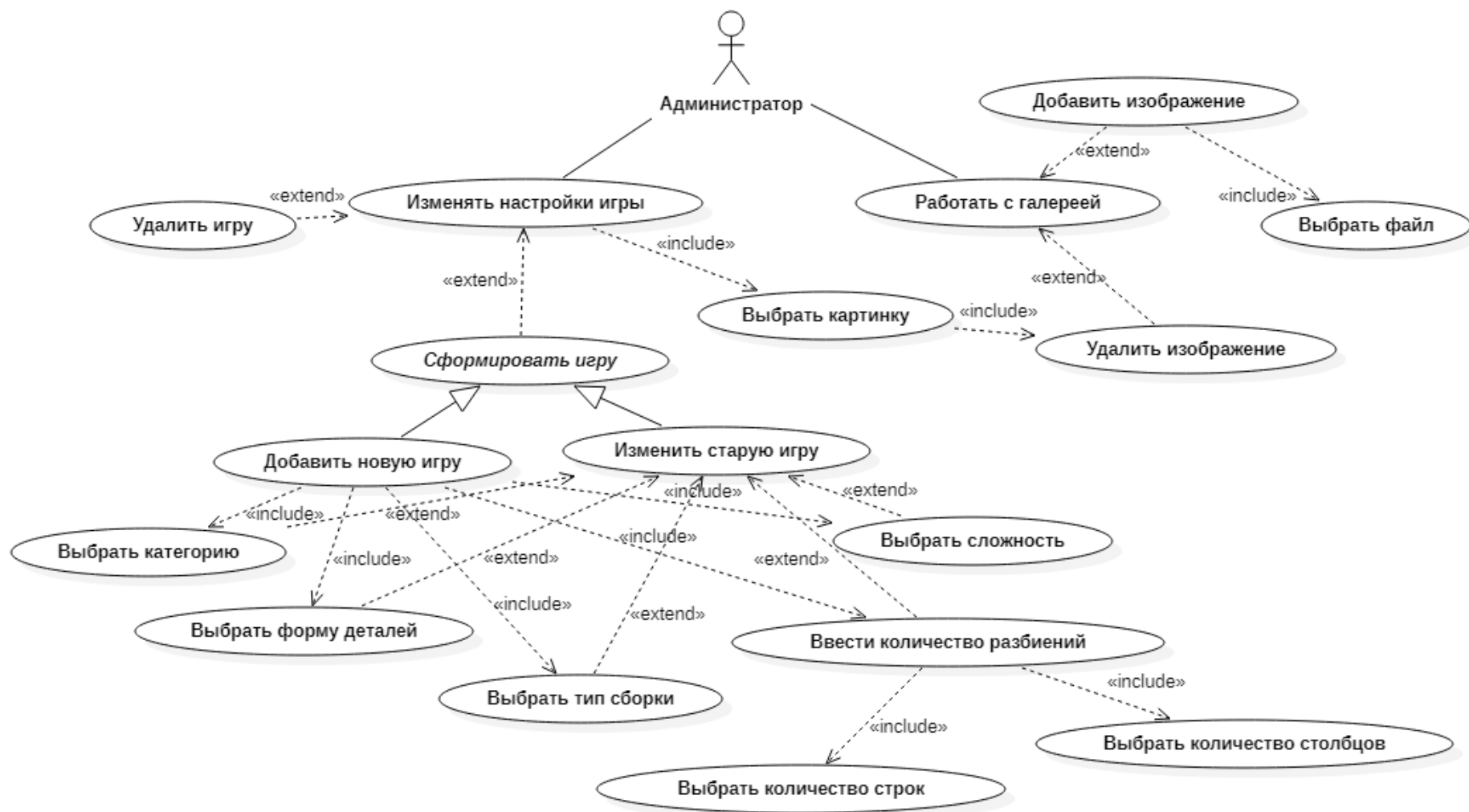


Рисунок 39 – Диаграмма вариантов использования системы (для роли «Администратор»)

На рисунке 40 приведена диаграмма вариантов использования системы для пользователя с ролью «Игрок». Игрок может выбрать игру, исходя из желаемых параметров (по уровню сложности и по категории картинки). Пользователь с ролью «Игрок» имеет возможность посмотреть исходную картинку, взять подсказку, переместить фрагмент с ленты на поле, из кучи на поле, а также ему доступно перемещение фрагментов на игровом поле.

2.5.3 Сценарии

Сценарий (scenario) - определённая последовательность действий, которая описывает действия актёров и поведение моделируемой системы в форме обычного текста [26].

В контексте языка UML сценарий используется для дополнительной иллюстрации взаимодействия актёров и вариантов использования.

Рассмотрим несколько сценариев.

Вариант использования: «Выбрать количество столбцов».

Краткое описание. Позволяет администратору необходимое количество столбцов при создании новой игры. Включается в вариант использования «Добавить новую игру».

Актант. Администратор.

Предусловия. Компьютер включён, вход в систему в роли администратора выполнен успешно. На экране показан интерфейс добавления новой игры. Администратор выбрал пункт меню «Добавление игры».

Основной поток событий.

- 1 Система выводит на экран форму добавления новой игры (см. рисунок 23). На форме отображаются поля «Форма детали», «Размер», «Тип сборки», «Категория», «Уровень», «Выбор картинки», кнопка «Добавить», справа находится меню с вкладками «Галерея», «Категории», «Уровни сложности».



Рисунок 40 – Диаграмма вариантов использования системы (для роли «Игрок»)

2 Игрок щелкает левой кнопкой мыши (ЛКМ) на правый выпадающий список поля «Размер».

A1: Администратор выбирает ЛКМ форму детали в поле «Форма детали».

A2: Администратор щелкает ЛКМ на левый выпадающий список поля «Размер».

A3: Администратор щелкает ЛКМ на выпадающий список поля «Тип сборки».

A4: Администратор щелкает ЛКМ на выпадающий список поля «Категория».

A5: Администратор щелкает ЛКМ в любом месте поля «Выбор картинки».

A6: Администратор выбирает ЛКМ уровень сложности в поле «Уровень».

A7: Администратор выбирает ЛКМ пункт меню «Галерея».

A8: Администратор выбирает ЛКМ пункт меню «Категории».

A9: Администратор выбирает ЛКМ пункт меню «Уровни сложности».

3 Система раскрывает выпадающий список с доступными размерами количества столбцов (от 4 до 12).

4 Игрок выбирает количество столбцов.

5 Система закрывает список и запоминает выбранное значение. Вариант использования завершается успешно.

Альтернативы.

A1: Администратор выбирает ЛКМ форму детали в поле «Форма детали».

A1.1 Переход к варианту использования «Выбрать форму детали».

A2 Администратор щелкает ЛКМ на левый выпадающий список поля «Размер».

А2.1 Переход к варианту использования «Задать количество строк».

А3: Администратор щелкает ЛКМ на выпадающий список поля «Тип сборки».

А3.1 Переход к варианту использования «Выбрать тип сборки».

А4: Администратор щелкает ЛКМ на выпадающий список поля «Категория».

А4.1 Переход к варианту использования «Выбрать категорию».

А5: Администратор щелкает ЛКМ в любом месте поля «Выбор картинки».

А5.1 Переход к варианту использования «Выбрать картинку».

А6: Администратор выбирает ЛКМ уровень сложности в поле «Уровень».

А6.1 Переход к варианту использования «Выбрать уровень сложности».

А7: Администратор выбирает ЛКМ пункт меню «Галерея».

А7.1 Система закрывает форму «Добавление игры».

А7.2 Система открывает форму «Галерея».

А7.3 Вариант использования завершается.

А8: Администратор выбирает ЛКМ пункт меню «Категории».

А8.1 Система закрывает форму «Добавление игры».

А8.2 Система открывает форму «Категории».

А8.3 Вариант использования завершается.

А9: Администратор выбирает ЛКМ пункт меню «Уровни сложности».

А9.1 Система закрывает форму «Добавление игры».

А9.2 Система открывает форму «Уровни сложности».

А9.3 Вариант использования завершается.

Постусловия. При успешном завершении на экране – форма приложения с добавленными играми.

Вариант использования: «Задать уровень сложности».

Краткое описание. Позволяет администратору определить игровую сложность. Включается в вариант использования «Добавить новую игру».

Актант. Администратор.

Предусловия. Компьютер включён, вход в систему выполнен успешно. На экране показан интерфейс добавления новой игры. Администратор выбрал пункт меню «Добавление игры».

Основной поток событий.

1 Система выводит на экран форму формирования игры (см. рисунок 1). На форме отображаются поля «Форма детали», «Размер», «Тип сборки», «Категория», «Уровень», «Выбор картинки», кнопка «Добавить», справа находится меню с вкладками «Галерея», «Категории», «Уровни сложности». При изменении существующей игры поля заполняются установленными ранее значениями.

2 Игрок щелкает ЛКМ на необходимое значение в поле «Уровни сложности» (количество уровней сложности 5).

A1: Администратор выбирает ЛКМ форму детали в поле «Форма детали».

A2: Администратор щелкает ЛКМ на левый выпадающий список поля «Размер».

A3: Администратор щелкает ЛКМ на правый выпадающий список поля «Размер».

A4: Администратор щелкает ЛКМ на выпадающий список поля «Тип сборки».

A5: Администратор щелкает ЛКМ на выпадающий список поля «Категория».

A6: Администратор щелкает ЛКМ в любом месте поля «Выбор картинки».

A7: Администратор выбирает ЛКМ пункт меню «Галерея».

A8: Администратор выбирает ЛКМ пункт меню «Категории».

А9: Администратор выбирает ЛКМ пункт меню «Уровни сложности».

3 Система выделяет выбранное значение белым цветом.

4 Система запоминает выбранное значение. Вариант использования завершается успешно.

Альтернативы.

А1: Администратор выбирает ЛКМ форму детали в поле «Форма детали».

А1.1 Переход к варианту использования «Выбрать форму детали».

А2 Администратор щелкает ЛКМ на левый выпадающий список поля «Размер».

А2.1 Переход к варианту использования «Задать количество строк».

А3 Администратор щелкает ЛКМ на правый выпадающий список поля «Размер».

А3.1 Переход к варианту использования «Задать количество столбцов».

А4: Администратор щелкает ЛКМ на выпадающий список поля «Тип сборки».

А4.1 Переход к варианту использования «Выбрать тип сборки».

А5: Администратор щелкает ЛКМ на выпадающий список поля «Категория».

А5.1 Переход к варианту использования «Выбрать категорию».

А6: Администратор щелкает ЛКМ в любом месте поля «Выбор картинки».

А6.1 Переход к варианту использования «Выбрать картинку».

А7: Администратор выбирает ЛКМ пункт меню «Галерея».

А7.1 Система закрывает форму «Добавление игры».

А7.2 Система открывает форму «Галерея».

А7.3 Вариант использования завершается.

А8: Администратор выбирает ЛКМ пункт меню «Категории».

А8.1 Система закрывает форму «Добавление игры».

А8.2 Система открывает форму «Категории».

А8.3 Вариант использования завершается.

А9: Администратор выбирает ЛКМ пункт меню «Уровни сложности».

А9.1 Система закрывает форму «Добавление игры».

А9.2 Система открывает форму «Уровни сложности».

А9.3 Вариант использования завершается.

Постусловия. При успешном завершении на экране – форма приложения с добавленными играми.

2.5.4 Диаграмма состояний

Главное назначение диаграммы состояний – описать возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение моделируемой системы в течение всего ее жизненного цикла. Диаграмма состояний представляет динамическое поведение сущностей, на основе спецификации их реакции на восприятие некоторых конкретных событий. Системы, которые реагируют на внешние действия от других систем или от пользователей, иногда называют реактивными. Если такие действия инициируются в произвольные случайные моменты времени, то говорят об асинхронном поведении модели [27].

На рисунке 41 представлена обобщённая диаграмма состояний. Цепочка переходов начинается с состояния «Запуск приложения», в котором осуществляется соединение с БД. В случае если система не смогла установить соединение с БД, она переходит в состояние «Завершение работы с приложением».

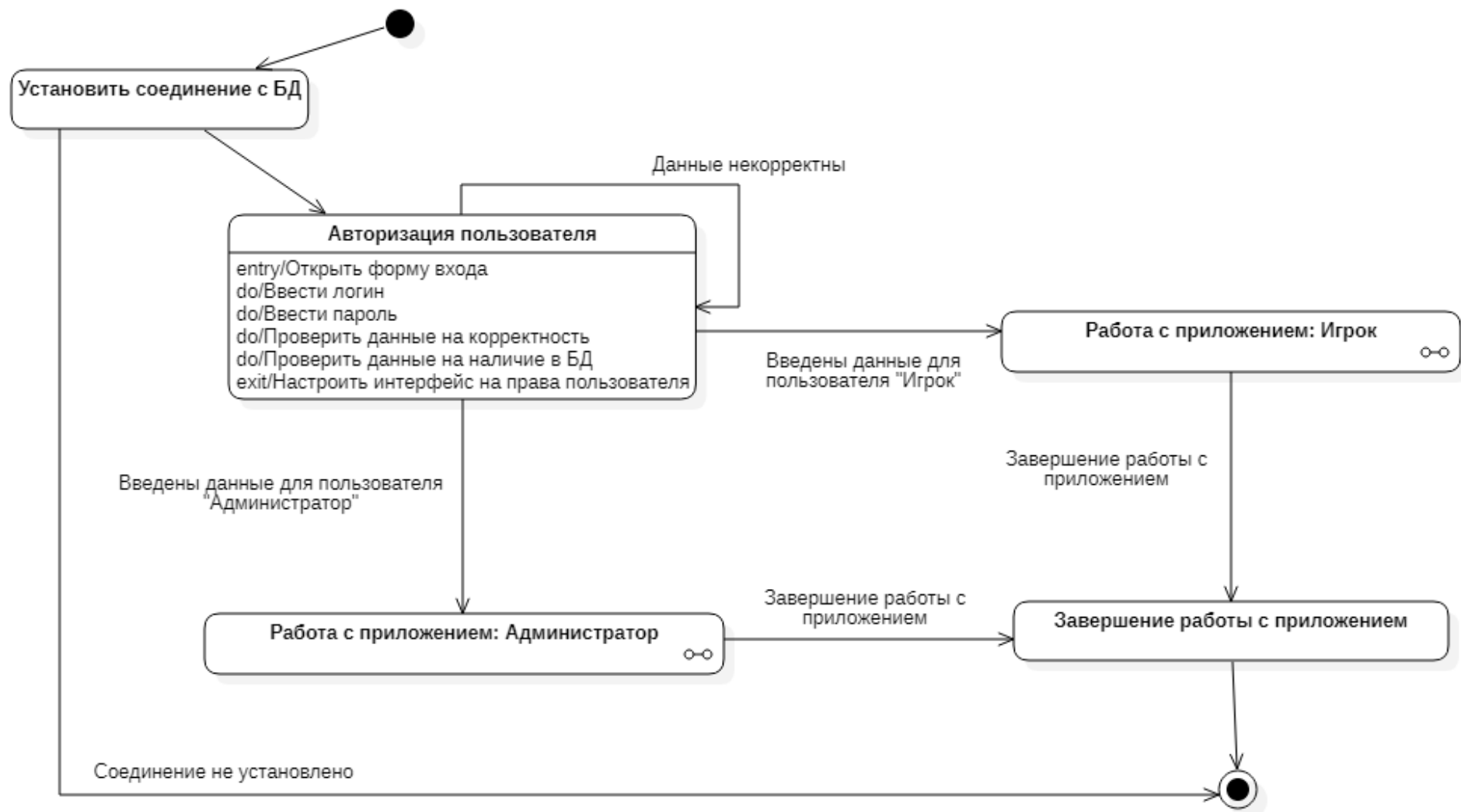


Рисунок 41 – Диаграмма состояний системы (общая часть)

Далее пользователь переходит к авторизации, где ему предлагается ввести его учётные данные. Система проверяет учётные данные и в случае их корректности и наличие в БД настраивает интерфейс на права пользователя и допускает к работе с приложением. В случае некорректности введённых данных, пользователю предлагается ввести их повторно. В аналогичное состояние пользователь может перейти, нажав соответствующий элемент управления, находясь в состоянии «Работа с приложением».

На рисунке 42 приведена диаграмма состояний системы для пользователя «Администратор». По умолчанию система ожидает действий пользователя. Если администратором выбрана вкладка меню «Галерея», система переходит в состояние «Работа с галереей». При нажатии на кнопку «Добавить» осуществляется переход в состояние, в котором происходит добавление картинки из файла. При выборе картинки и нажатии кнопки «Удалить», происходит удаление картинки из БД и обновление формы. При выборе пункта меню «Категории» система переходит в состояние работы с категориями, в котором администратор получает возможность изменить или удалить необходимые игры. Для перехода в состояние «Работа с уровнями сложности» администратору необходимо выбрать пункт меню «Уровни сложности». В этом состоянии у него есть возможность удалить или изменить существующие игры. Состояние «Изменение игры» наступает, когда администратор нажал кнопку «Изменить» в состоянии работы с категориями или уровнями сложности. Там он может задать новый уровень сложности, размеры, форму детали, категорию, тип сборки. Такой же функционал администратору доступен в состоянии «Добавление игры», в которое он может перейти нажав кнопку «Добавить игру». Также в этом состоянии ему доступно выбрать новое изображение из галереи. По нажатию клавиши «esc» система переходит в состояние «Завершение работы с приложением».

На рисунке 43 приведена диаграмма состояний системы для пользователя «Игрок».

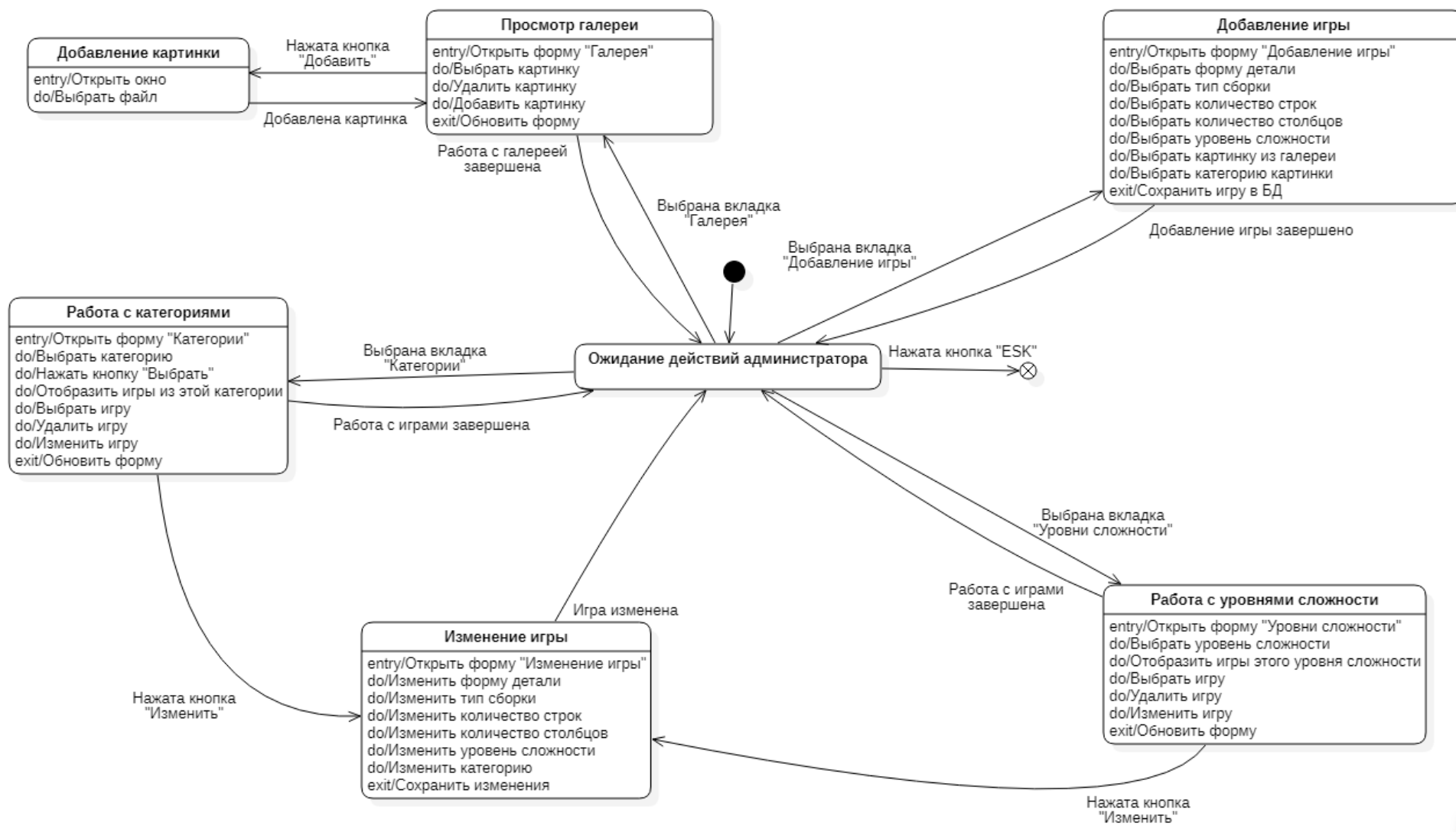


Рисунок 42 – Диаграмма состояний (Работа с приложением: Администратор)

По умолчанию система ожидает действий пользователя. Если игроком выбрана вкладка «Сохранённые игры», система переходит в состояние «Сохранённые игры», где игрок может выбрать незаконченную игру и удалить её, нажав кнопку «Удалить». При нажатии на кнопку «Играть» осуществляется переход в состояние «Игра», в котором происходит сборка пазла в соответствии с параметрами игры. Также в этом состоянии игрок может взять подсказку, посмотреть полную картинку или прочитать необходимую информацию о системе (состояние «Справочная информация о системе»). Также это состояние наступает при выборе вкладки «О системе». При выборе пунктов меню «Категории» и «Уровни сложности», система переходит в состояния «Игры по категориям» и «Игры по уровню сложности» соответственно. Там у игрока есть возможность выбрать необходимую игру и нажать кнопку «Играть», после чего система перейдёт в состояние «Игра». При выборе вкладки «Рейтинг», система переходит в состояние «Просмотр рейтинга», в котором игрок сможет посмотреть топ 10 игроков приложения. Для перехода в состояние «Настройки», игроку необходимо выбрать пункт меню «Настройки». Там пользователь сможет включить/выключить музыкальное сопровождение и звуки. При нажатии кнопки «i», система перейдёт в состояние «Справочная информация о разработчиках». По нажатию клавиши «esc» система завершает работу.

2.5.5 Диаграмма деятельности

Для моделирования процесса выполнения операций в языке UML используются диаграммы деятельности. Применяемая в них графическая нотация во многом похожа на нотацию диаграммы состояний, поскольку на диаграммах деятельности также присутствуют обозначения состояний и переходов. Отличие заключается в семантике состояний, которые используются для представления деятельности и действий, а также в отсутствии на переходах сигнатуры событий. Каждое состояние на диаграмме деятельности соответствует выполнению некой операции, а

переход в следующее состояние происходит только после завершения выполнения этой операции. Диаграмма деятельности представляется в форме графа деятельности, вершинами которого являются состояния действия или деятельности, а дугами – переходы от одного состояния действия к другому.

Диаграммы деятельности – частный случай диаграмм состояний. Они позволяют реализовать в языке UML особенности процедурного и синхронного управления, обусловленного завершением внутренних действий и деятельности. В контексте языка UML деятельность представляет собой совокупность отдельных вычислений, выполняемых автоматом. При этом отдельные элементарные вычисления могут приводить к результату или действию. На диаграмме деятельности отображается логика или последовательность перехода от одной деятельности к другой, при этом внимание фиксируется на результате деятельности. Сам же результат может привести к изменению состояния системы или возвращению некоторого значения [28].

На рисунке 44 приведена диаграмма деятельности системы для работы с галереями. Система открывает форму «Галерея», после чего пользователь может выбрать картинку и нажать на кнопку «Удалить», после чего картинка удалится из СУБД и форма обновится. Также администратору доступно добавление картинки. Для этого ему необходимо нажать на кнопку «Добавить», система откроет окно, и администратор сможет выбрать необходимый файл. После чего происходит проверка на наличие такого же файла в СУБД. Если картинка уже была добавлена ранее, система откроет форму «Галерея». Если изображения нет, то оно добавляется в СУБД и затем система открывает обновлённую форму «Галерея».

На рисунке 45 приведена диаграмма деятельности системы для добавления игры. Система открывает форму «Добавление игры», где администратор задаёт уровень сложности, количество строк и столбцов, форму детали, тип сборки, картинку и её категорию. После чего система делает кнопку «Добавить» активной. После нажатия кнопки «Добавить», новая игра добавляется в СУБД.

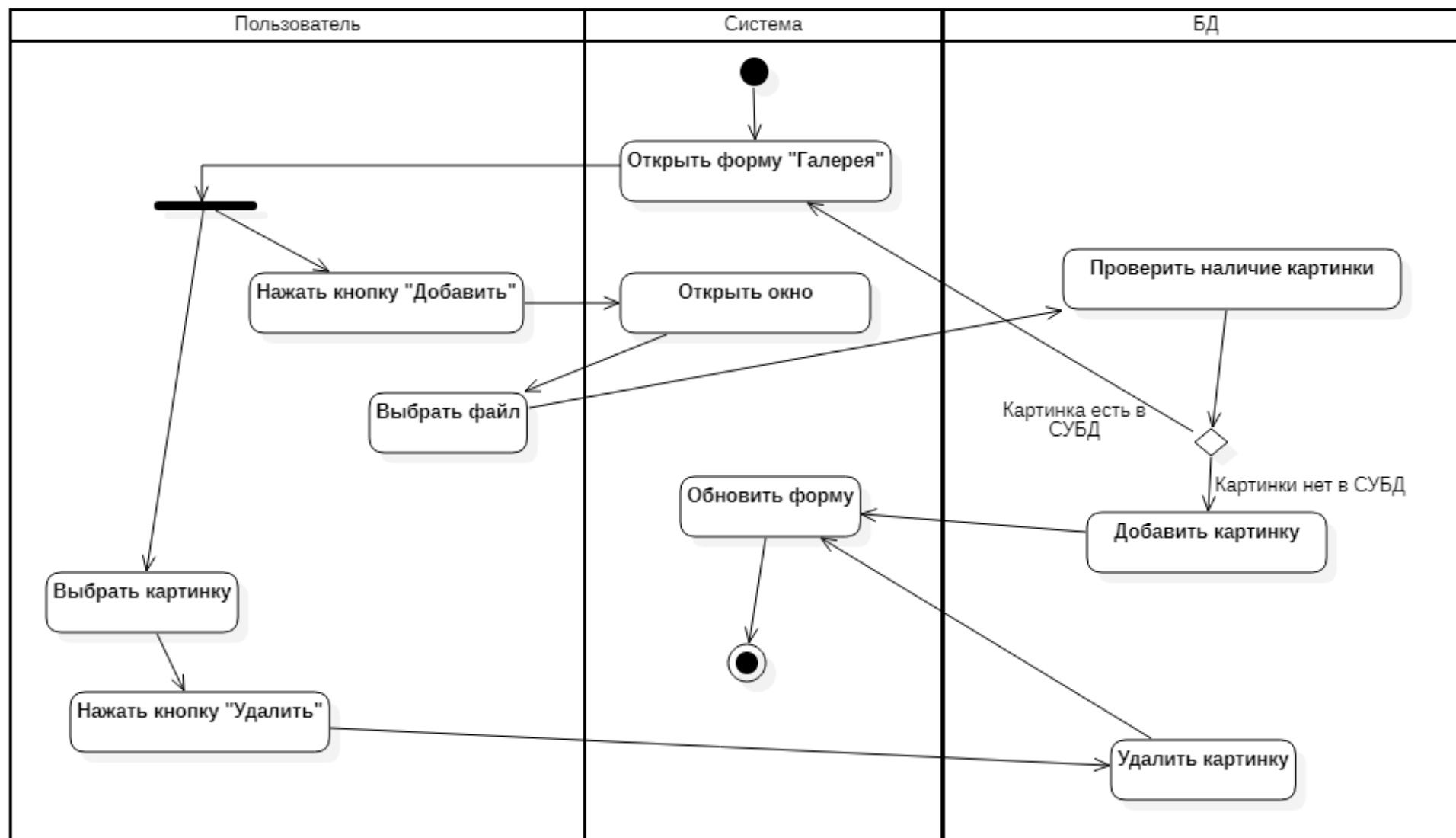


Рисунок 44 – Диаграмма деятельности (работа с галереей)

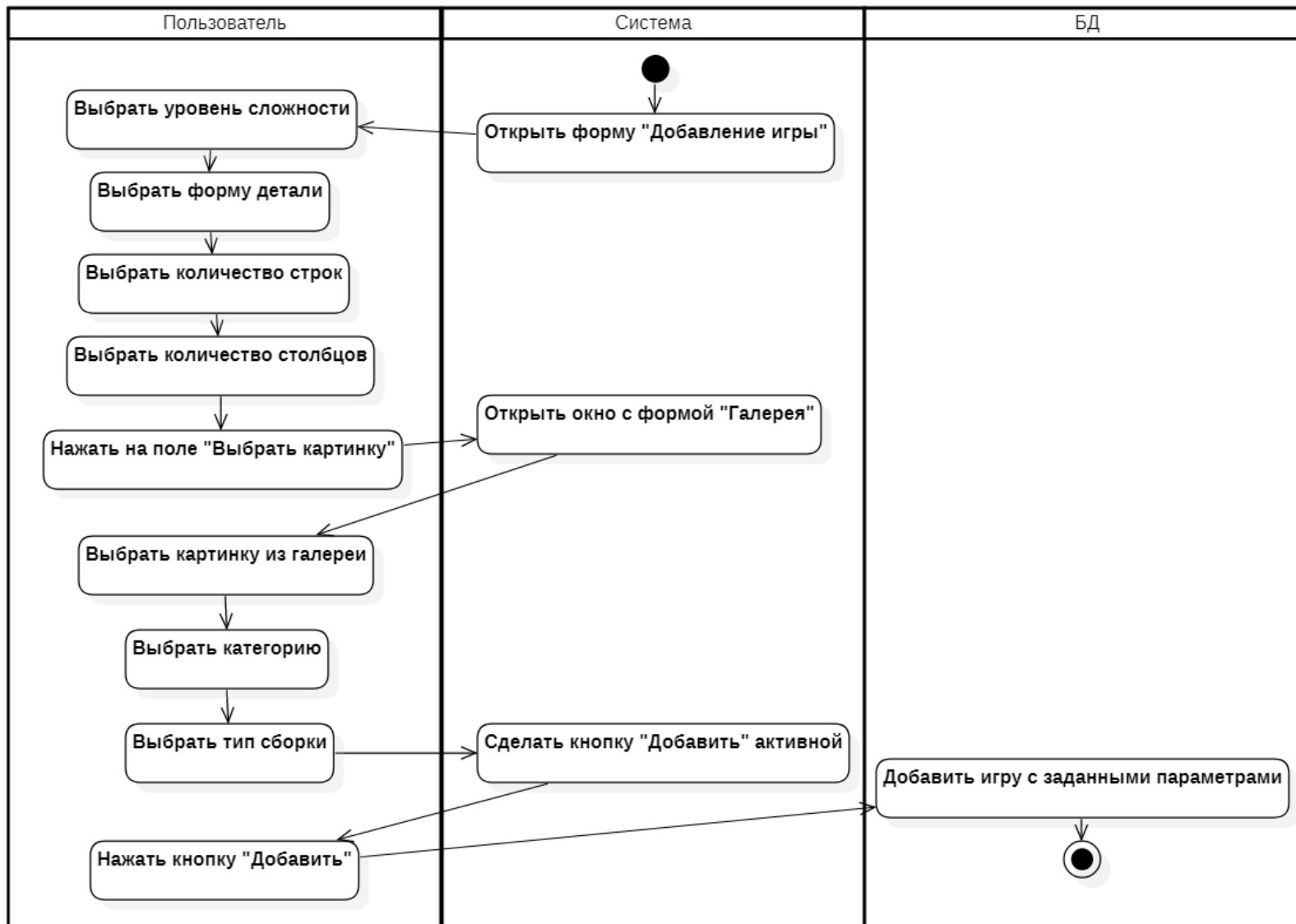


Рисунок 45 – Диаграмма состояний (добавление игры)

2.5.6 Диаграмма последовательности

Диаграмма последовательности UML (sequence diagram) – такая диаграмма, на которой показаны взаимодействия объектов, упорядоченные по времени их проявления. Основные элементы диаграммы последовательности это: обозначения объектов (прямоугольники), вертикальные линии, отображающие течение времени при деятельности объекта, и стрелки, показывающие выполнение действий объектами [29].

На рисунках 46 и 47 приведены диаграммы последовательности системы для вариантов использования «Выбрать количество столбцов» и «Задать уровень сложности» соответственно. Диаграммы построены на основании сценариев, приведённых в п.2.5.3.

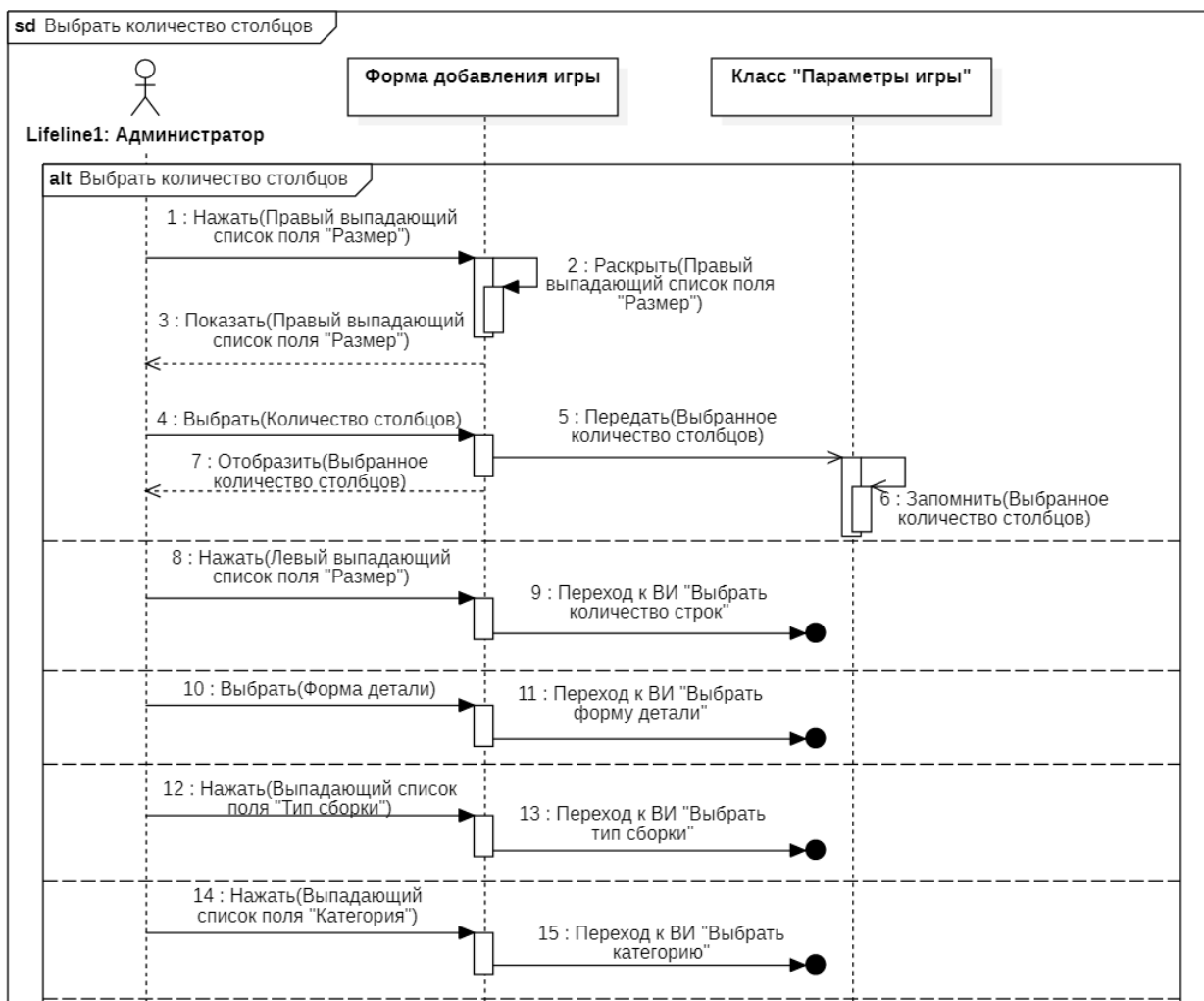


Рисунок 46 – Диаграмма последовательности «Выбрать количество столбцов» (начало)

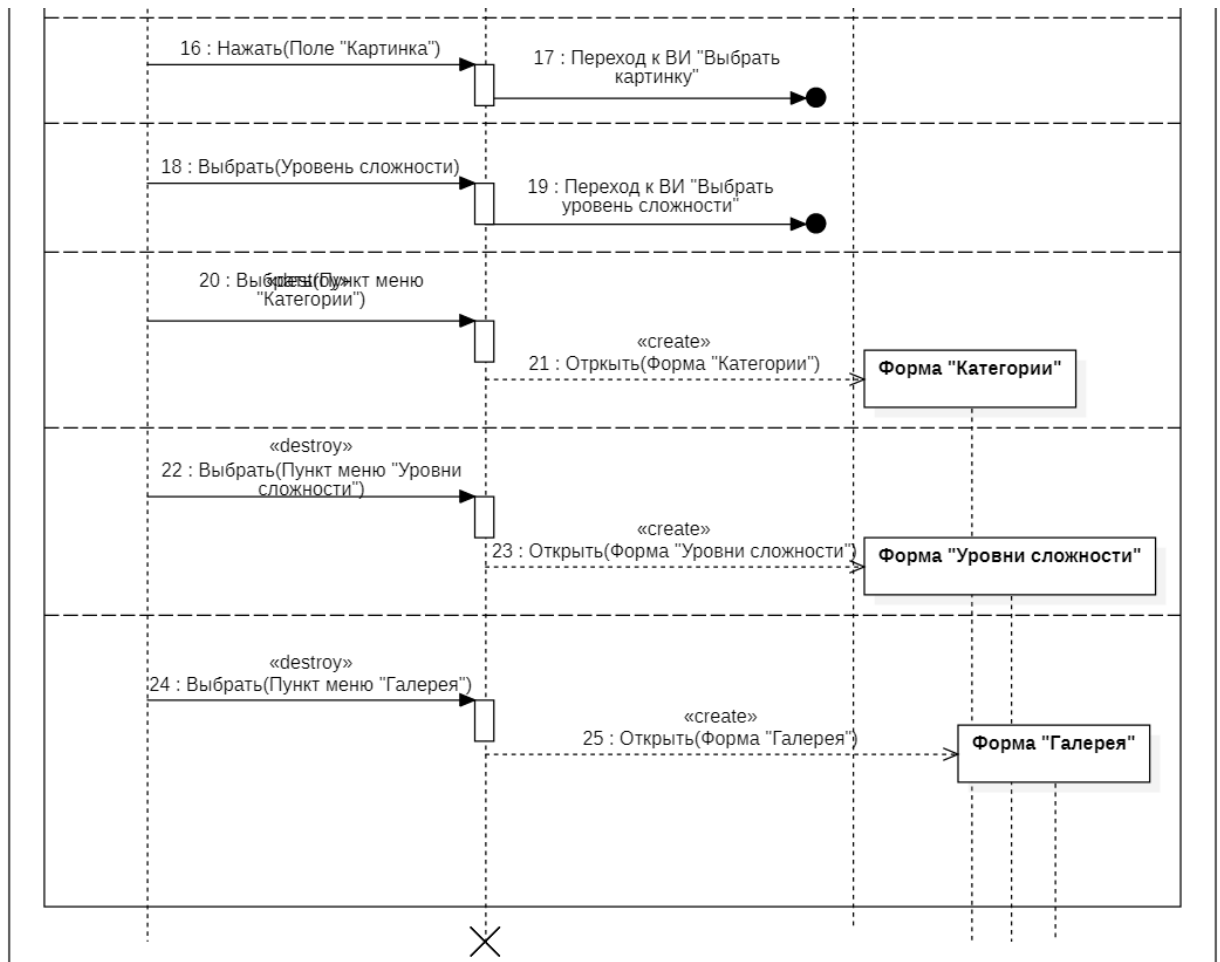


Рисунок 46 – Диаграмма последовательности «Выбрать количество столбцов» (окончание)

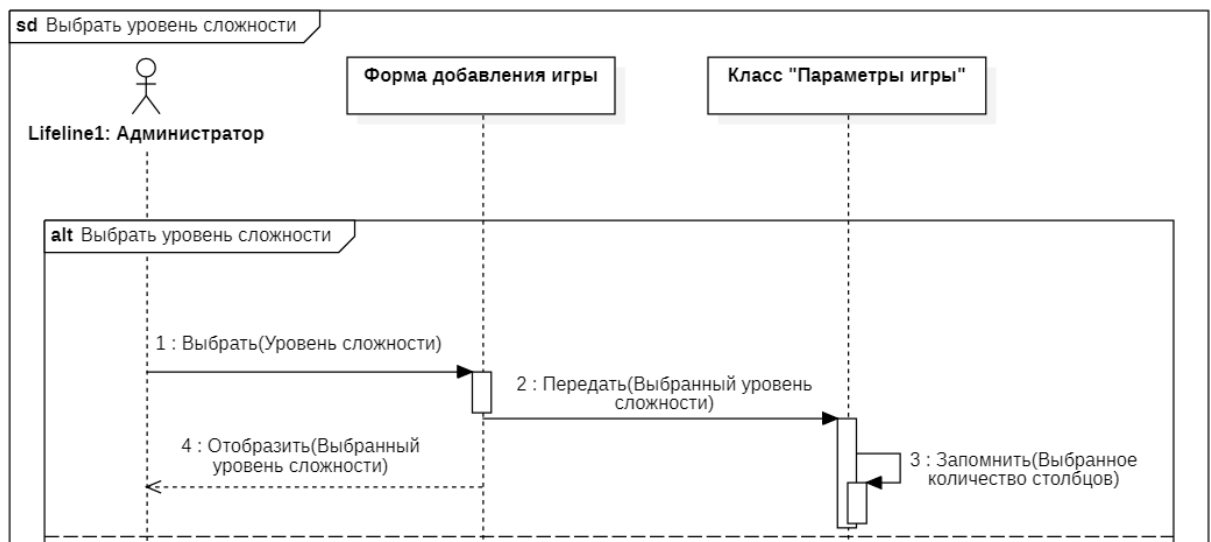


Рисунок 47 – Диаграмма последовательности «Выбрать уровень сложности» (начало)

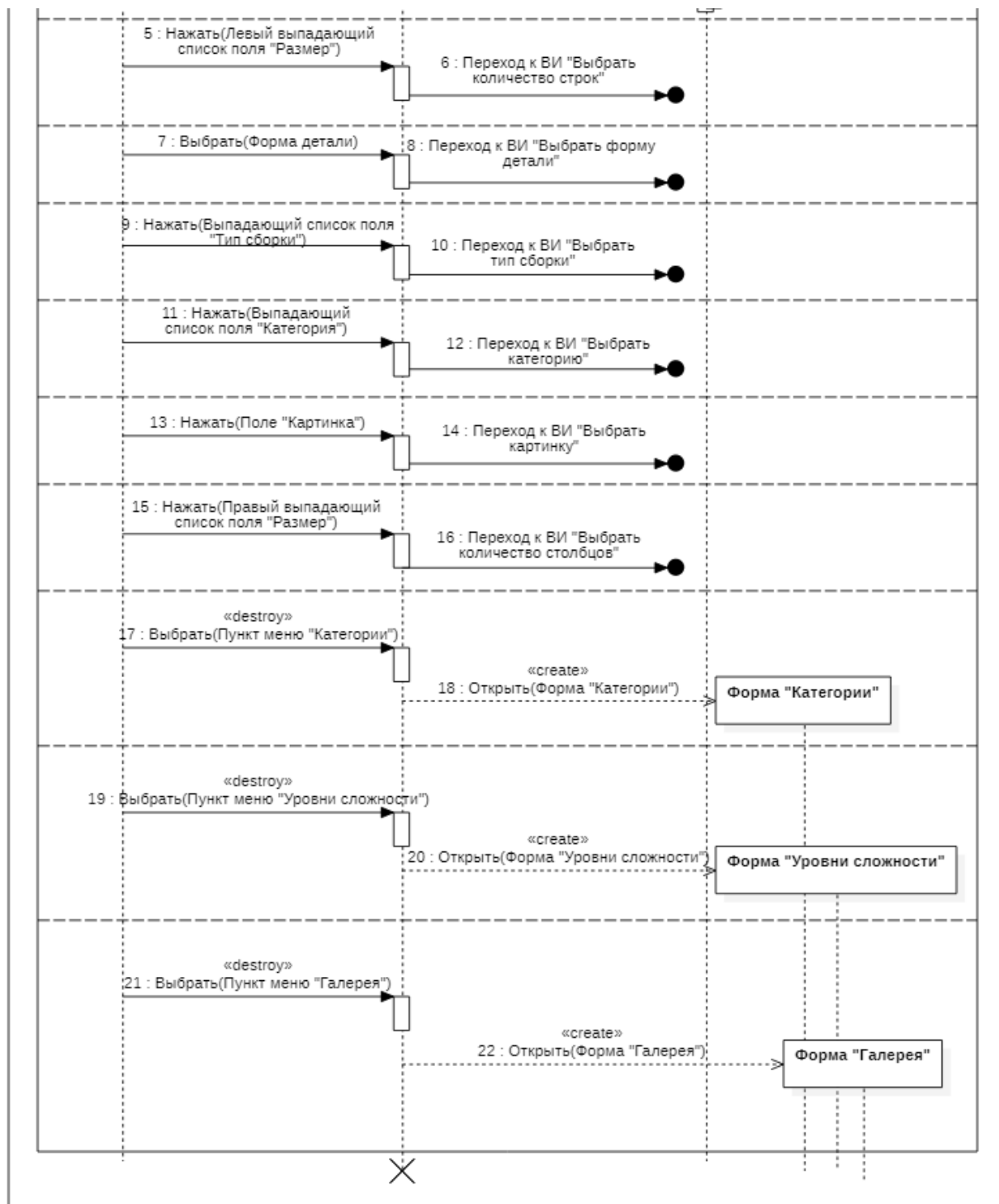


Рисунок 47 – Диаграмма последовательности «Выбрать уровень сложности»
(окончание)

2.5.7 Диаграмма классов

Диаграммы классов – это наиболее часто используемый тип диаграмм, которые создаются при моделировании объектно-ориентированных систем, они показывают набор классов, интерфейсов и коопераций, а также их связи.

На практике диаграммы классов применяют для моделирования статического представления системы, они служат основой для целой группы взаимосвязанных диаграмм – диаграмм компонентов и диаграмм размещения [30].

На рисунке 48 приведена диаграмма классов системы (этап проектирования). В таблице 5 приведено описание классов.

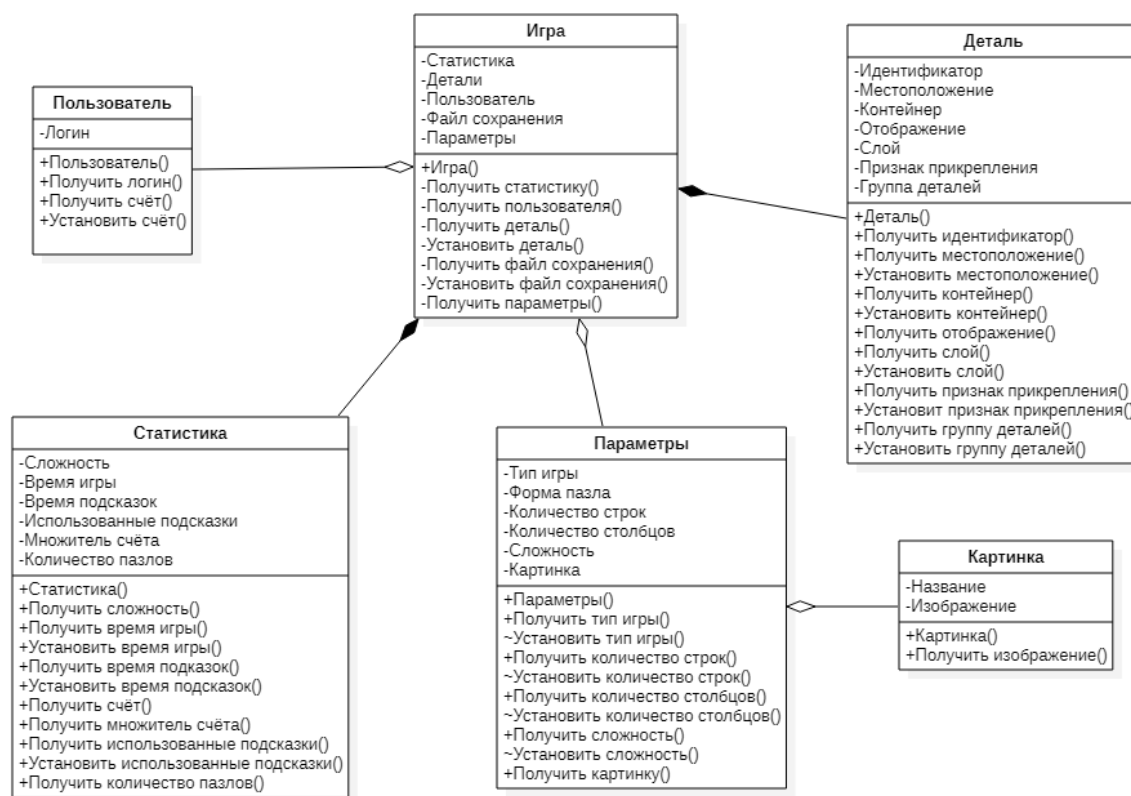


Рисунок 48 – Диаграмма классов системы

Таблица 5 – Описание классов системы

Название класса	Назначение
1	2
Пользователь	Класс нужен для хранения имени пользователя и его счёта.
Игра	Класс предназначен для хранения статистики игры, деталей на поле, осуществления действий со счётом пользователя, сохранения и загрузки игровых параметров.

Продолжение таблицы 5

1	2
Статистика	Класс нужен для хранения игровой статистики, которая после используется для подсчёта счёта.
Деталь	Класс предназначен для хранения информации о каждом фрагменте на экране. Он хранит в себе полную информацию о позиции фрагмента, которая включает его местоположения на экране, занимаемого им слоя Z – буфера и родительского контейнера. Также с деталью ассоциирован признак закрепления, который определяет, возможно ли с ней дальнейшее взаимодействие. В классе детали хранится ссылка на её группу. Любые взаимодействия с фрагментами отображаются также на всех деталях группы. Каждый пазл имеет уникальное отображение на интерфейсе.
Параметры игры	Класс предназначен для хранения информации о настройках каждой конкретной игры. Параметры игры связаны с определённой картинкой. В процессе игры эти параметры могут быть получены, но не могут быть изменены.
Картинка	Служит для хранения картинок.

2.6 Логическая модель данных

Логическая информационная модель – модель данных, в которой учитывается способ логического хранения данных в памяти ЭВМ. При построении модели базы данных (БД) используются следующие понятия.

Сущность – объект предметной области, который можно отличить от других понятий по некоторым признакам. Сущность состоит из множества своих экземпляров. Каждая сущность обладает свойствами – атрибутами [31].

Атрибут – определенное свойство сущности. Именно набор атрибутов, в общем случае уникальный для каждой сущности, позволяет выделить ее среди других объектов и назвать уникальным именем.

Атрибут или набор атрибутов, используемый для идентификации экземпляра сущности, называется ключом сущности. В случае если для идентификации экземпляра используется один атрибут, ключ называется простым; в противном случае ключ составной. Каждый экземпляр сущности однозначно определяется ключом [32].

Логическая модель БД разрабатываемой системы приведена на рисунке 49.

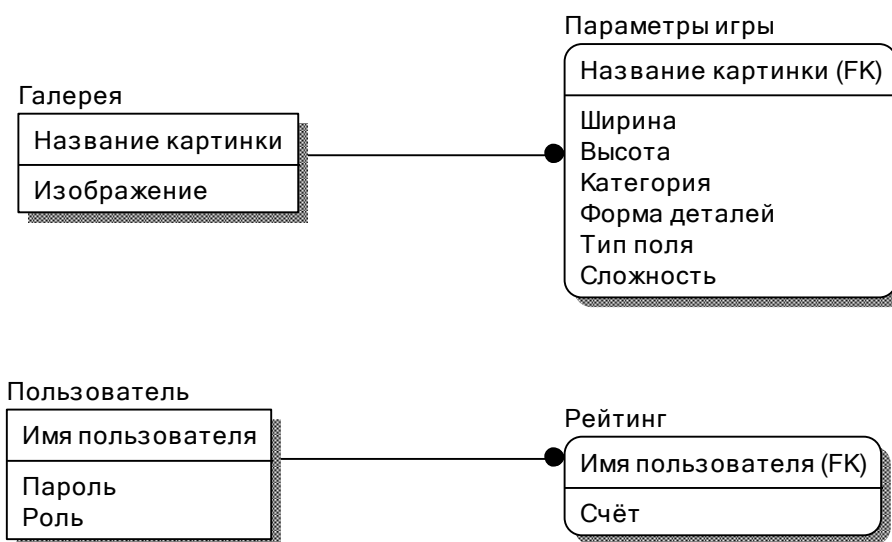


Рисунок 49 – Логическая модель данных

Описание объектов рассматриваемой предметной области, которые хранятся в базе данных, приведено в таблицах 6-9.

Таблица 6 – Сущность «Галерея»

Идентификатор	Тип данных	Описание
Название картинки	Строка	Идентификатор картинки
Изображение	Массив двоичных данных	Изображение формата JPEG, загруженное администратором в виде потока двоичных данных.

Таблица 7 – Сущность «Пользователь»

Идентификатор	Тип данных	Описание
Имя пользователя	Строка	Логин пользователя, используемый для его идентификации
Пароль	Строка	Пароль пользователя, используемый для входа в приложение
Роль	Логическое	Роль пользователя в приложении: 0 – игрок, 1 – администратор

Таблица 8 – Сущность «Параметры игры»

Идентификатор	Тип данных	Описание
1	2	3
Ширина	Целое	Количество деталей, на которое разбивается изображение по горизонтали
Высота	Целое	Количество деталей, на которое разбивается изображение по вертикали
Категория	Целое	Категория изображения в галерее. Она определяется следующими значениями: 0 – природа, 1 – искусство, 2 – животные, 3 – абстракция, 4 – для детей, 5 – градиент
Сложность	Целое	Сложность сборки пазла, определяемая администратором

Продолжение таблицы 8

1	2	3
Форма деталей	Целое	Форма деталей, на которое разбивается изображение. Фрагменты принимают следующие значения формы: 0 – треугольный 1 – квадратный 2 – фигурный
Тип поля	Целое	Вариант изначального положения деталей. Тип поля распределён следующим образом: 0 – на поле, 1 – в ленте, 2 – в куче

Таблица 9 – Сущность «Рейтинг»

Идентификатор	Тип данных	Описание
Счёт	Целое	Максимальный счёт, достигнутый пользователем в процессе игры.

2.7 Разработка и описание алгоритмов обработки данных

Алгоритм – это точный набор инструкций, описывающих порядок действий некоторого исполнителя для достижения результата, решения некоторой задачи за конечное время. Схема алгоритма является формой представления алгоритма с помощью графических символов. Графические символы, их размеры, а также правила построения блок-схем определены государственными стандартами [33].

Составление схемы алгоритма является важным и необходимым этапом решения задачи. Но, т.к. в таком виде алгоритм не может быть выполнен непосредственно ЭВМ, этот этап является промежуточным, значительно облегчающим процесс составления программы [34].

На рисунке 50 представлена схема алгоритма сборки пазла. Реализованный алгоритм состоит из следующих этапов.

Этап 1. Инициализация значений параметров статистики.

В дальнейшем она используется для подсчёта рейтинг. Определены следующие параметры:

- timer, tipsTimer – таймеры, подсчитывающий длительность сборки и подсчитывающий длительность отображения картинки-подсказки. Определён класс таймеров, которые имеет следующие методы работы с ними:

- Start() – начинает подсчёт времени.
- Stop() – приостанавливает подсчёт времени.
- Time() – выводит подсчитанное время в секундах.

- fragmentsCount – количество пазлов на поле. Определяется как: $\text{fragmentsCount} = \text{Height} \times \text{Width} \times \text{CountInCell}$, где Height и Width – количество разбиений по ширине и высоте соответственно. CountInCell – количество пазлов в ячейке, равно 2, если форма деталей треугольная и 1, в остальных случаях.

- difficulty – уровень сложности игры, принимает значение от 1 до 5.
- tipsCount – счётчик количества оставшихся подсказок.
- usedTipsCount – счётчик количества уже использованных подсказок.

- fieldType – тип игрового поля, принимает одно из следующих значений: поле, лента, куча.

- Начинается отсчёт времени.

Этап 2. Ожидание действия игрока.

Возможно возникновение четырёх видов событий:

- Событие нажатия кнопки «Картинка». Выполняются следующие последовательные шаги:

- 1 Начинается отсчёт времени отображения картинки.

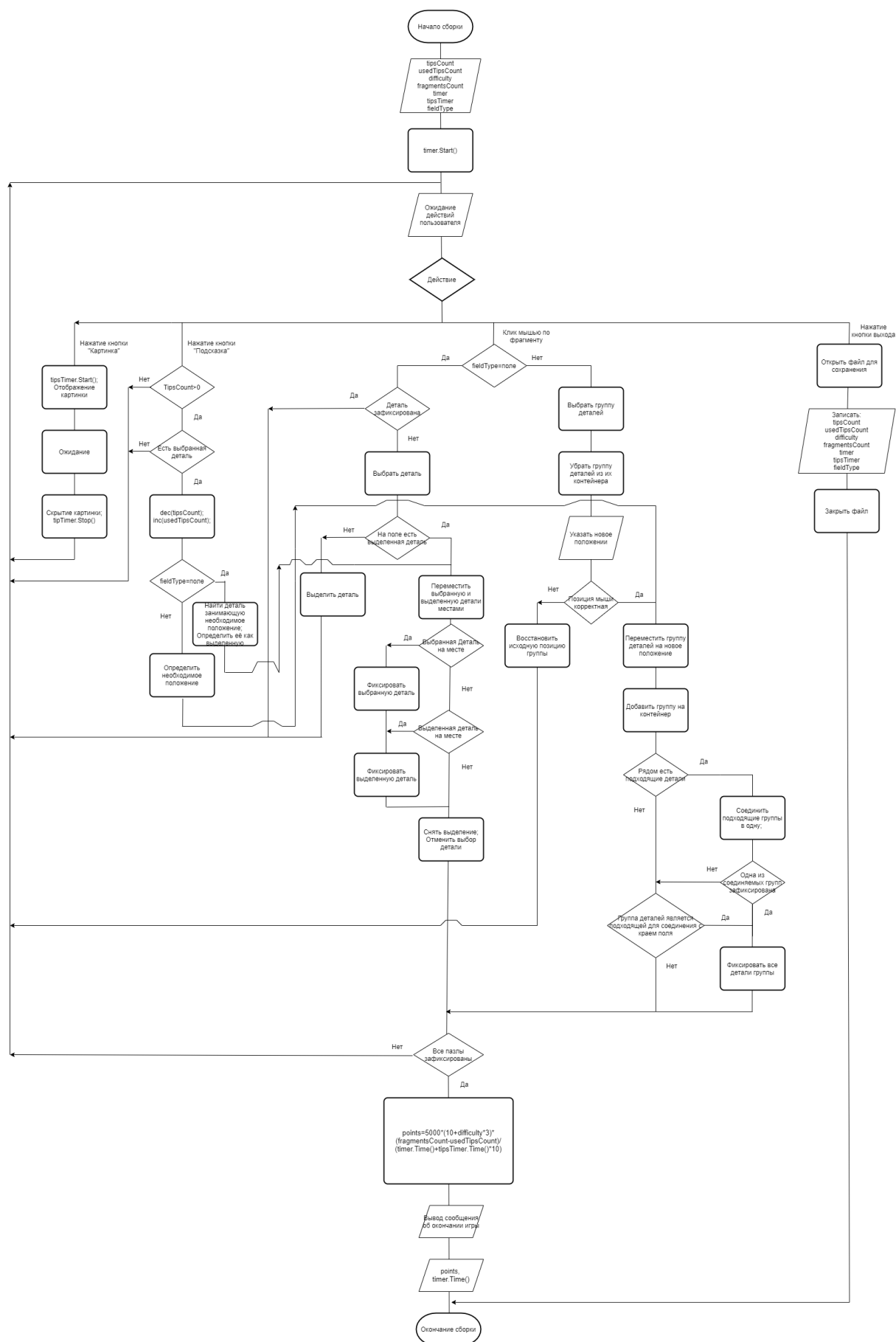


Рисунок 50 – Схема алгоритма сборки пазла

2 Ожидается сигнал от пользователя о завершении отображения картинки.

3 Скрытие картинки и остановка отсчёта времени отображения.

4 Возврат к ожиданию действий игрока.

– Событие нажатия кнопки «Подсказка». Выполняются следующие последовательные шаги:

1 Если есть ещё неиспользованные подсказки и на поле присутствует выбранная деталь, перейти к следующим шагам, иначе возвратиться к ожиданию действий игрока.

2 Обновить значения счётчиков подсказок.

3 Определить необходимое положение фрагмента и в зависимости от типа игрового поля перейти к функциям перестановки деталей события клика мыши по фрагменту.

– Клик мышью по фрагменту. Проверяется тип игры, возможно два исхода.

1 Исход 1. Тип игры равен «поле».

– Если деталь зафиксирована, возвратиться к ожиданию действий игрока.

– Выбрать деталь.

– Если на поле нет выделенной детали, то выделить выбранную деталь возвратиться к ожиданию действий игрока, иначе перейти к следующим шагам.

– Переместить выбранную и выделенную деталь местами.

– Если выделенная или выбранная деталь на нужном месте, то зафиксировать её.

– Снять выделение и отменить выбор детали.

– Если все детали зафиксированы, перейти к выводу сообщения о победе, иначе возвратиться к ожиданию действий игрока.

2 Исход 2. Тип игры равен «куча» или «лента»

- Выбрать группу деталей.
- Убрать группу деталей из их контейнера. В случае снятия детали с ленты её содержимое автоматически обновляется.
- Получить новое положение группы от пользователя.
- Проверяется корректность нового положения. Курсор мыши не должен находиться за пределами игрового поля, или на ленте, в случае выбора группы, которая содержит более одной детали. Если корректность нарушается, восстановить исходную позицию выбранной группы деталей и вернуться к ожиданию действий от пользователя, иначе перейти к следующему шагу.
- Переместить деталь на новое положение.
- Добавить группу деталей на контейнер.
- Если есть рядом подходящие детали, соединить соответствующие им группы с выбранной.
- Если детали одной из присоединённых групп зафиксированы или основная группа расположена рядом с краем поля, при этом находясь на верном положении, то зафиксировать всю группу.
- Если все детали зафиксированы, перейти к выводу сообщения о победе, иначе возвратиться к ожиданию действий игрока.
- Нажатие кнопки выхода. Выполняются следующие последовательные шаги:
 - Создание и открытие файла сохранения.
 - Сохранение карты игрового поля.
 - Сохранение параметров статистики.
 - Закрывание файла.
 - Выход из процесса сборки.

Этап 3. Вывод сообщения о победе.

Выполняются следующие шаги:

- Подсчёт количества очков. Рейтинг игрока считается по следующей формуле: $points = 5000 \times \frac{(10+difficulty \times 3) \times (fragmentsCount - usedTipsCount)}{timer.Time() + 10 \times tipsTimer.Time()}$
- Отображение на экране сообщения о завершении игры и вывод количества очков, длительности игры.

2.8 Выбор и обоснование комплекса программных средств

Для реализации системы был выбран язык программирования C#, среда разработки Visual Studio 2022, платформа .Net 6 и система управления БД MySQL 8.0.

2.8.1 Выбор языка программирования

Язык программирования C# был разработан в 1993–2001 годах группой инженеров компании Microsoft под руководством Андерса Хейлсберга и Скотта Вильтаумота как язык разработки приложений для платформы Microsoft [35].

C# – это язык с C-подобным синтаксисом. Здесь он близок в этом отношении к C++ и Java.

Будучи объектно-ориентированным языком, он много перенял у Java и C++. Как и Java, C# изначально предназначался для веб-разработки, и примерно 75% его синтаксических возможностей такие же, как у Java. C# также называют «очищенной версией Java». Ещё 10% наш герой позаимствовал из C++ и 5% – из Visual Basic. Оставшиеся 10% C# – это реализация собственных идей разработчиков. Объектно-ориентированный подход позволяет строить с помощью C# крупные, но в то же время гибкие, масштабируемые и расширяемые приложения.

C# уже давно поддерживает много полезных функций [35]:

- инкапсуляция;
- наследование;

- полиморфизм;
- перегрузка операторов;
- статическая типизация.

C# на протяжении долгого времени уверенно держит позиции в рейтинге наиболее востребованных на рынке разработки языков [35].

2.8.2 Выбор операционной системы

ОС представляет собой набор программ, которые управляют работой компьютера. Основные функции ОС [36]:

- контролирует загрузку процессора;
- управляет ресурсами оперативной памяти, карт памяти или жёстким диском;
- отвечает за распределение задач и процессов;
- обновляет, устанавливает и удаляет приложения и программы [38].

Во время проектирования была выбрана операционная система Windows 10.

Начиная с Windows 95 и заканчивая Windows 10 – именно операционная система Microsoft считается самой популярной и наиболее знакомой простым пользователям из разных уголков мира. Программное обеспечение компании остаётся основой современных компьютеров. Его выбирают из-за простоты использования, быстрого запуска и возобновления работы. Последние версии операционной системы отличаются повышенной безопасностью и вниманием к конфиденциальности пользователей [37].

2.8.3 Выбор среды программирования

Microsoft Visual Studio – линейка продуктов компании Майкрософт, включающих интегрированную среду разработки программного обеспечения и ряд других инструментальных средств. Данные продукты позволяют разрабатывать как консольные приложения, так и приложения с графическим интерфейсом, в том числе с поддержкой технологии Windows Forms, а также

веб-сайты, веб-приложения, веб-службы как в родном, так и в управляемом кодах для всех платформ, поддерживаемых Microsoft Windows, Windows Mobile, Windows CE, .NET Framework, Xbox, Windows Phone .NET Compact Framework и Microsoft Silverlight [38].

Visual Studio 2022 – это первый выпуск 64-битной версии Visual Studio.

Visual Studio 2022 также включает ряд улучшений в области редактирования и отладки. Новая версия также предоставляет функцию Hot Reload, которая позволяет разработчикам редактировать исходный код во время работы приложений в Visual Studio 2022 и из .NET CLI. Среди других новых и улучшенных возможностей – возможность Live Preview и кросс-платформенное тестирование на Linux.

.NET 6 доступен для Windows, Linux и macOS. Он обеспечивает единую платформу для облачных, настольных, IoT и мобильных приложений благодаря использованию одних и тех же библиотек .NET и возможности легкого обмена кодом между платформами. В этом выпуске улучшена производительность, особенно в части файлового ввода-вывода [39].

2.8.4 Выбор системы управления базами данных

MySQL – свободная реляционная система управления базами данных.

MySQL является решением для малых и средних приложений. Входит в состав серверов WAMP, AppServ, LAMP и в портативные сборки серверов Денвер, XAMPP, VertrigoServ. Обычно MySQL используется в качестве сервера, к которому обращаются локальные или удалённые клиенты, однако в дистрибутив входит библиотека внутреннего сервера, позволяющая включать MySQL в автономные программы [40].

Не вдаваясь в технические аспекты, основные процессы, происходящие в среде MySQL, одинаковы [41]:

- MySQL создаёт базу данных для хранения и управления данными, определяющими отношения каждой таблицы.

- Клиенты могут делать запросы, вводя определённые команды SQL на MySQL.

- Приложение сервера ответит запрошенной информацией и появится на стороне клиента.

MySQL действительно не единственная СУРБД на рынке, но она является одной из самых популярных и уступает только Oracle Database, когда оценивается с использованием таких важных параметров, как количество упоминаний в результатах поиска, профессиональных профилей в LinkedIn и частоты технических дискуссий на интернет-форумах [41].

3 Реализация системы

3.1 Разработка и описание интерфейса пользователя

На рисунке 51 приведена начальная форма приложения. Здесь пользователь должен ввести логин и пароль и нажать кнопку «Войти» или «Регистрация», после чего система выполнит вход, либо же зарегистрирует нового пользователя.

При авторизации пользователя в роли «Администратор» откроется форма с галереей. Данная экранная форма представлена на рисунке 52. Здесь находятся картинки, которые добавлены администратором, но ещё не используются в играх. Администратору предоставлена возможность удалить картинку из галереи, а также добавить новую картинку из файла (*.png). Данный процесс показан на рисунке 53.

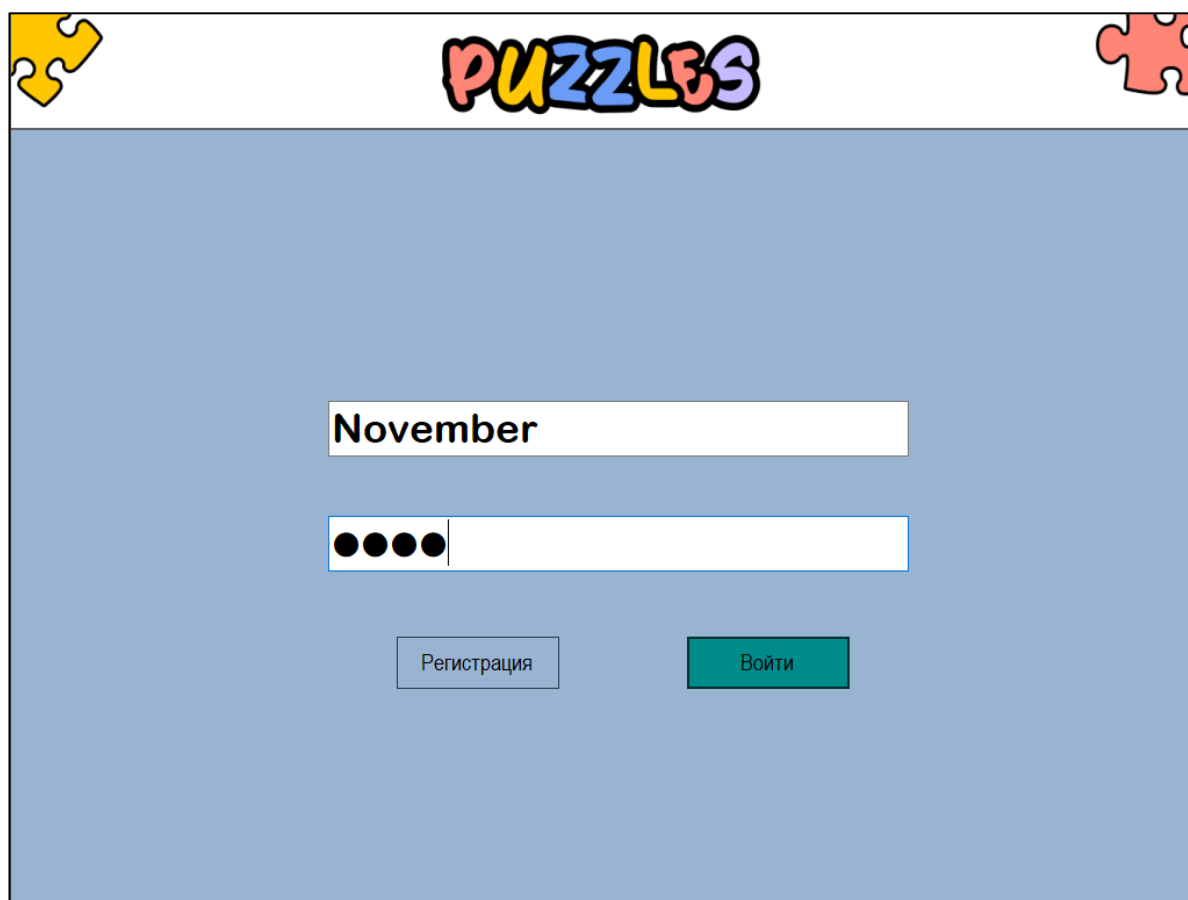


Рисунок 51 – Начальная форма приложения

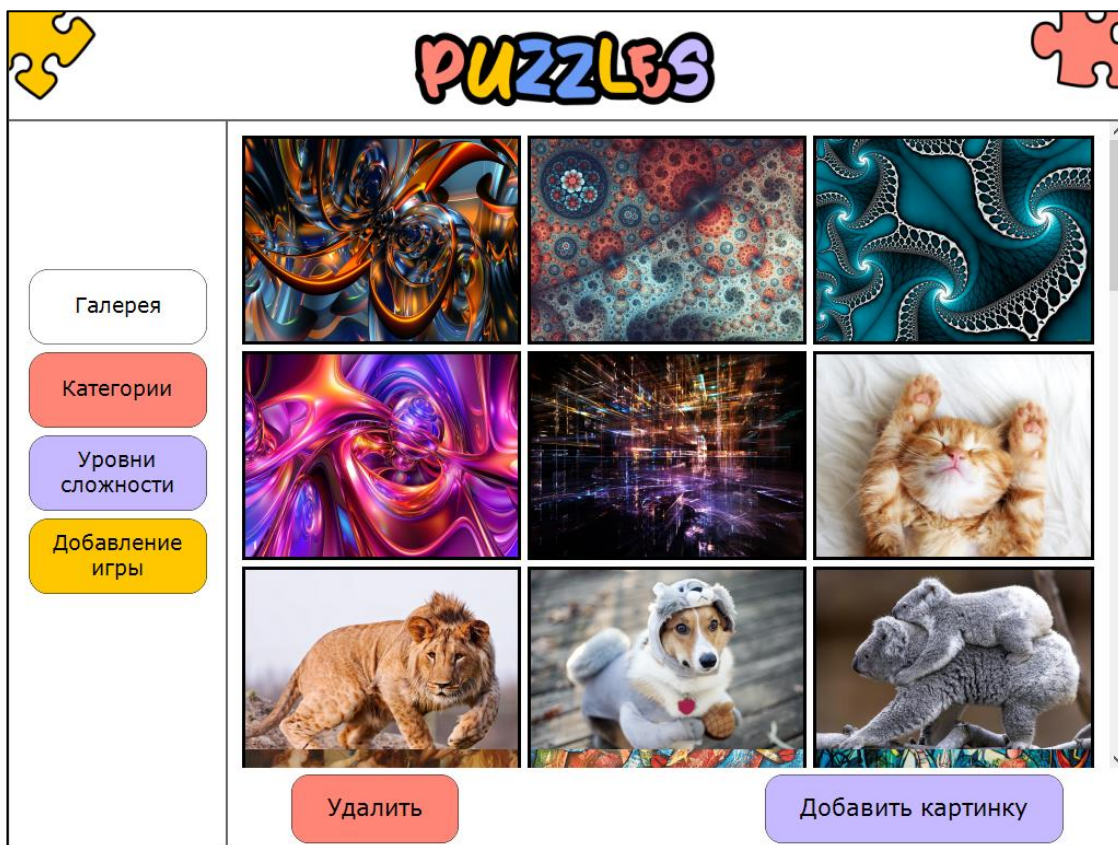


Рисунок 52 – Экранная форма вкладки «Галерея»

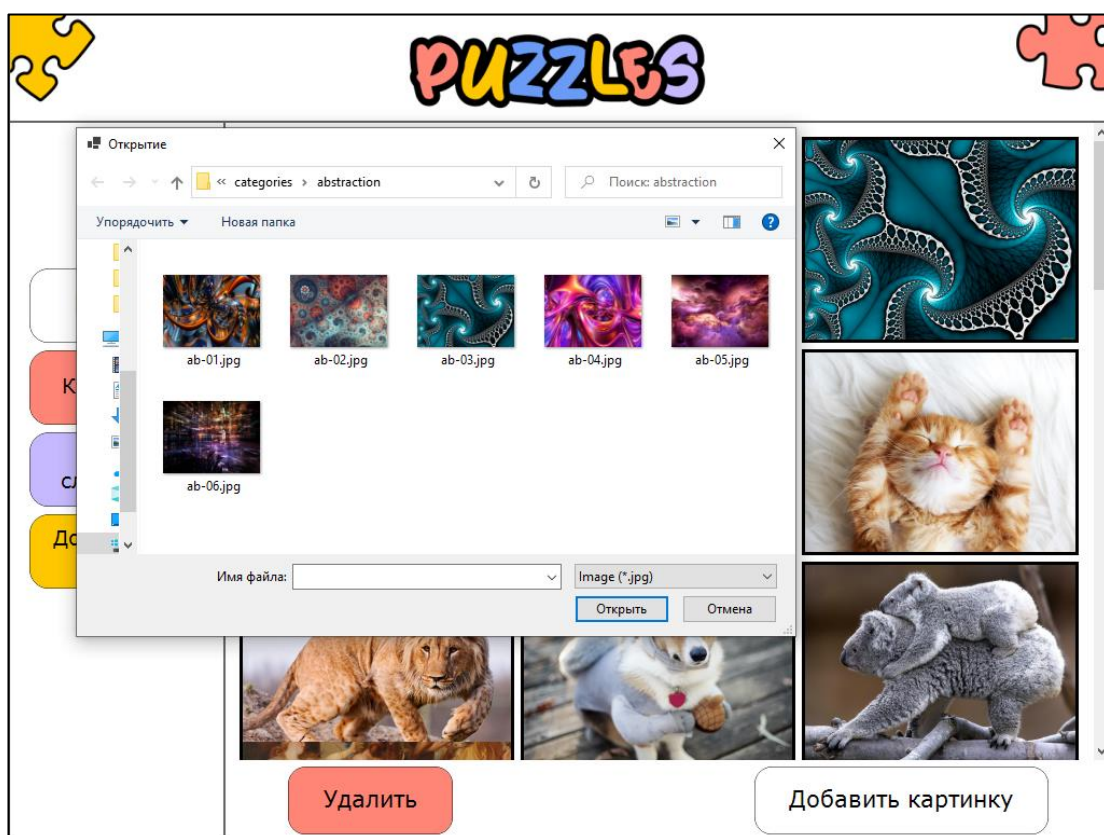


Рисунок 53 – Добавление новой картинки

Вкладки «Категории» и «Уровни сложности» (см. рисунок 54) позволяют производить навигацию по каталогу игр, которые доступны пользователю с ролью «Игрок». Также в функции администратора входят удаление игры или изменение её настроек. Для этих действий необходимо сначала выбрать пазл (он обводится зелёной рамкой), а затем нажать на соответствующую кнопку. При нажатии на кнопку «Удалить», данная игра удалится из каталога игр, а также из БД. При нажатии на кнопку «Изменить игру», администратор увидит форму изменения игры, которая изображена на рисунке 55.

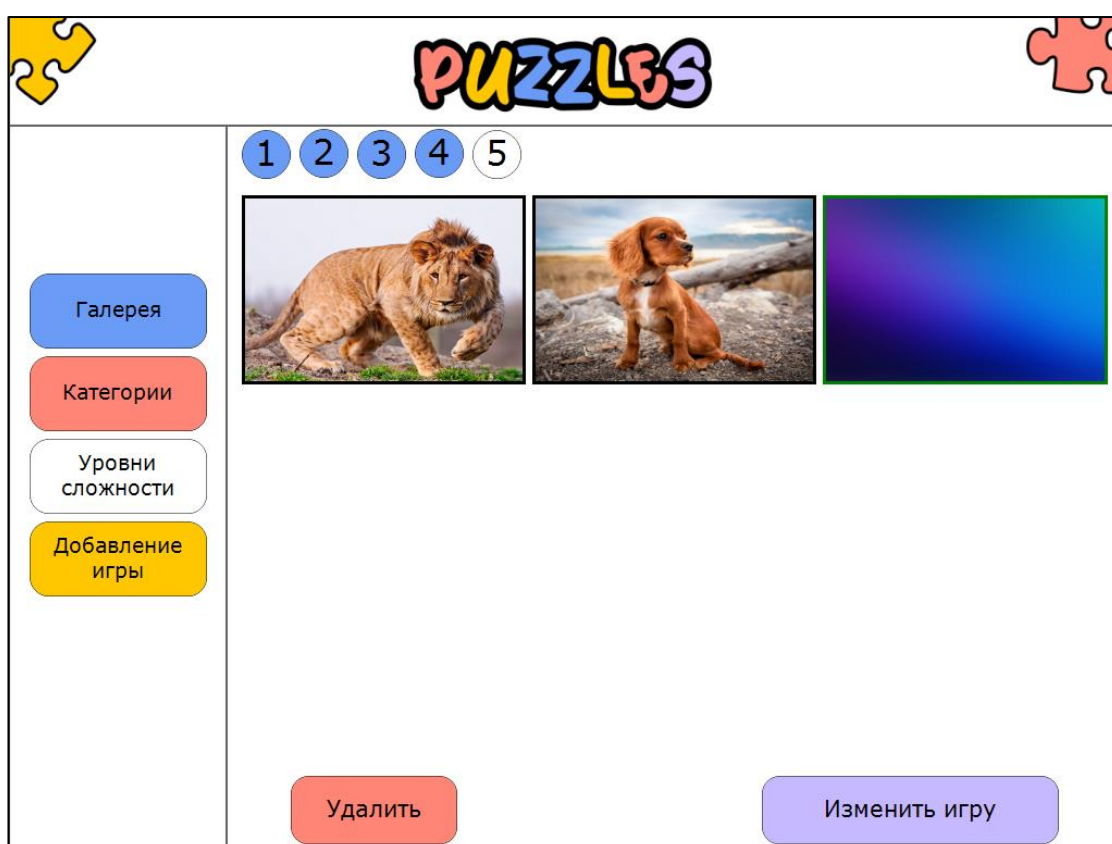


Рисунок 54 – Экранная форма вкладки «Уровни сложности»

В форме изменения игры, администратор может изменять параметры (форму детали, размерность, тип сборки, категорию и уровень сложности), указанные ранее для этой игры. После чего необходимо нажать кнопку «Изменить». После сохранения изменённая игра появится в каталоге игр.

Рисунок 55 – Экранная форма изменения игры

Во вкладке «Добавление игры» (см. рисунок 56) у администратора есть возможность добавить в каталог игр новый пазл. Для этого нужно выбрать картинку из галереи, задать для неё уровень сложности и остальные необходимые параметры (размерность, тип сборки, категорию) и нажать кнопку «Добавить игру».

Рисунок 56 – Экранная форма вкладки «Добавление игры»

При выборе роли «Игрок» система откроет вкладку «Сохранённые игры», где хранятся все пазлы, которые он не закончил собирать (см. рисунок 57).

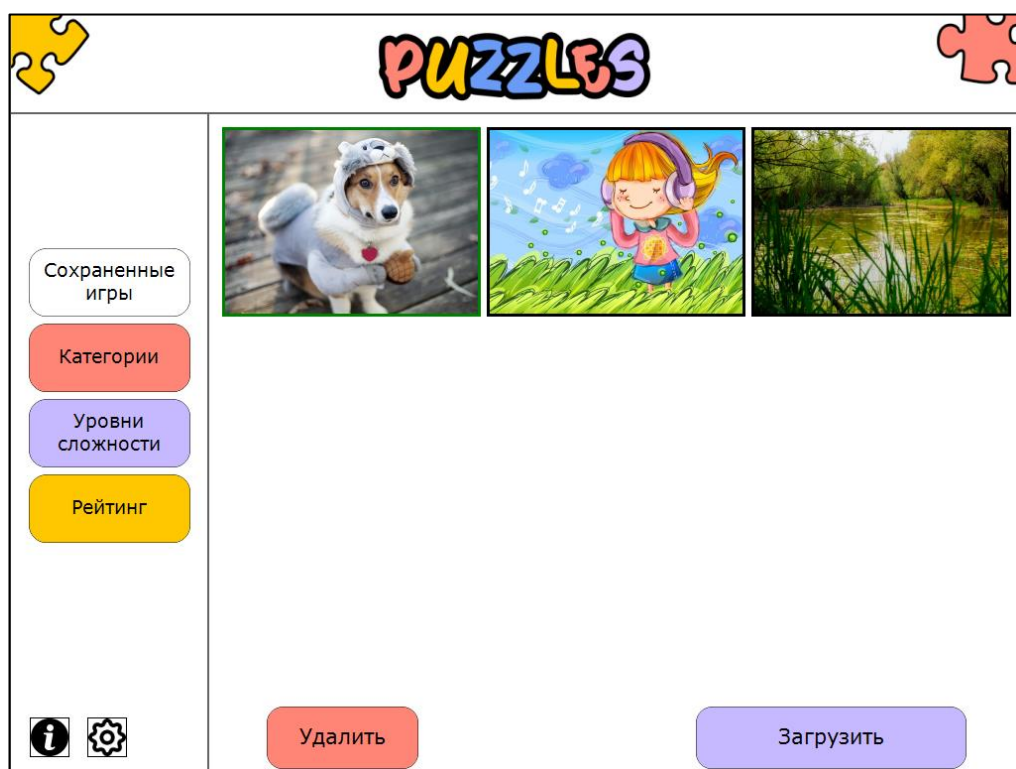


Рисунок 57 – Экранная форма вкладки «Сохранённые игры»

Вкладка «Категории» представлена на рисунке 58.

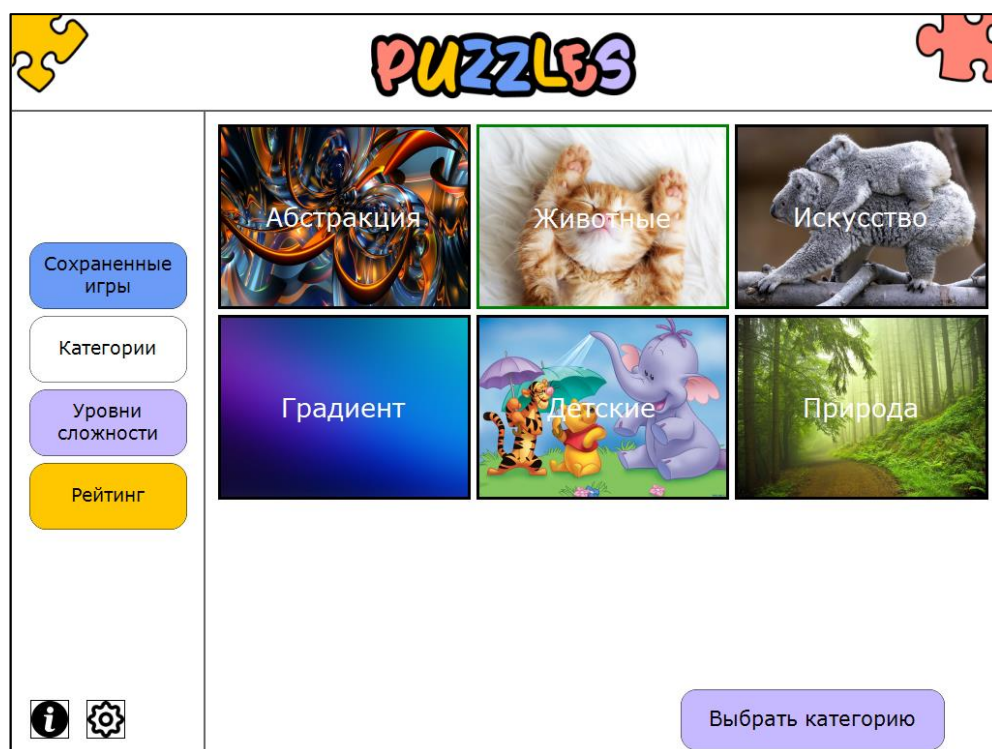


Рисунок 58 – Экранная форма вкладки «Категории»

В данной вкладке все картинки распределены по категориям. После выбора конкретной категории, пользователь сможет выбрать интересующий его пазл из возможных картинок данной категории (см. рисунок 59).

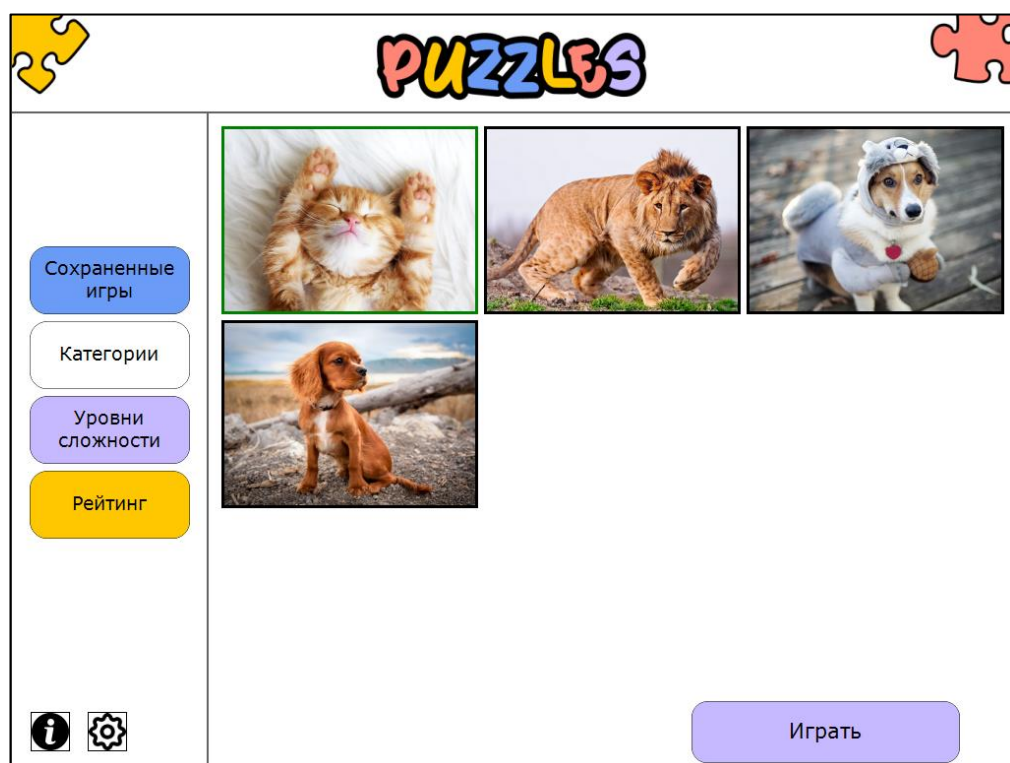


Рисунок 59 – Экранная форма категории «Животные»

Во вкладке «Рейтинг», пользователь может изучить список из 10 игроков, занимающих лидерские позиции (см. рисунок 60).



Рисунок 60 – Экранная форма вкладки «Рейтинг»

Во вкладке «Уровни сложности» игрок может выбрать пазл, исходя из необходимого ему уровня сложности. Переключаться между уровнями можно через специальные кнопки над картинками. Вкладка «Уровни сложности» представлена на рисунке 61.

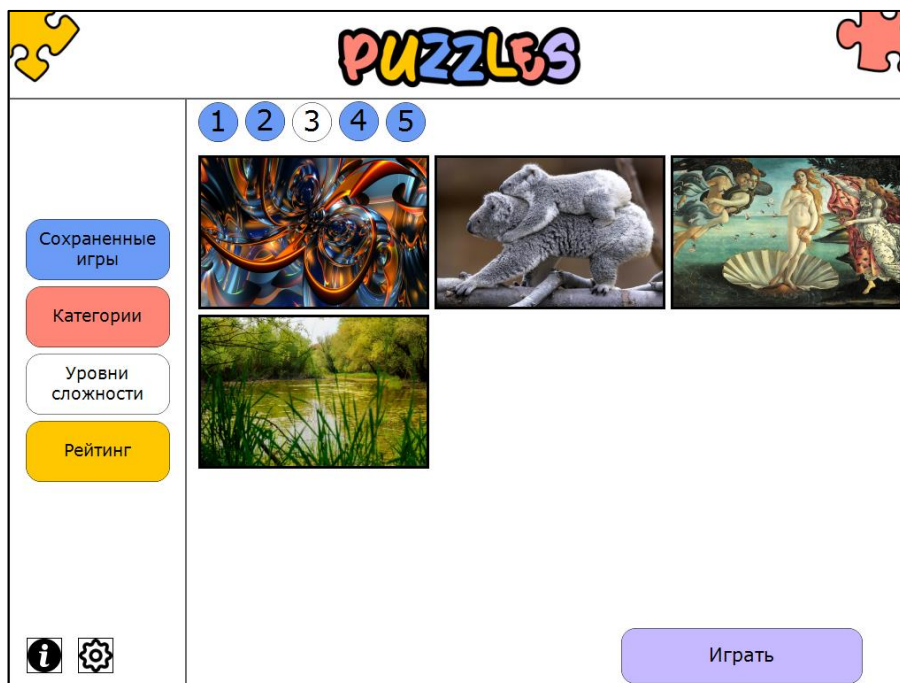


Рисунок 61 – Экранная форма вкладки «Уровни сложности»

При нажатии на кнопку , система откроет информацию о разработчиках (см. рисунок 62).

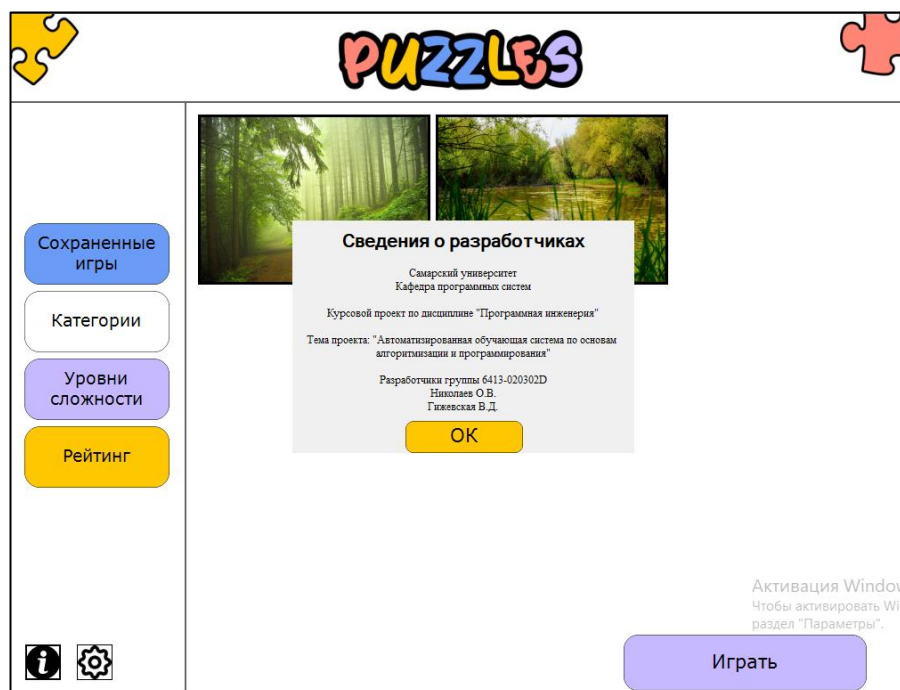


Рисунок 62 – Экранная форма «Сведения о разработчиках»

При нажатии на кнопку «Настройки» система откроет окно, где игрок сможет включить/выключить звук или музыкальное сопровождение, а также прочитать всю интересующую его информацию о системе. Форма «Настройки» показана на рисунке 63.

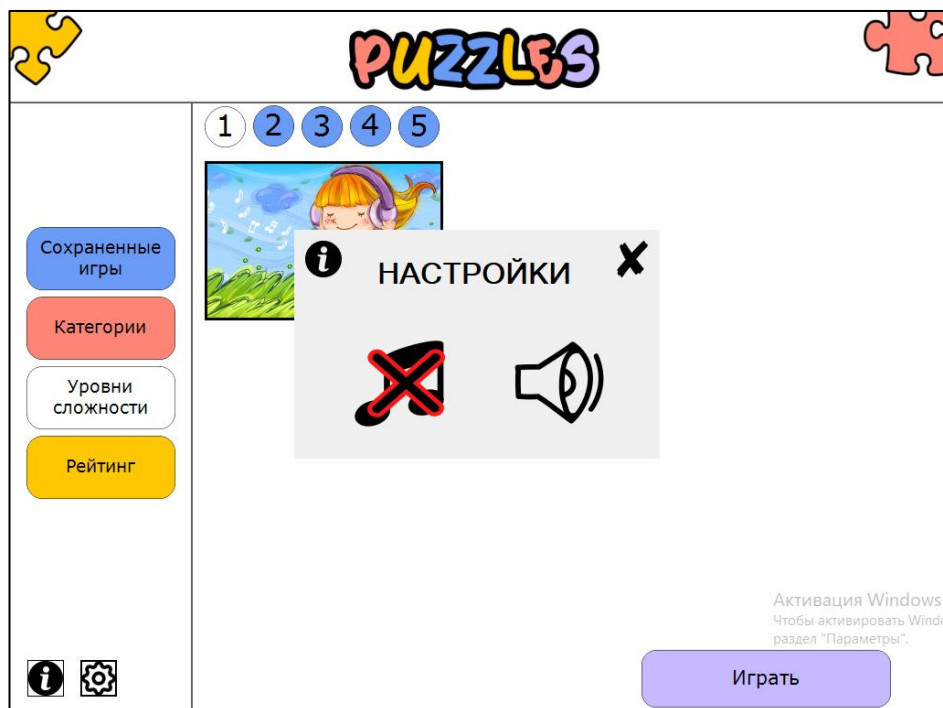


Рисунок 63– Экранная форма настроек приложения

Для начала игры пользователю необходимо выбрать картинку (она подсветится зелёным), а затем нажать кнопку «Играть». После этого система откроет игровое поле.

На рисунке 64 показан прототип игрового поля с типом сборки «на ленте».

Для того чтобы пользователь смог увидеть исходную картинку, ему необходимо будет нажать на иконку , в правом верхнем углу. Пример просмотра исходной картинки показан на рисунке 65.

Взятие подсказки происходит следующим образом: пользователь нажимает на иконку  в правом верхнем углу, затем система ставит фрагмент в место, на которое он должен встать. После установки фрагмента или группы фрагментов в правильное место, перемещение становится невозможным (см. рисунок 66).

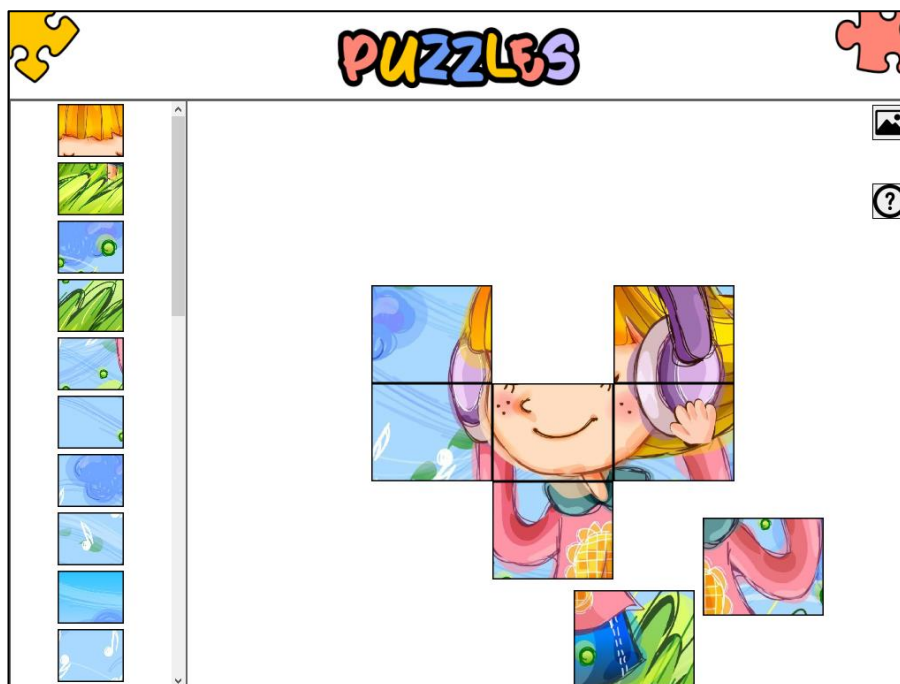


Рисунок 64 – Экранная форма игрового поля с типом сборки «на ленте»



Рисунок 65 – Экранная форма просмотра исходной картинки

Для каждого уровня сложности задано конкретное количество подсказок. После того, как все возможные подсказки для этой игры будут потрачены, кнопка подсказки исчезнет из игрового меню. Это показано на рисунке 66.

При нажатии иконки , система откроет пользователю страницу с информацией о данном приложении.

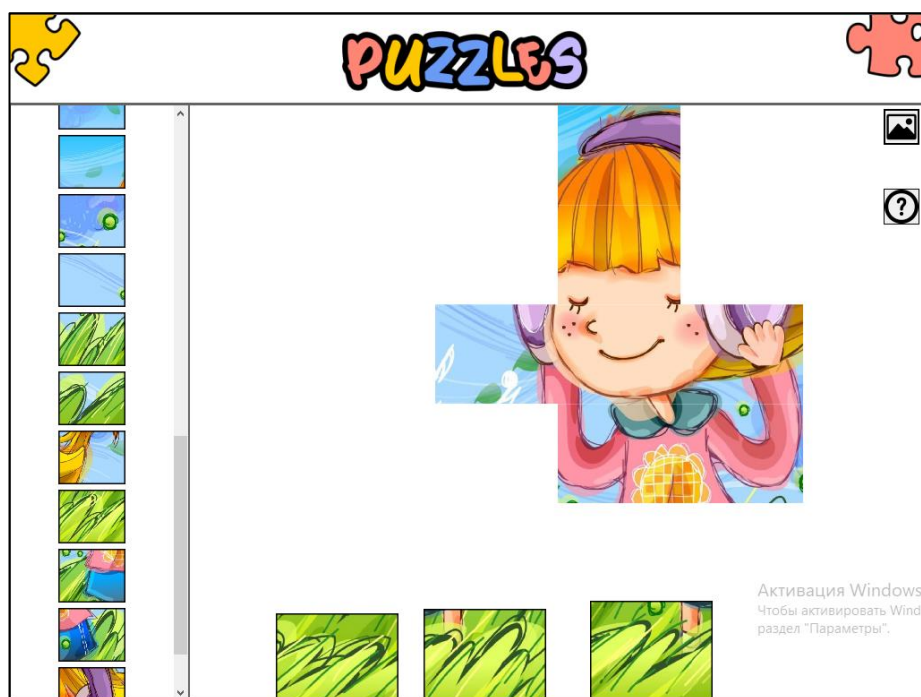


Рисунок 66 – Игровое поле после взятия подсказки

При типе сборки «в куче», пользователю необходимо искать подходящие друг к другу фрагменты. При типе сборки «вперемешку», игрок должен выбрать два фрагмента, которые он хочет поменять местами. При правильной сборке фрагменты будут склеиваться.

После завершения игры пользователю начислятся баллы, которые в дальнейшем учитываются в рейтинге (см. рисунок 67).

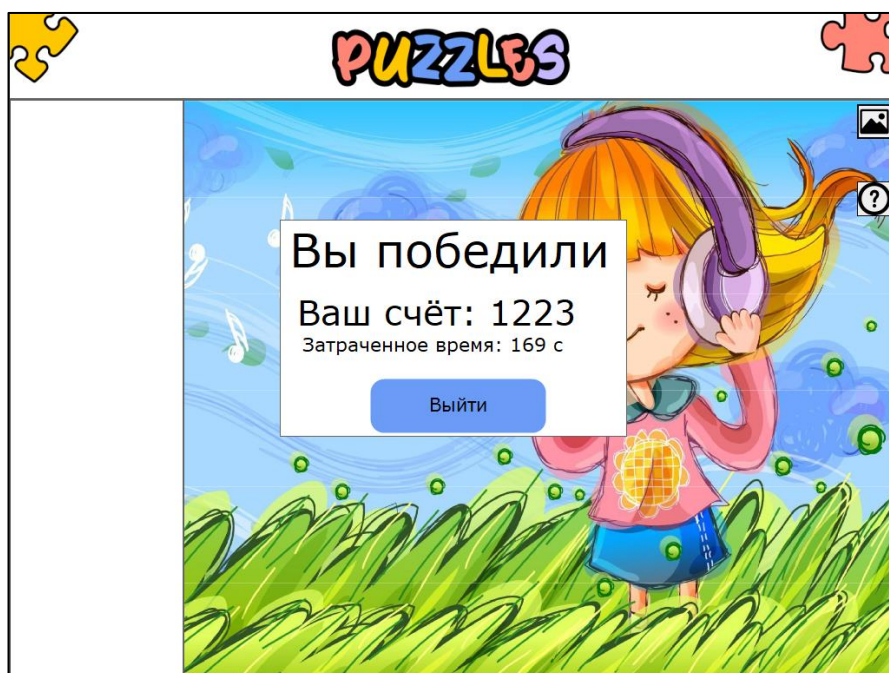


Рисунок 67 – Окончание игры

Если пользователь захочет выйти из приложения, ему необходимо нажать на клавишу «esc», после чего система уточнит его выбор и после нажатия на кнопку «Выйти» завершит работу приложения (см. рисунок 68).

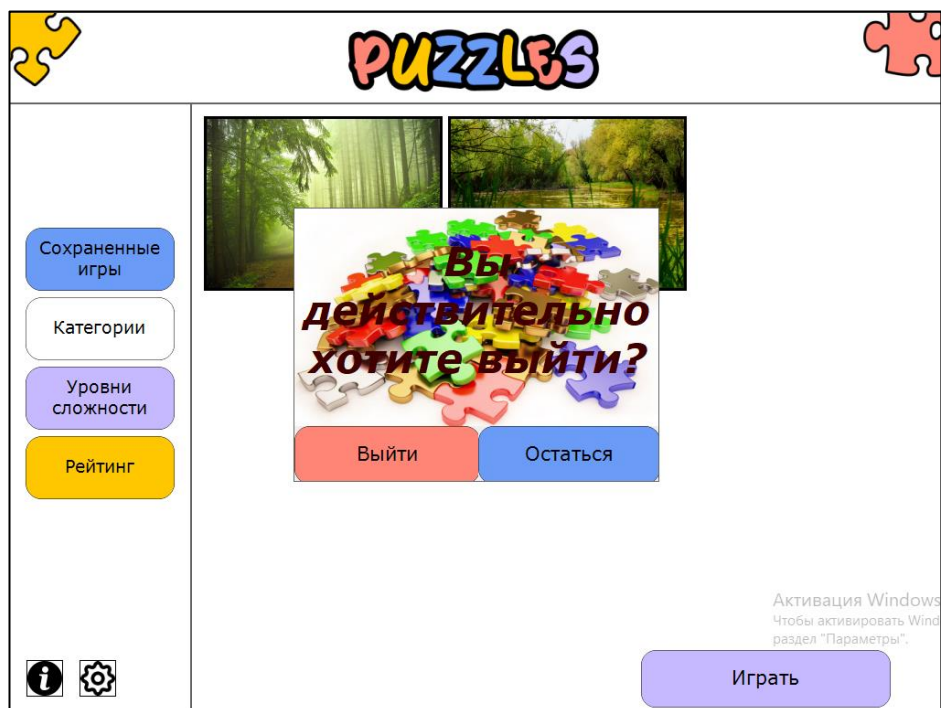


Рисунок 68 – Выход из приложения

3.2 Диаграммы реализации

Диаграммы реализации предназначены для отображения состава компилируемых и выполняемых модулей системы, а также связей между ними. Диаграммы реализации разделяются на два конкретных вида: диаграммы компонентов (component diagrams) и диаграммы развертывания (deployment diagrams) [42].

3.2.1 Диаграмма компонентов

Диаграмма компонентов описывает особенности физической реализации приложения, определяет архитектуру приложения и устанавливает зависимость между компонентами, в роли которых выступает исполняемый код. Диаграмма компонентов отображает общие зависимости между компонентами. Основными графическими элементами диаграммы являются компоненты, интерфейсы и зависимости между ними [43].

Диаграмма компонентов отображена на рисунке 69. Компоненты, входящие в диаграмму представлены в таблице 10.

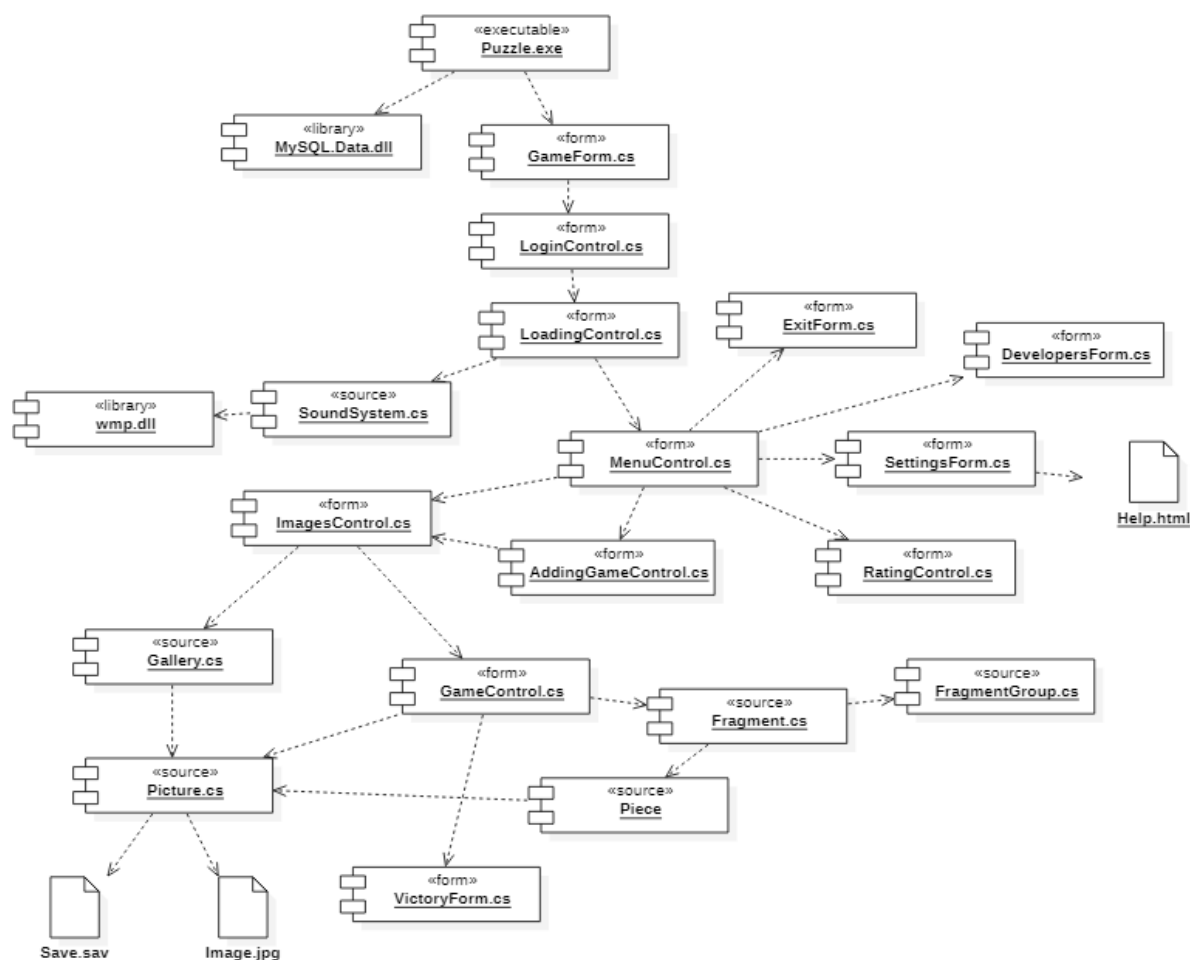


Рисунок 69 – Диаграмма компонентов системы

Таблица 10 – Основные компоненты системы

Название	Назначение
1	2
Puzzle.exe	Исполняемый файл программы
MySQL.Data.dll	Библиотека MySQL, отвечающая за взаимодействие с сервером базы данных
GameForm.cs	Основная форма приложения, отвечает за правильное отображение окна в полноэкранном режиме.
LoginControl.cs	Элемент управления, отвечающий за регистрацию и вход в систему.

Продолжение таблицы 10

1	2
LoadingControl.cs	Элемент управления, отображающий процесс загрузки данных с сервера
SoundSystem.cs	Исходный файл класса, определяющий звуки, используемые в игре
Wmp.dll	Библиотека Windows Media Player, используемая для работы со звуком
MenuControl.cs	Элемент управления, отображающий меню, которое содержит множество переходов на другие элементы управления
ExitForm.cs	Форма подтверждения выхода из приложения
DevelopersForm.cs	Форма, отображающая информацию о разработчиках
SettingsForm.cs	Форма включения/выключения звука и музыки
RatingControl.cs	Элемент управления, который отображает 10 лучших игроков
AddingGameControl.cs	Элемент управления добавления параметров для новой игры
ImagesControl.cs	Элемент управления, представляющий таблицу изображений, отображающее подмножество всех доступных изображений, выбранное по определённому критерию
Gallery.cs	Класс, определяющий методы работы с галереей
Picture.cs	Класс, определяющий название файла, загруженное изображение и параметры игры, связанные с ним
GameControl.cs	Элемент управления, регулирующий процесс сборки пазла.

Продолжение таблицы 10

1	2
Fragment.cs	Класс фрагмента, регулирующий перестановку деталей на экране, отвечающий за их соединение и привязку местоположения.
Piece.cs	Класс, отвечающий за отрисовку каждой детали
FragmentGroup.cs	Класс, содержащий список каждой детали, входящей в данную группу
VictoryForm.cs	Форма с сообщением о победе и о достигнутом счёте

3.2.2 Диаграмма развертывания

Диаграмма развёртывания применяется для представления общей конфигурации и топологии распределённой системы и содержит распределение компонентов по определённым узлам. Диаграмма показывает наличие физических соединений, маршрутов передачи информации между аппаратными устройствами, задействованными в реализации системы, содержит графические изображения процессоров, устройств, процессов и связей между ними [44].

Диаграмма развёртывания системы представлена на рисунке 70. На ней отображены следующие взаимосвязи. На компьютере клиента должна быть установлена операционная система Windows. В её файловой системе осуществляется взаимодействие с ресурсами для полноценной работы приложения, такими как файлы изображения и файлы сохранения. Приложение запускается через exe файл. В него встроен файл справки, который отображается браузером по умолчанию. Приложение взаимодействует с сервером базы данных, осуществляя подключение через глобальную сеть.

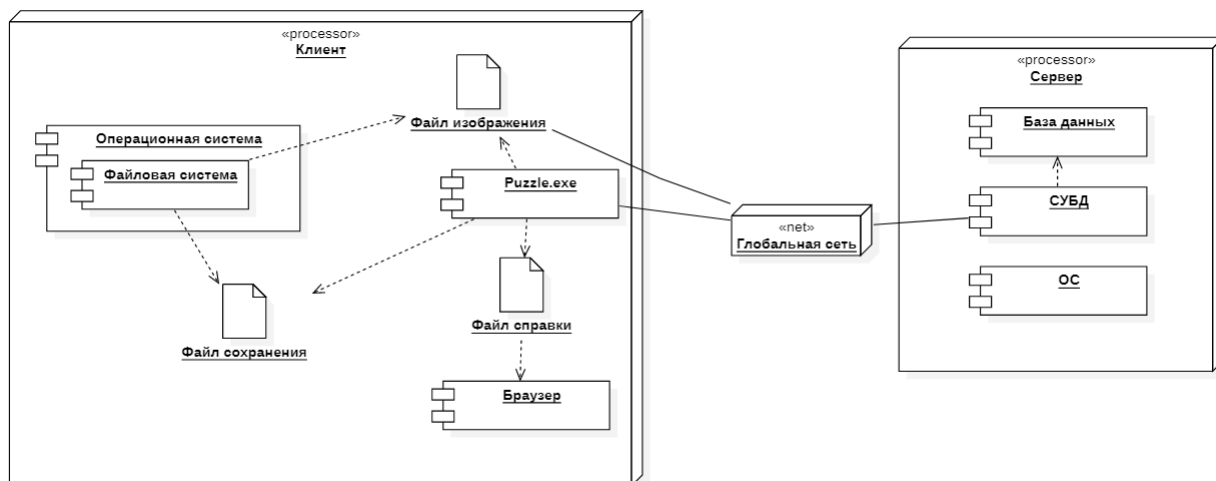


Рисунок 70 – Диаграмма развёртывания системы

3.2.3 Диаграмма классов

В соответствии со спецификацией, приведенной в п. 2.5.6, и с учетом выбранного языка программирования (см. п. 2.8.1) разработана диаграмма классов системы (этап реализации), приведённая на рисунке 71.

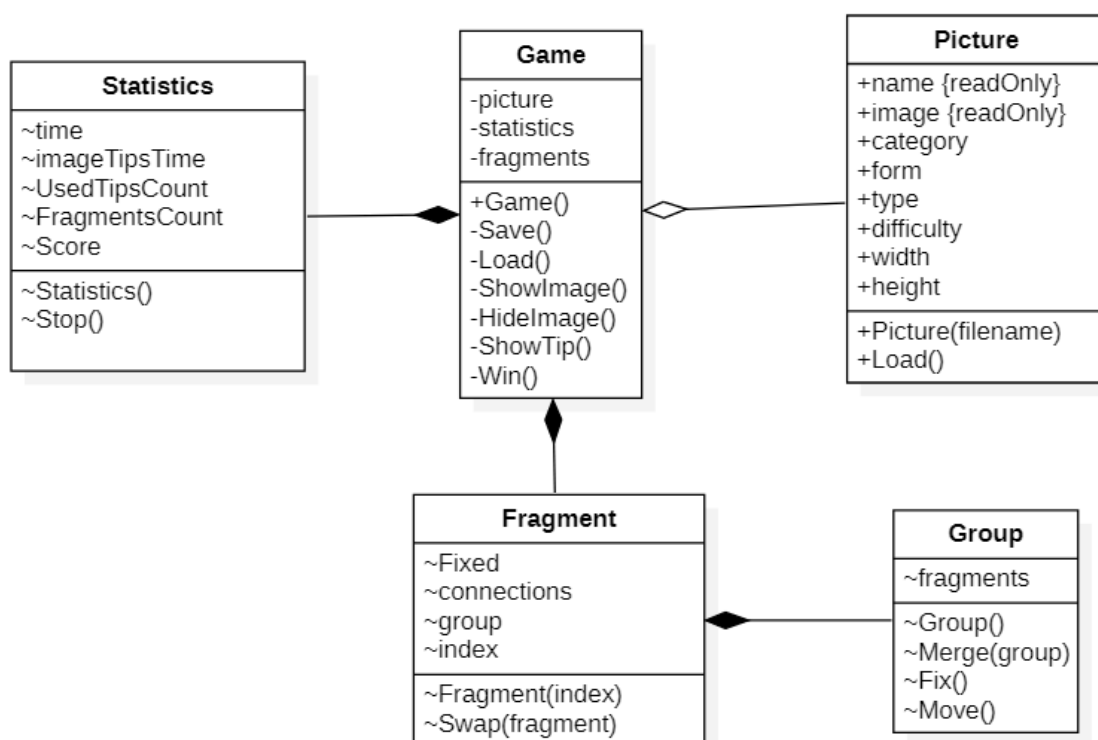


Рисунок 71 – Описание классов системы (этап реализации)

3.3 Физическая модель данных

Физическое проектирование является последним этапом проектирования базы данных, при выполнении которого принимается решение о способах реализации разрабатываемой базы данных. Во время логического проектирования была определена логическая структура базы данных (которая описывает отношения и ограничения в рассматриваемой прикладной области).

Физическая модель базы данных содержит все детали, необходимые конкретной СУБД для создания базы: наименования таблиц и столбцов, типы данных, определения первичных и внешних ключей [45].

На рисунке 72 представлена физическая модель данных системы.

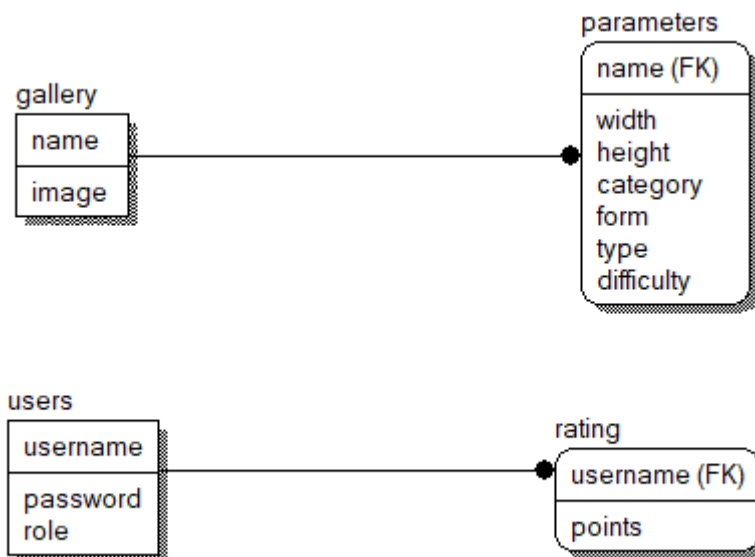


Рисунок 72 – Физическая модель данных системы

В таблицах 11-14 приведено описание сущностей БД.

Таблица 11 – Сущность «Rating»

Атрибут	Тип	Размер (байт)
points	bigint	8

Таблица 12 – Сущность «Gallery»

Атрибут	Тип	Размер (байт)
name	varchar(256)	258
image	mediumblob	16777218
Итого:		16777476

Таблица 13 – Сущность «Users»

Атрибут	Тип	Размер (байт)
username	varchar(8)	10
password	varchar(10)	12
role	tinyint	1
Итого:		23

Таблица 14 – Сущность «Parameters»

Атрибут	Тип	Размер (байт)
width	tinyint	1
height	tinyint	1
category	tinyint	1
form	tinyint	1
type	tinyint	1
difficulty	tinyint	1
Итого:		6

3.4 Выбор и обоснование комплекса технических средств

3.4.1 Расчёт объёма занимаемой памяти клиентской части системы

Расчёт объёма внешней памяти

Для расчёта необходимого объёма свободной внешней памяти, необходимой для функционирования клиентской части системы, воспользуемся следующей формулой:

$$V_{\text{ЖД}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{СПО}} + V_{\text{Ф}} + V_{\text{справки}},$$

где $V_{\text{ОС}}$ – объем памяти, занимаемый операционной системой (операционная система Windows 10 Домашняя, $V_{\text{ОС}} = 32$ Гб);

$V_{\text{ПР}}$ – объем памяти, занимаемый непосредственно файлами приложения ($V_{\text{ПР}} = 6,99$ Мб);

$V_{\text{СПО}}$ – объем памяти, занимаемый сопутствующим программным обеспечением (библиотеки MySql.Data.dll, System.Management.dll, .NET Desktop Runtime 6.0.0, Internet Explorer 9; дадим оценку сверху $V_{\text{СПО}}$ в 3 Гб);

$V_{\text{Ф}}$ – объем памяти, которые используется для хранения файлов, которые создаются в процессе выполнения программы (дадим оценку сверху в 250 Мб);

$V_{\text{справки}}$ – объем памяти, необходимый для хранения файла справки, необходимого для работы программы ($V_{\text{справки}} = 10$ Мб).

Таким образом, суммарный объем внешней памяти составит:

$$V_{\text{ЖД}} = 32 \text{ Гб} + 3 \text{ Гб} + 6,99 \text{ Мб} + 10 \text{ Мб} + 250 \text{ Мб} \approx 35,25 \text{ Гб}.$$

Расчёт объёма ОЗУ

Для расчёта необходимого объёма ОЗУ воспользуемся следующей формулой:

$$V_{\text{ОЗУ}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{БД}} + V_{\text{браузера}},$$

где $V_{\text{ОС}}$ – ОЗУ, занимаемое операционной системой (2 Гб);

$V_{\text{ПР}}$ – ОЗУ, которое займёт само приложение (не превысит 700 Мб);

$V_{\text{БД}}$ – объем данных из базы сервера, который может быть одновременно загружен в оперативную память (дадим ему оценку сверху в 5 Мб).

$V_{\text{браузера}}$ – ОЗУ, занимаемое браузером (для Яндекс.браузера оценим сверху значением в 300 Мб).

Суммарные объёмы ОЗУ составит:

$$V_{\text{ОЗУ}} = 2 \text{ Гб} + 700 \text{ Мб} + 5 \text{ Мб} + 300 \text{ Мб} \approx 3 \text{ Гб}.$$

Таким образом, 3 Гб оперативной памяти можно считать минимально необходимым для функционирования системы.

3.4.2 Минимальные требования, предъявляемые к системе

Для корректного функционирования клиентской части системы необходимо:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 3 Гб;
- объем свободного дискового пространства – не менее 10 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- процессор – Intel Pentium не менее 2,2 ГГц;
- монитор с разрешающей способностью не ниже 800 x 600;
- подключение к интернету;
- операционная система Windows 7 и выше;
- браузер Yandex 22.1.0 и выше;
- .NET Desktop Runtime 6.0.0.

Серверная часть системы расположена на кластере Yandex и определяется его требованиями:

- система управления базами данных MySQL 8.0;
- процессор Intel Cascade Lake 2 Core;
- доля vCPU 20%;
- оперативная память 2 Гб;

- размер хранилища 30 Гб;
- пропускная способность 30Мб/с;
- публичный IP-адрес.

ЗАКЛЮЧЕНИЕ

В процессе выполнения курсового проекта была разработана автоматизированная система «Игра «Puzzle», предназначенная для создания и сборки пазлов.

В первом разделе приведены основные понятия предметной области, рассмотрена классификация пазлов, описаны характеристики систем-аналогов, приведён их сравнительный анализ. На основе проведённого анализа выполнена объектная декомпозиция, отражённая в диаграмме объектов, и сформулирована постановка задачи.

Во втором разделе была описана структурная схема системы, разработаны функциональная спецификация и прототипы экранных форм системы. Также был разработан информационно-логический проект по методологии UML, в который вошли все канонические диаграммы. Кроме того, в этом разделе был выбран комплекс программных средств.

В третьем разделе приведено описание интерфейса пользователя, диаграммы реализации, сделаны ресурсные расчёты и определены технические требования для функционирования системы.

Разработанная система может использоваться пользователями разных возрастов для разнообразия своего досуга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Пазл – история и описание игрушки [Электронный ресурс] // ИгрушкаПоиск: [сайт]. URL: <https://www.i-igrushki.ru/igrushkapedia/pazl.html> (дата обращения: 25.09.2021).
- 2 Буч Г., Рамбо Д., Якобсон А. Язык UML. Руководство пользователя. Изд. 2-е. М.: ДМК Пресс, 2006. 546 с.
- 3 Буч Г., Рамбо Д., Якобсон А. Язык UML ... С. 31.
- 4 Леоненков, А. В. Нотация и семантика языка UML [Электронный ресурс]/ А.В. Леоненков. – Интернет-университет информационных технологий. URL: <http://www.intuit.ru/department/pl/umlbasics> (дата обращения: 01.10.2021).
- 5 Пазлы: виды, назначение и влияние [Электронный ресурс] // Наша речь: [сайт]. URL: <https://logoprav.ru/nadosuge/pazly> (дата обращения: 01.10.2021).
- 6 Советы, как правильно и быстро собирать пазлы с любым количеством деталей [Электронный ресурс] // Паззлись: [сайт]. URL: <https://паззлись.рф/kak-pravilno-sobirat> (дата обращения: 01.10.2021).
- 7 Пазлы – что это такое [Электронный ресурс] // Ёптель: [сайт]. URL: <https://yoptel.ru/pazly-что-это-такое.html> (дата обращения: 01.10.2021).
- 8 Пазл – ДИКИЕ ЖИВОТНЫЕ [Электронный ресурс] // Яндекс.Игры: [сайт]. URL: <https://clck.ru/Y23M6> (дата обращения: 01.10.2021).
- 9 Сказочный пазл [Электронный ресурс] // Яндекс.Игры: [сайт]. URL: <https://clck.ru/Y23MW> (дата обращения: 01.10.2021).
- 10 Анализ предметной области. [Электронный ресурс] // Интуит: [сайт]. URL: <http://www.intuit.ru/studies/courses/574/430/lecture/9749> (дата обращения: 04.10.2021).
- 11 Архитектура программного обеспечения. [Электронный ресурс] // Википедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Архитектура_программного_обеспечения (дата обращения: 19.10.2021).

12 4 типа архитектуры программного обеспечения. Детальный обзор существующих подходов [Электронный ресурс] // MEDIUM: [сайт]. URL: <https://medium.com/nuances-of-programming/4-типа-архитектурыпрограммно-о-обеспечения-917133174724> (дата обращения: 19.10.2021).

13 Архитектура «Клиент-Сервер». [Электронный ресурс] // ITELON: [сайт]. URL: <https://itelon.ru/blog/arkhitektura-klient-server> (дата обращения: 19.10.2021).

14 Протокол передачи данных. [Электронный ресурс] // Википедия: [сайт]. URL: https://ru.wikipedia.org/wiki/Протокол_передачи_данных (дата обращения: 19.10.2021).

15 Сетевые протоколы: типы и назначение. [Электронный ресурс] // WORLD-X: [сайт]. URL: <https://wd-x.ru/setevye-protokoly-tipy-i-naznachenie> (дата обращения: 19.10.2021).

16 Буч Г., Рамбо Д., Якобсон А. Язык UML ... С. 496.

17 Большой Российский энциклопедический словарь. М.: БРЭ, 2003.

18 Автоматизация управления типовыми объектами производства [Текст]. - М.: КНИТУ, 2020 (дата обращения: 13.10.2021).

19 Структурный подход к проектированию ИС. [Электронный ресурс] // CITforum: [сайт]. URL: http://citforum.ru/database/case/glava2_1.shtml (дата обращения: 13.10.2021).

20 Разработка спецификации требований [Электронный ресурс] // StudFiles: [сайт]. URL: <https://studfile.net/preview/2113734/page:6/> (дата обращения: 20.10.2021).

21 Требования к ПО [Электронный ресурс] // СТУДЕПЕДИЯ: [сайт]. URL: <https://studepedia.org/index.php?vol=2&post=20737> (дата обращения: 20.10.2021).

22 Функциональные и нефункциональные требования [Электронный ресурс] // StudFiles: [сайт]. URL: <https://studfile.net/preview/2152457/page:4/> (дата обращения: 29.10.2021).

23 Разработка прототипа интерфейса пользователя системы [Электронный ресурс] // StudFiles: [сайт]. URL: <https://studfile.net/preview/2114024/page:7/> (дата обращения: 01.11.2021).

24 Информационно-логическое проектирование. [Электронный ресурс] // Studwood.ru: [сайт]. URL: https://studwood.ru/1884856/informatika/informatsionno_logicheskoe_proektirovanie (дата обращения: 15.11.2021).

25 UML [Электронный ресурс] // Аналитик: [сайт]. URL: <https://www.sites.google.com/site/infoprobusinessanalysis/project-definition/uml> (дата обращения: 15.11.2021).

26 Дополнительные обозначения языка UML для бизнес-моделирования [Электронный ресурс] // Helpiks.org: [сайт]. URL: <https://helpiks.org/8-35678.html> (дата обращения: 19.11.2021).

27 Диаграмма состояний [Электронный ресурс] // Студопедия.нет: [сайт]. URL: https://studopedia.net/1_40401_diagramma-sostoyaniy-ee-naznachenie-ispolzovanie-elementi-graficheskoy-notatsii-diagrammi-sostoyaniy-sostoyanie-perehod-sobitie-storozhevoe-uslovie-virazhenie-deystviya.html (дата обращения: 23.11.2021).

28 Диаграмма деятельности (activity diagram) [Электронный ресурс] // masters.donntu.org: [сайт]. URL: <https://masters.donntu.org/2005/kita/shapovalova/library/uml2.html> (дата обращения: 24.11.2021).

29 Диаграмма последовательности [Электронный ресурс] // StudFiles: [сайт]. URL: <https://studfile.net/preview/5187971/page:4/> (дата обращения: 24.11.2021).

30 Диаграмма классов [Электронный ресурс] // Студопедия: [сайт]. URL: <https://studopedia.info/10-59449.html> (дата обращения: 25.11.2021).

31 Модель «Сущность-связь» [Электронный ресурс] // Студопедия: [сайт]. URL: https://studopedia.ru/7_172654_model-sushchnost--svyaz.html (дата обращения: 24.11.2021).

32 Краткое описание ER-метода [Электронный ресурс] // Кафедра алгоритмических языков ВМК МГУ: [сайт]. URL: https://al.cs.msu.ru/system/files/ER_method.pdf (дата обращения: 03.12.2021).

33 Преобразование информации в данные. Алгоритмы преобразования данных [Электронный ресурс] // Мультиурок: [сайт]. URL: <https://multiurok.ru/files/3-algoritmy-preobrazovaniia-dannykh-1-poniatie-alg.html> (дата обращения: 10.12.2021).

34 Конспект Способы описания алгоритмов [Электронный ресурс] // Инфорурок: [сайт]. URL: <https://infourok.ru/konspekt-sposobi-opisaniya-algoritmov-966802.html> (дата обращения: 10.12.2021).

35 Язык программирования C#: история, специфика, место на рынке [Электронный ресурс] // GeekBrains: [сайт]. URL: <https://gb.ru/posts/yazyk-programirovaniya-c-sharp-istoriya-spezifika-mesto-na-rynke> (дата обращения: 15.12.2021).

36 Обзор основных видов операционных систем [Электронный ресурс] // geekon.media: [сайт]. URL: <https://geekon.media/vidy-operacionnyh-sistem-raznye-os-na-kompyuter-i-smartfon/> (дата обращения: 15.12.2021).

37 ТОП-10 операционных систем для ПК: чем отличаются и какие у них преимущества [Электронный ресурс] // TRASHBOX.RU: [сайт]. URL: <https://trashbox.ru/link/best-operating-systems-for-pc> (дата обращения: 15.12.2021).

38 Знакомство с Visual Studio [Электронный ресурс] // Хабр: [сайт]. URL: <https://habr.com/ru/sandbox/79099/> (дата обращения: 15.12.2021).

39 Официальный релиз Microsoft Visual Studio 2022 и .NET 6 [Электронный ресурс] // comss.ru: [сайт]. URL: <https://www.comss.ru/page.php?id=9818> (дата обращения: 15.12.2021).

40 MySQL [Электронный ресурс] // Википедия: [сайт]. URL: <https://ru.wikipedia.org/wiki/MySQL> (дата обращения: 15.12.2021).

41 Что Такое MySQL: Объяснение MySQL Для Начинающих [Электронный ресурс] // HOSTINGER: [сайт]. URL: <https://www.hostinger.ru/rukovodstva/shto-takoe-mysql/> (дата обращения: 15.12.2021).

42 Диаграммы реализации [Электронный ресурс] // maksakov-sa.ru: [сайт]. URL: <http://www.maksakov-sa.ru/ModelUML/DiagrReal/index.html> (дата обращения: 23.12.2021).

43 Диаграмма компонентов (component diagram) [Электронный ресурс] // Студопедия: [сайт]. URL: https://studopedia.ru/6_33179_diagramma-klassev-class-diagram.html (дата обращения: 23.12.2021).

44 Соединения и зависимости на диаграмме развертывания [Электронный ресурс] // Студопедия.нет: [сайт]. URL: https://studopedia.net/4_36259_soedineniya-i-zavisimosti-na-diagramme-razvertivaniya.html (дата обращения: 23.12.2021).

45 Физическое проектирование базы данных [Электронный ресурс] // Студопедия: [сайт]. URL: https://studopedia.ru/8_74489_fizicheskoe-proektirovanie-bazi-dannih.html (дата обращения: 23.12.2021).

ПРИЛОЖЕНИЕ А

Руководство пользователя

А.1 Назначение системы

Данная система предназначена для создания и сборки пазла, с её помощью администратор сможет настраивать игры разных уровней сложности, а игрок составлять пазлы, используя систему подсказок.

А.2 Условия работы системы

Для корректной работы клиентской части системы необходимо наличие соответствующих программных и аппаратных средств.

1 Требования к техническому обеспечению:

- ЭВМ типа: x86-64 совместимый;
- процессор типа x86 или x64 тактовой частоты 2200 МГц и выше;
- монитор с разрешающей способностью не ниже 800 x 600;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- подключение к интернету;
- объем ОЗУ: не менее 3 Гб;
- объём ПЗУ: не менее 40 Гб.

2 Требования к программному обеспечению:

- операционная система Windows 7 Home и выше;
- установленная платформа .NET Desktop версии 6.0.0 и выше;
- браузер Yandex 22.1.0 и выше.

Серверная часть системы расположена на кластере Yandex и определяется его требованиями:

- система управления базами данных MySQL 8.0;
- процессор Intel Cascade Lake 2 Core;

- доля vCPU 20%;
- оперативная память 2 Гб;
- размер хранилища 30 Гб;
- пропускная способность 30Мб/с;
- публичный IP-адрес.

А.3 Установка системы

Система поставляется в виде zip-архива. Данный файл необходимо распаковать в любую директорию на жёстком диске. Запускаемым файлом системы является файл Puzzle.exe. Если не будет установлена соответствующая библиотека исполнения, то приложение направит пользователя на страницу её установки. Необходимо загрузить .Net desktop runtime с разрядностью, соответствующей устройству и запустить инсталлятор. После установки платформы приложение может быть запущено.

А.4 Работа с системой

На рисунке А.1 приведена начальная форма приложения. Здесь пользователь должен ввести логин и пароль и нажать кнопку «Войти» или «Регистрация», после чего система выполнит вход, либо же зарегистрирует нового пользователя.

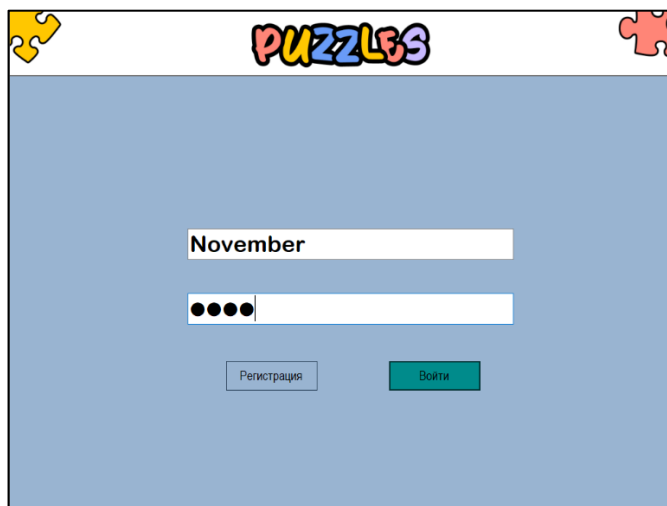


Рисунок А.1 – Начальная форма приложения

А.4.1 Работа с системой в режиме администратора

При авторизации пользователя в роли «Администратор» откроется форма с галереей. Данная экранная форма представлена на рисунке А.2. Здесь находятся картинки, которые добавлены администратором, но ещё не используются в играх. Администратору предоставлена возможность удалить картинку из галереи, а также добавить новую картинку из файла (*.png). Данный процесс показан на рисунке А.3.

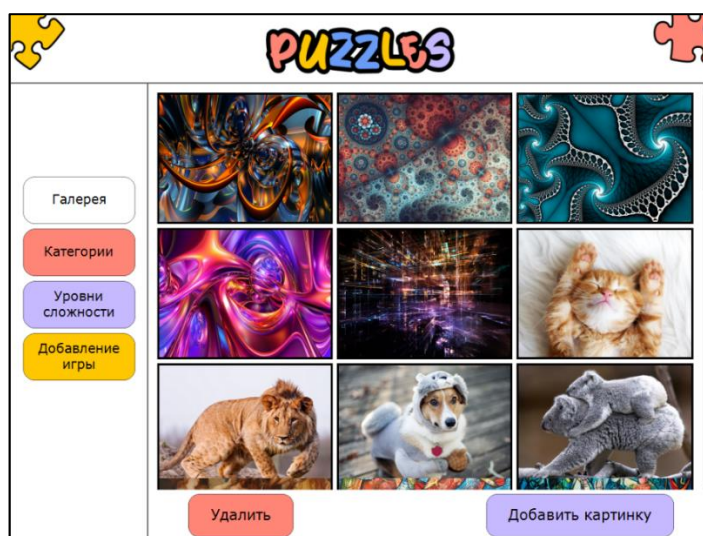


Рисунок А.2 – Экранная форма вкладки «Галерея»

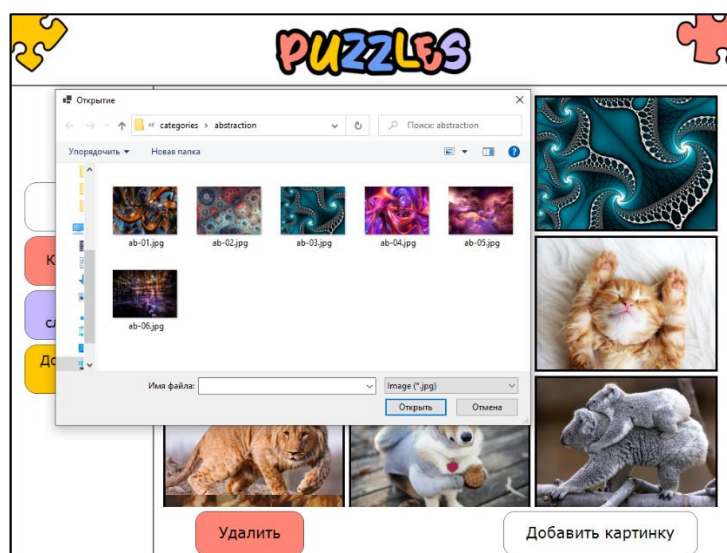


Рисунок А.3 – Добавление новой картинки

Вкладки «Категории» и «Уровни сложности» (см. рисунок А.4) позволяют производить навигацию по каталогу игр, которые доступны

пользователю с ролью «Игрок». Также в функции администратора входят удаление игры или изменение её настроек. Для этих действий необходимо сначала выбрать пазл (он обводится зелёной рамкой), а затем нажать на соответствующую кнопку. При нажатии на кнопку «Удалить», данная игра удалится из каталога игр, а также из БД. При нажатии на кнопку «Изменить игру», администратор увидит форму изменения игры, которая изображена на рисунке А.5.

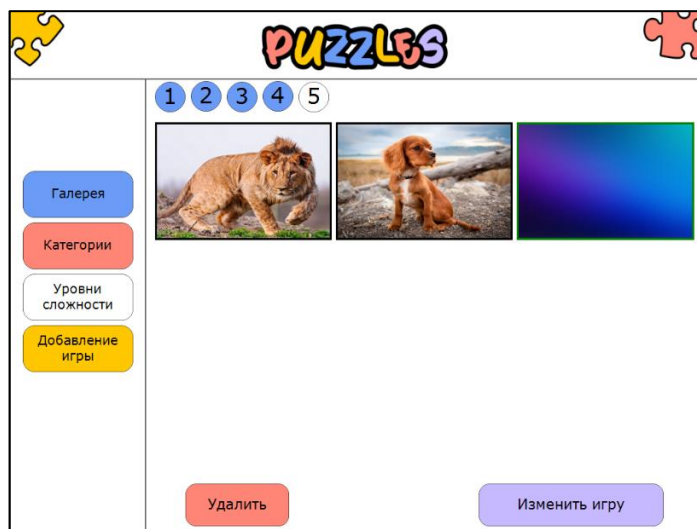


Рисунок А.4 – Экранная форма вкладки «Уровни сложности»

В форме изменения игры, администратор может изменять параметры (форму детали, размерность, тип сборки, категорию и уровень сложности), указанные ранее для этой игры. После чего необходимо нажать кнопку «Изменить». После сохранения изменённая игра появится в каталоге игр.

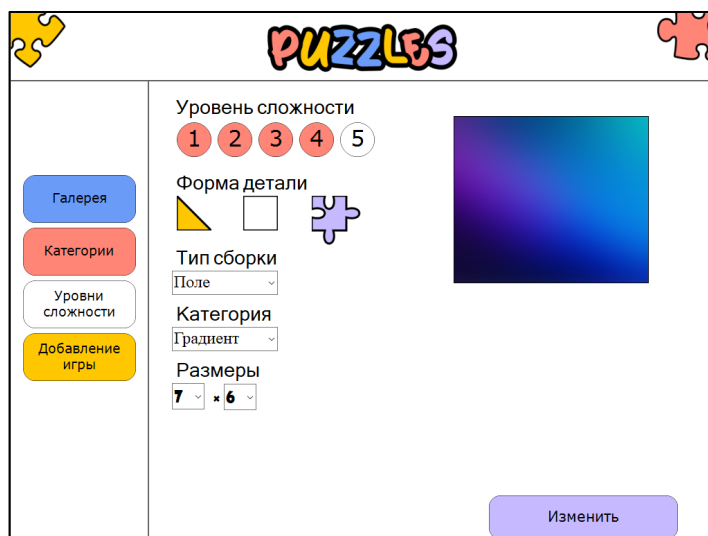


Рисунок А.5 – Экранная форма изменения игры

Во вкладке «Добавление игры» (см. рисунок А.6), у администратора есть возможность добавить в каталог игр новый пазл. Для этого нужно выбрать картинку из галереи, задать для неё уровень сложности и остальные необходимые параметры (размерность, тип сборки, категорию) и нажать кнопку «Добавить игру».

The screenshot shows the 'Add Game' interface. On the left is a sidebar with buttons: 'Галерея' (Gallery), 'Категории' (Categories), 'Уровни сложности' (Difficulty Levels), and 'Добавление игры' (Add Game). The main area contains the following controls: 'Уровень сложности' (Difficulty Level) with radio buttons 1, 2, 3, 4, 5; 'Форма детали' (Piece Shape) with three icons (triangle, square, and a standard puzzle piece); 'Тип сборки' (Assembly Type) with a dropdown menu set to 'Лента' (Strip); 'Категория' (Category) with a dropdown menu set to 'Животные' (Animals); 'Размеры' (Dimensions) with two dropdown menus set to '7' and '5'; and a 'Добавить игру' (Add Game) button at the bottom right. A preview image of a kitten is shown on the right.

Рисунок А.6 – Экранная форма вкладки «Добавление игры»

А.4.2 Работа с системой в режиме пользователя

При выборе роли «Игрок» система откроет вкладку «Сохранённые игры», где хранятся все пазлы, которые он не закончил собирать (см. рисунок А.7).

The screenshot shows the 'Saved Games' interface. On the left is a sidebar with buttons: 'Сохранённые игры' (Saved Games), 'Категории' (Categories), 'Уровни сложности' (Difficulty Levels), and 'Рейтинг' (Rating). The main area displays three puzzle images: a dog, a girl, and a landscape. At the bottom, there are icons for information and settings, and buttons for 'Удалить' (Delete) and 'Загрузить' (Load).

Рисунок А.7 – Экранная форма вкладки «Сохранённые игры»

Вкладка «Категории» представлена на рисунке А.8.

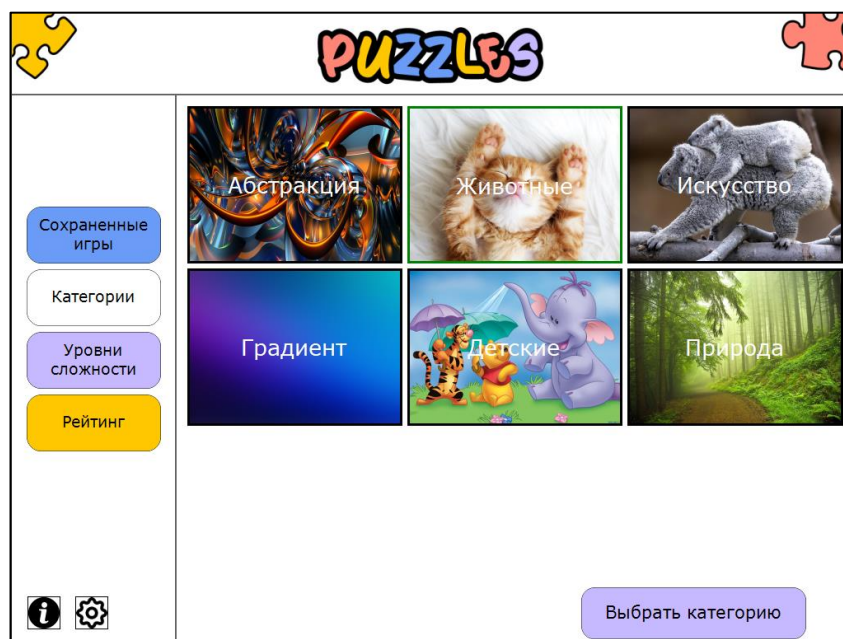


Рисунок А.8 – Экранная форма вкладки «Категории»

В данной вкладке все картинки распределены по категориям. После выбора конкретной категории, пользователь сможет выбрать интересующий его пазл из возможных картинок данной категории (см. рисунок А.9).

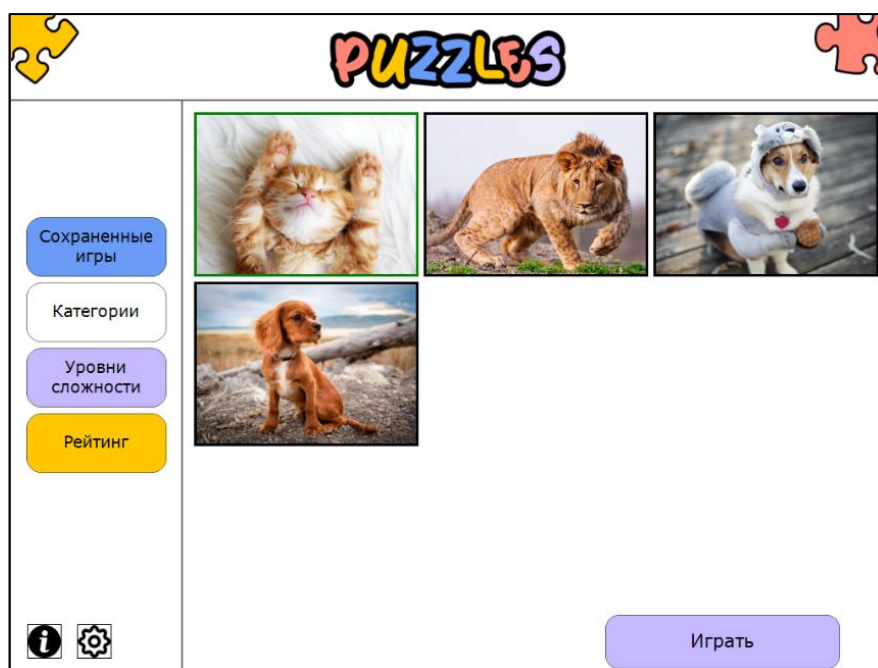


Рисунок А.9 – Экранная форма категории «Животные»

Во вкладке «Рейтинг», пользователь может изучить список из 10 игроков, занимающих лидерские позиции (см. рисунок А.10).



Рисунок А.10 – Экранная форма вкладки «Рейтинг»

Во вкладке «Уровни сложности» игрок может выбрать пазл, исходя из необходимого ему уровня сложности. Переключаться между уровнями можно через специальные кнопки над картинками. Вкладка «Уровни сложности» представлена на рисунке А.11.

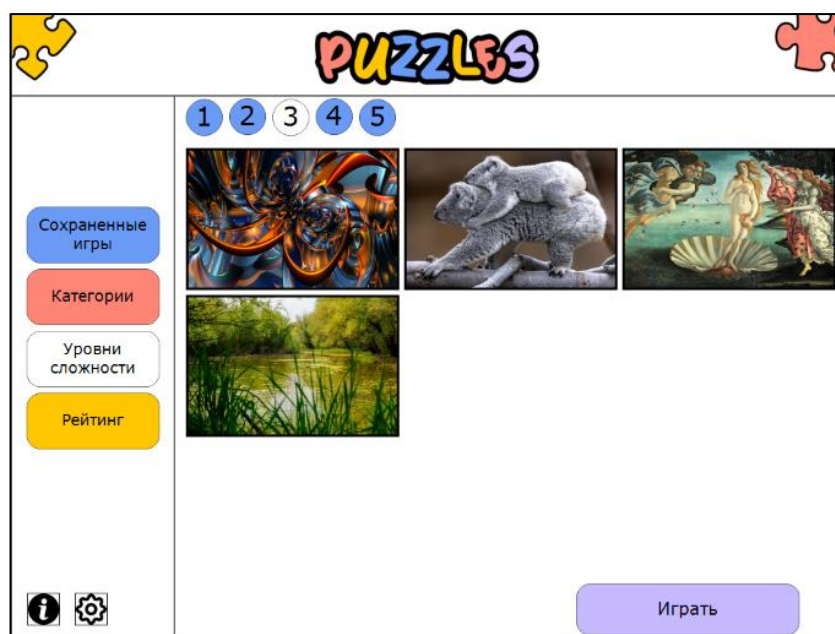


Рисунок А.11 – Экранная форма вкладки «Уровни сложности»

При нажатии на кнопку **i**, система откроет информацию о разработчиках (см. рисунок А.12).

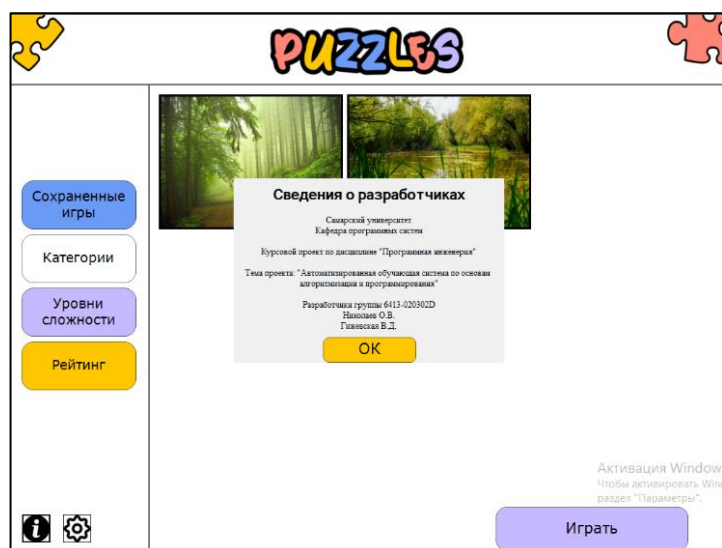


Рисунок А.12 – Экранная форма «Сведения о разработчиках»

При нажатии на кнопку «Настройки» система откроет окно, где игрок сможет включить/выключить звук или музыкальное сопровождение, а также прочитать всю интересующую его информацию о системе. Форма «Настройки» показана на рисунке А.13.



Рисунок А.13– Экранная форма настроек приложения

Для начала игры пользователю необходимо выбрать картинку (она подсветится зелёным), а затем нажать кнопку «Играть». После этого, система откроет игровое поле.

На рисунке А.14 показан прототип игрового поля с типом сборки «на ленте».

Для того чтобы пользователь смог увидеть исходную картинку, ему необходимо будет нажать на иконку , в правом верхнем углу. Пример просмотра исходной картинки показан на рисунке А.15.

Взятие подсказки происходит следующим образом: пользователь нажимает на иконку  в правом верхнем углу, затем система ставит фрагмент в место, на которое он должен встать. После установки фрагмента или группы фрагментов в правильное место, перемещение становится невозможным (см. рисунок А.16).

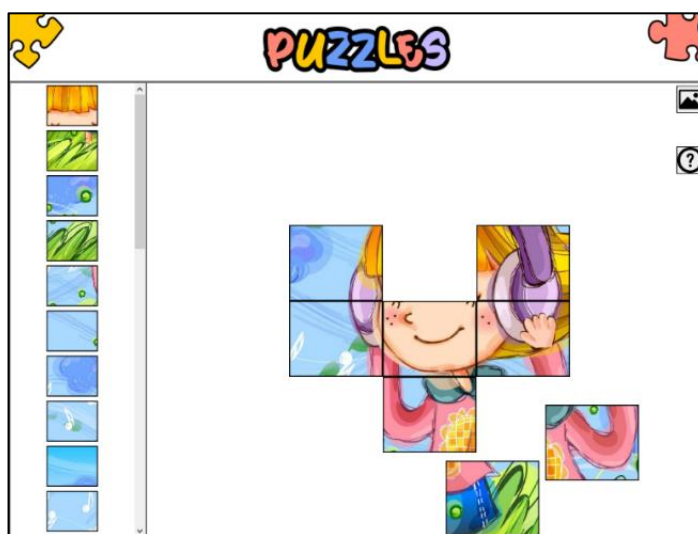


Рисунок А.14 – Экранная форма игрового поля с типом сборки «на ленте»



Рисунок А.15 – Экранная форма просмотра исходной картинки

Для каждого уровня сложности задано конкретное количество подсказок. После того, как все возможные подсказки для этой игры были

потрачены, кнопка подсказки исчезает из игрового меню. Это показано на рисунке А.16.

При нажатии иконки , система откроет пользователю страницу с информацией о данном приложении.

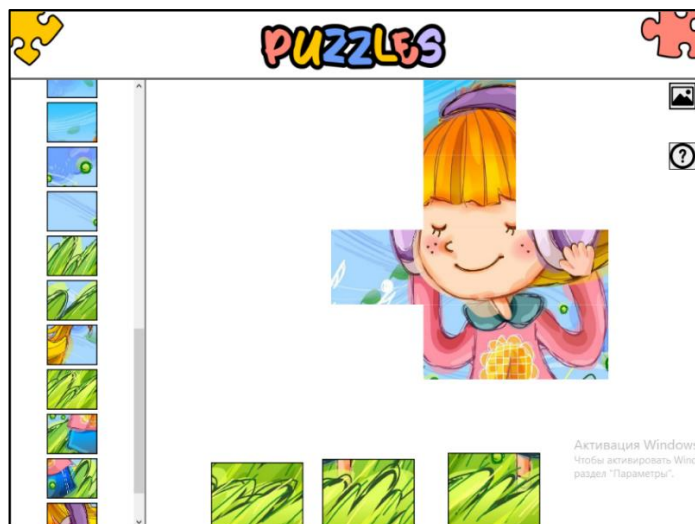


Рисунок А.16 – Игровое поле после взятия подсказки

При типе сборки «в куче», пользователю необходимо искать подходящие друг к другу фрагменты. При типе сборки «вперемешку», игрок должен выбрать два фрагмента, которые он хочет поменять местами. При правильной сборке фрагменты будут склеиваться.

После завершения игры пользователю будут начислены баллы, которые в дальнейшем учитываются в рейтинге (см. рисунок А.17).

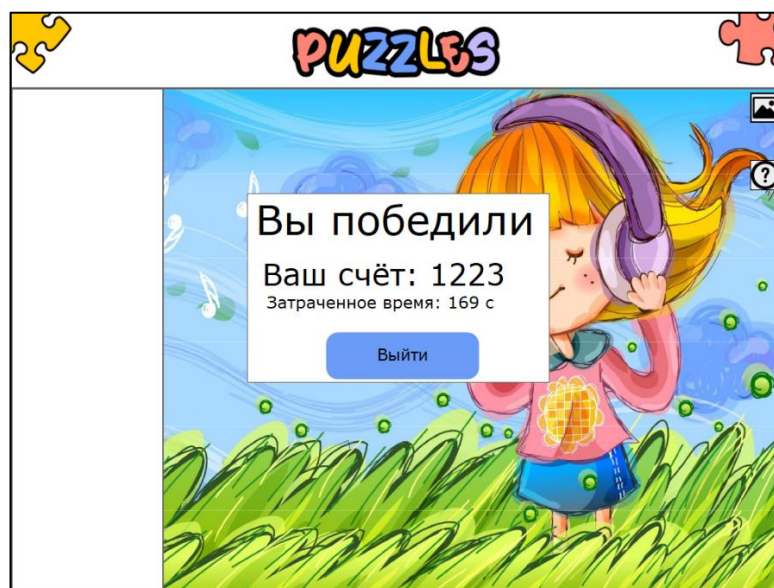


Рисунок А.17 – Окончание игры

Если пользователь захочет выйти из приложения, ему необходимо нажать на клавишу «esc», после чего система уточнит его выбор и после нажатия на кнопку «Выйти» завершит работу приложения (см. рисунок А.18).

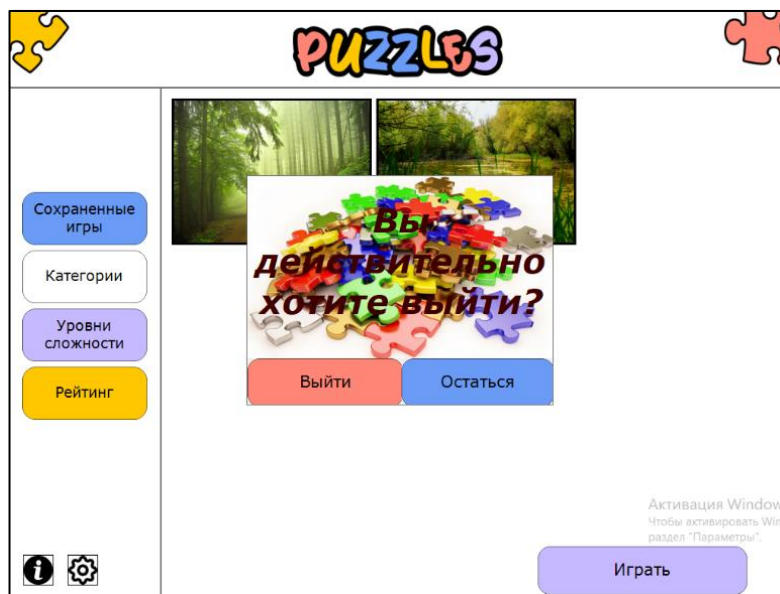


Рисунок А.18 – Выход из приложения

ПРИЛОЖЕНИЕ Б

Листинг элемента управления GameController

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Diagnostics;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Runtime.Serialization.Formatters.Binary;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Puzzle.GameControl
{
    public partial class GameController : UserControl
    {
        #region -- AntiFlickering --
        int originalExStyle = -1;
        private static bool enableFormLevelDoubleBuffering = true;
        public bool AntiFlicker
        {
            get => enableFormLevelDoubleBuffering;
            set
            {
                enableFormLevelDoubleBuffering = value;
            }
        }
        protected override CreateParams CreateParams
        {
            get
            {
                if (originalExStyle == -1)
                    originalExStyle = base.CreateParams.ExStyle;

                CreateParams cp = base.CreateParams;
                if (enableFormLevelDoubleBuffering)
                    cp.ExStyle |= 0x02000000; // WS_EX_COMPOSITED
                else
                    cp.ExStyle = originalExStyle;

                return cp;
            }
        }
        #endregion
        #region -- Cells --
        private int cellsCount = 0;
        private const int rows = 10;
        private int CellsCount
        {
            get => cellsCount;
            set
            {
                cellsCount = value;
                int expectedRows = cellsCount;
            }
        }
    }
}
```

```

        while (expectedRows > tapePanel.RowCount)
        {
            tapePanel.RowCount++;
            tapePanel.RowStyles.Add(new RowStyle(SizeType.Absolute, Height / rows));
        }
        while (expectedRows < tapePanel.RowCount)
        {
            tapePanel.RowCount--;
            tapePanel.RowStyles.RemoveAt(0);
        }
    }
}
#endregion
public class Statictics {
    [JsonIgnore]
    internal GameControl control;
    public Statictics() { }
    public Statictics(GameControl c) {
        control = c;
        time.Start();
        FragmentsCount = c.w * c.h * (c.form == Picture.Form.Triangular ? 2 : 1);
    }
    [JsonIgnore]
    public Stopwatch time=new();
    [JsonIgnore]
    public Stopwatch imageTipTime =new();
    public int TimeOffset { get; set; } = 0;
    public int TipTimeOffset { get; set; } = 0;
    public int UsedTipsCount { get; set; } = 0;
    public long FragmentsCount { get; set; } = 0;
    [JsonIgnore]
    public const int AverageSpeed = 50;
    [JsonIgnore]
    public int Time => (int)time.Elapsed.TotalSeconds+TimeOffset;
    [JsonIgnore]
    public int TipTime=> (int)time.Elapsed.TotalSeconds+TipTimeOffset;
    [JsonIgnore]
    public long Score=> (1000+300*control.p.difficulty)*(FragmentsCount-UsedTipsCount)*
AverageSpeed/(Time+10*TipTime);
    public void Stop() {
        time.Stop();
        imageTipTime.Stop();
        TimeOffset = Time;
        TipTimeOffset = TipTime;
        time.Reset();
        imageTipTime.Reset();
    }
}
Statictics statistics;
public GameControl()
{
    InitializeComponent();
}
internal Picture p;
public GameControl(Picture p,bool load=false){
    InitializeComponent();
    if (this.load=load)
        LoadGame(p);
    InitWithPicture(p);
}
private void InitWithPicture(Picture p) {
    this.p = p;
}

```

```

        form = p.form;
        type = p.type;
        img = p.image;
        w = p.width;
        h = p.height;
        if(tipsCount== -1)
            tipsCount = p.difficulty;
    }
    List<Fragment> fragments = new();
    Picture.Form form = Picture.Form.Square;
    Picture.Type type= Picture.Type.Heap;
    Bitmap bmp;
    Image img;
    int pieceWidth;
    int pieceHeight;
    Size pieceSize;
    Point delta;
    int w = 8;
    int h = 4;
    bool load=false;
    int tipsCount=-1;
    protected override void OnLoad(EventArgs e)
    {
        base.OnLoad(e);
        panel1.Visible = type == Picture.Type.Tape;
        heapPanel.Visible = false;
        imagePanel.BackgroundImage = img;
        imagePanel.BringToFront();
        imageButton.BringToFront();
        tipButton.BringToFront();
    }
    private void AddPiece(int index,Piece piece) {
        Fragment fragment;
        fragments.Add(fragment=new Fragment(index, bmp, piece,pieceSize));
        if(type==Picture.Type.Field)
            piece.Click += button_Click;
        else{
            piece.MouseDown += button_MouseDown;
            piece.MouseUp += button1_MouseUp;
            piece.MouseMove += button_MouseMove;
        }
        if (form != Picture.Form.Triangular)
        {
            if (index % w != 0)
                fragment.connections.left = index - 1;
            if (index % w != w - 1)
                fragment.connections.right = index + 1;
            if (index / w != 0)
                fragment.connections.up = index - w;
            if (index / w != h - 1)
                fragment.connections.down = index + w;
        }
        else {
            if (index < h * w)
            {
                if (index % w != 0)
                    fragment.connections.left = index - 1 + w * h;
                if (index / w != h - 1)
                    fragment.connections.down = index + w + w * h;
                fragment.connections.up = fragment.connections.right = index + w * h;
            }
            else
            {

```

```

        index -= w * h;
        if (index % w != w - 1)
            fragment.connections.right = index + 1;
        if (index / w != 0)
            fragment.connections.up = index - w;
        fragment.connections.left = fragment.connections.down = index;
    }
}
fragment.group = new FragmentGroup(fragment);
if (form == Picture.Form.Puzzle)
    (fragment.piece as PuzzlePiece).Truncate(fragment.connections.left != -
1,fragment.connections.right!=-1,fragment.connections.up!=-1,fragment.connections.down!=-1);
}

Fragment choosed;
private void button_Click(object sender,EventArgs args)
{
    Control panel = (sender as Control).Parent;
    Point p = panel.PointToClient(Cursor.Position);
    var controls = panel.Controls;
    Fragment
f=fragments.Where(z=>controls.Contains(z.piece)).OrderBy(z=>controls.GetChildIndex(z.piece)).FirstOrDefault(f
=>f.piece.HitTest(p),null);
    if (f == null) return;
    if (f.Fixed) return;
    if (panel == fieldPanel)
    {
        if (choosed == null)
        {
            choosed = f;
            f.piece.BringToFront();
            f.Choosed = true;
        }
        else
            if (choosed == f)
            {
                choosed = null;
                f.Choosed = false;
            }
        else
        {
            if (choosed.piece.Type == f.piece.Type)
            {
                choosed.Choosed = false;
                Swap(choosed, f);
                choosed = null;
            }
        }
    }
}
}
public List<T> Shuffle<T>(List<T> list)
{
    Random random = new Random();
    for (int i = list.Count - 1; i >= 1; i--)
    {
        int j = random.Next(0, i);
        (list[i], list[j]) = (list[j], list[i]);
    }
    return list;
}
private void playSound()
{
    if(Program.enableSound)

```

```

        SoundSystem.fragment.Play();
    }
    private void Swap(Fragment a, Fragment b)
    {
        (a.piece.Location, b.piece.Location) = (b.piece.Location, a.piece.Location);
        if (GetLocation(a.finalIndex).Equals(a.piece.Location))
        {
            a.Fix();
            playSound();
        }
        if (GetLocation(b.finalIndex).Equals(b.piece.Location))
        {
            b.Fix();
            playSound();
        }
        a.piece.Refresh();
        b.piece.Refresh();
        testWin();
    }
    private void testWin() {
        if (fragments.All(z => z.Fixed))
        {
            statistics.Stop();
            if(Program.enableSound)
                SoundSystem.win.Play();
            Program.SendCommand("postRecord", statistics.Score.ToString());
            new DialogForms.VictoryForm(statistics).ShowDialog();
            Dispose();
        }
    }

    private void tapePanel_Paint(object sender, PaintEventArgs e)
    {
        foreach (RowStyle row in tapePanel.RowStyles)
            row.Height = Height / rows;
    }
    bool isDragged;
    Point startMousePosition;
    Point startLocation;
    Control startPanel;
    private void button_MouseDown(object sender, MouseEventArgs e)
    {
        Control panel = (sender as Control).Parent;
        Point p = panel.PointToClient(Cursor.Position);
        var controls = panel.Controls;
        Fragment f = fragments.Where(z => controls.Contains(z.piece)).OrderBy(z =>
controls.GetChildIndex(z.piece)).FirstOrDefault(f => f.piece.HitTest(p), null);
        if (f == null) return;
        if (f.Fixed) return;
        if (e.Button == MouseButtons.Left)
        {
            isDragged = true;
            choosed = f;
            startPanel = panel;
            startLocation = f.piece.Location;
            int shiftY = 0;
            if(panel == tapePanel)
                shiftY=panel1.AutoScrollPosition.Y;
            Point pieceLocation = new Point(startLocation.X+panel.Left,startLocation.Y+ shiftY);
            if (tapePanel.Contains(choosed.piece))
            {
                tapePanel.Controls.Remove(choosed.piece);
            }
        }
    }

```

```

        CellsCount--;
        if (form == Picture.Form.Puzzle)
            (choosed.piece as PuzzlePiece).EnableResizing = false;
    }
    foreach (int c in choosed.group.fragments) {
        Fragment one=fragments[c];
        Controls.Add(one.piece);
        one.piece.BringToFront();
    }
    MoveGroup(f.group, new Point(pieceLocation.X - f.piece.Location.X, pieceLocation.Y -
f.piece.Location.Y));
    p = choosed.piece.PointToClient(Cursor.Position);
    startMousePosition = new Point(p.X * pieceSize.Width / choosed.piece.Width, p.Y *
pieceSize.Height / choosed.piece.Height);
    choosed.piece.Size = pieceSize;
}
else
{
    isDragged = false;
}
}

private void button_MouseMove(object sender, MouseEventArgs e)
{
    if (choosed == null) return;
    if (isDragged)
    {
        Point p=choosed.piece.PointToClient(Cursor.Position);
        Point offset = new (p.X-startMousePosition.X, p.Y-startMousePosition.Y);
        MoveGroup(choosed.group, offset);
    }
}
Fragment lastchoosed=null;
private void button1_MouseUp(object sender, MouseEventArgs e)
{
    if (choosed == null) return;
    isDragged = false;
    if (tapePanel.ClientRectangle.Contains(tapePanel.PointToClient(Cursor.Position)) &&
choosed.group.fragments.Count == 1)
    {
        CellsCount++;
        if (form==Picture.Form.Puzzle)
            (choosed.piece as PuzzlePiece).EnableResizing = true;
        tapePanel.Controls.Add(choosed.piece);
        choosed.piece.BringToFront();
    }
    else if (fieldPanel.ClientRectangle.Contains(fieldPanel.PointToClient(Cursor.Position)))
    {
        foreach (int c in choosed.group.fragments)
        {
            Fragment one = fragments[c];
            one.piece.Left -= fieldPanel.Location.X;
            one.piece.Top -= fieldPanel.Location.Y;
            fieldPanel.Controls.Add(one.piece);
            one.piece.BringToFront();
        }
        List<int> connections = new(choosed.group.connections);
        if(fieldPanel.Contains(choosed.piece))
        foreach (int c in connections)
            if (!choosed.group.fragments.Contains(c))
                if(fieldPanel.Contains(fragments[c].piece))

```



```

        MoveToConnect(fragments[c]);
        MoveToEdge();
    }
    else {
        foreach (int c in choosed.group.fragments) {
            Fragment one = fragments[c];
            if(startPanel==tapePanel)
                CellsCount++;
            if (form == Picture.Form.Puzzle)
                (one.piece as PuzzlePiece).EnableResizing = true;
            startPanel.Controls.Add(one.piece);
            one.piece.BringToFront();
        }
        MoveGroup(choosed.group, new Point(startLocation.X - choosed.piece.Left, startLocation.Y -
choosed.piece.Top));
    }
    lastchoosed = choosed;
    choosed = null;
}

void MoveToConnect(Fragment f) {
    int fIndex = choosed.finalIndex%(w*h);
    int sIndex = f.finalIndex % (w * h);
    int dx = sIndex % w - fIndex % w;
    int dy = sIndex / w - fIndex / w;
    Point first=GetLocation(dy * w + dx);
    first.Offset(choosed.piece.Location);
    Point second = f.piece.Location;
    if (Math.Abs(first.X - second.X) < delta.X)
        if (Math.Abs(first.Y - second.Y) < delta.Y)
        {
            MoveGroup(f.group, new Point(first.X-second.X, first.Y-second.Y));
            choosed.group.addGroup(f.group);
            playSound();
            foreach (int c in f.group.fragments.ToArray())
            {
                fragments[c].group = choosed.group;
                fragments[c].piece.BringToFront();
            }
        }
}

void MoveGroup(FragmentGroup group, Point offset)
{
    foreach (int c in group.fragments)
    {
        fragments[c].piece.Left += offset.X;
        fragments[c].piece.Top+=offset.Y;
    }
}

void MoveToEdge() {
    if (!choosed.group.fragments.Any(z => fragments[z].IsEdge())&& !choosed.group.fragments.Any(z
=> fragments[z].Fixed)) return;
    Point first = choosed.piece.Location;
    Point second = GetLocation(choosed.finalIndex);
    if (Math.Abs(first.X - second.X) < delta.X)
        if (Math.Abs(first.Y - second.Y) < delta.Y)
        {
            MoveGroup(choosed.group,new Point(second.X-first.X,second.Y-first.Y));
            playSound();
            foreach (int c in choosed.group.fragments)
                fragments[c].Fix();
        }
    testWin();
}

```

```

        Point GetLocation(int index) => new Point(index%(w*h) % w*pieceWidth, index % (w * h) /
w*pieceHeight);

private void GameControl_SizeChanged(object sender, EventArgs e)
{
    int widthThird = img.Width / w;
    int heightThird = img.Height / h;
    int imgWidth = widthThird;
    int imgHeight = heightThird;
    if (form == Picture.Form.Puzzle)
    {
        imgWidth = imgWidth * 3 / 2;
        imgHeight = imgHeight * 3 / 2;
    }
    pieceWidth = fieldPanel.Width / w;
    pieceHeight = fieldPanel.Height / h;

    pieceSize = form == Picture.Form.Puzzle ? new(pieceWidth * 3 / 2, pieceHeight * 3 / 2) :
new(pieceWidth, pieceHeight);
    delta = new(pieceWidth / 10, pieceHeight / 10);
    List<(Point point, int index)> locations = new();
    List<(Point point, int index)> secondLocations = new();
    for (int k = 0; k <= (form == Picture.Form.Triangular ? 1 : 0); k++)
        for (int i = 0; i < h; i++)
        {
            for (int j = 0; j < w; j++)
            {
                int index = i * w + j + k * h * w;
                bmp = new Bitmap(imgWidth, imgHeight);
                using (Graphics g = Graphics.FromImage(bmp))
                {
                    g.DrawImage(img, new Rectangle(0, 0, imgWidth, imgHeight), new Rectangle(j *
widthThird, i * heightThird, imgWidth, imgHeight), GraphicsUnit.Pixel);
                    AddPiece(index, getNewPiece(k));
                    if (k == 0)
                        locations.Add((GetLocation(index), index));
                }
            }
        }
    if (load) {
        foreach (int index in save.tape)
        {
            CellsCount++;
            tapePanel.Controls.Add(fragments[index].piece);
            if (form == Picture.Form.Puzzle)
                (fragments[index].piece as PuzzlePiece).EnableResizing = true;
        }
        foreach ((int index, int X, int Y) in save.field)
        {
            choosed = fragments[index];
            Piece piece = fragments[index].piece;
            fieldPanel.Controls.Add(piece);
            piece.Location = new Point(X, Y);
            piece.BringToFront();
            if (p.type != Picture.Type.Field)
            {
                List<int> connections = new(choosed.group.connections);
                foreach (int c in connections)
                {
                    if (!choosed.group.fragments.Contains(c))
                        if (fieldPanel.Contains(fragments[c].piece))
                            MoveToConnect(fragments[c]);
                }
                MoveToEdge();
            }
        }
    }
}

```

```

        else if (piece.Location.Equals(GetLocation(fragments[index].finalIndex)))
            fragments[index].Fix();
        choosed = null;
    }
}
else
if (type == Picture.Type.Field)
{
    if (form == Picture.Form.Triangular)
    {
        secondLocations.AddRange(locations);
        Shuffle(secondLocations);
        for (int j = 0; j < secondLocations.Count; j++)
        {
            Fragment f = fragments[j + w * h];
            fieldPanel.Controls.Add(f.piece);
            f.piece.Location = secondLocations[j].point;
        }
    }
    Shuffle(locations);
    for (int j = 0; j < locations.Count; j++)
    {
        Fragment f = fragments[j];
        fieldPanel.Controls.Add(f.piece);
        f.piece.Location = locations[j].point;
    }
}
else if (type == Picture.Type.Tape)
{
    foreach (Fragment f in Shuffle(new List<Fragment>(fragments)))
    {
        CellsCount++;
        if (form == Picture.Form.Puzzle)
            (f.piece as PuzzlePiece).EnableResizing = true;
        tapePanel.Controls.Add(f.piece);
    }
}
else
{
    foreach (Fragment f in fragments)
    {
        fieldPanel.Controls.Add(f.piece);
        Random random = new Random();
        int x = random.Next(pieceWidth);
        int y = random.Next(pieceHeight * (h - 1));
        f.piece.Location = new Point(x, y);
    }
}
if (statistics == null)
    statistics = new(this);
}

private Piece getNewPiece(int type) =>
    form switch
    {
        Picture.Form.Square => new Piece(),
        Picture.Form.Triangular => new TriangularPiece(type),
        Picture.Form.Puzzle => new PuzzlePiece(pieceWidth, pieceHeight),
        _ => new Piece()
    };

volatile Thread thread = null;
public void ShowImage() {

```

```

        statistics.imageTipTime.Start();
        if (tip_activated) return;
        tip_activated = true;
        imagePanel.Visible = true;
        imageButton.BackColor = Color.Lime;
    }
    public void HideImage()
    {
        thread = null;
        imagePanel.Visible = false;
        imageButton.BackColor = Color.White;
        statistics.imageTipTime.Stop();
        tip_activated = false;
    }
    private Piece tip_piece = null;
    private bool tip_chosedBefore=false;
    private bool tip_activated=false;
    public void ShowTip() {
        if (tipsCount <= 0) return;
        if (tip_activated) return;
        //tip_activated = true;
        if (choosed == null && lastchoosed == null) return;
        Fragment f = choosed ?? lastchoosed;
        if (type == Picture.Type.Field)
        {
            Point p = GetLocation(f.finalIndex);
            Swap(fragments.First(z => z.piece.Location.Equals(p)&&z.piece.Type==f.piece.Type), f);
            lastchoosed = choosed = null;
        }
        else {
            if (lastchoosed.piece.Parent == fieldPanel)
            {
                fieldPanel.Controls.Add(f.piece);
                Point p = GetLocation(f.finalIndex);
                Point p2 = f.piece.Location;
                Point offset = new(p.X - p2.X, p.Y - p2.Y);
                MoveGroup(f.group, offset);
                playSound();
                foreach (int c in f.group.fragments)
                    fragments[c].Fix();
                testWin();
                lastchoosed = null;
            }
        }
        statistics.UsedTipsCount++;
        tipsCount--;
        if (tipsCount == 0) tipButton.Visible = false;
    }
    public void HideTip()
    {
        if (tip_piece == null) return;
        thread = null;
        if (choosed == null && lastchoosed == null) return;
        Fragment f = choosed ?? lastchoosed;
        fieldPanel.Controls.Remove(tip_piece);
        tip_piece.Dispose();
        if (tipsCount == 0) tipButton.Visible = false;
        tipButton.BackColor = Color.White;
        f.Choosed = tip_chosedBefore;
        tip_activated = false;
    }
    private void imageButton_Click(object sender, EventArgs e)

```

```

    {
        ShowImage();
        thread = new Thread(() => {
            Thread.Sleep(3000);
            if (thread == null) return;
            Invoke(new Action(() => HideImage()));
        });
        thread.Start();
    }
    private void tipButton_Click(object sender, EventArgs e)
    {
        ShowTip();
    }
    public class Save {
        public List<(int index,int X, int Y)> field = new();
        public List<int> tape = new();
        public int tipsCount = 0;
        public Statictics statistics;
    }
    Save save = null;
    private void SaveGame() {
        Directory.CreateDirectory("saves");
        save = new();
        foreach (Fragment f in fragments.Where(z => tapePanel.Contains(z.piece)).OrderBy(z =>
tapePanel.Controls.GetChildIndex(z.piece)))
            save.tape.Add(f.finalIndex);
        foreach (Fragment f in fragments.Where(z => fieldPanel.Contains(z.piece)))
            save.field.Add((f.finalIndex, f.piece.Location.X, f.piece.Location.Y));
        save.tipsCount = tipsCount;
        save.statistics = statistics;
        File.Delete($"saves/{p.name}.sav");
        using (FileStream stream = File.OpenWrite($"saves/{p.name}.sav"))
            JsonSerializer.Serialize(stream,save, new JsonSerializerOptions { IncludeFields = true });
    }
    public void Exit(bool save=false) {
        thread=null;
        statistics.Stop();
        if(save)
            SaveGame();
        Dispose();
        Program.game = null;
    }
    public void LoadGame(Picture p)
    {
        using (FileStream stream = File.OpenRead($"saves/{p.name}.sav"))
            save=JsonSerializer.Deserialize<Save>(stream, new JsonSerializerOptions { IncludeFields = true
});

        File.Delete($"saves/{p.name}.sav");
        tipsCount = save.tipsCount;
        if (tipsCount == 0) tipButton.Visible = false;
        statistics = save.statistics;
        statistics.control = this;
        statistics.time.Start();
    }

    private void helpButton_Click(object sender, EventArgs e)
    {
        Program.Help();
    }
}
}

```